NEW HORIZON
COLLEGE OF ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

COURSE MATERIAL

Academic year: 2025-2026

Semester: 8

Course Code & Name: 22ECE824 & Quantum Computing

Course Coordinator: Mr. A.Narayana Kiran

Prepared by:

Mr. A.Narayana Kiran

Approved by:

Dr. Aravinda K
HOD-ECE

QUANTUM COMPUTING

Course Code	**22ECE824**	**CIE Marks**	**50**
L:T:P:S	**3:0:0:0**	**SEE Marks**	**50**
Hrs / Week	**3**	**Total Marks**	**100**
Credits	**3**	**Exam Hours**	**3**

Course outcomes: At the end of the course, the student will be able to:

22ECE824.1	Understand the fundamental concepts of quantum mechanics, quantum computing and quantum information Theory
22ECE824.2	Apply the concepts from linear algebra to model the quantum systems from a computational perspective
22ECE824.3	Analyze the fundamental differences between classical and quantum computational models
22ECE824.4	Construct quantum circuits using standard quantum gates and implement core quantum algorithms
22ECE824.5	Compare the principles of quantum information theory and apply it for real world use cases like quantum cryptography
22ECE824.6	Develop simple quantum machine learning model to solve real-world problems

Mapping of Course Outcomes to Program Outcomes and Program Specific Outcomes:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO 11	PO 12	PSO 1	PSO 2
22ECE824.1	2	-	-	-	-	-	-	-	-	-	-	-	3	2
22ECE824.2	3	-	-	-	-	-	-	-	-	-	-	-	3	2
22ECE824.3	3	3	-	-	-	-	-	-	-	-	-	-	3	2
22ECE824.4	3	3	2	-	3	-	-	-	-	-	-	2	3	2
22ECE824.5	3	3	2	-	-	1	1	-	-	-	-	2	3	2
22ECE824.6	3	3	2	2	3	-	-	-	-	3	-	2	3	2

MODULE-1	**Introduction to Quantum computing**	**22ECE824.1**	**8 Hours**

Need of Quantum computing, prehistory of Quantum computing, Quantum Principles, Superposition, entanglement, measurement, Quantum bits, multiple qubits, Blosch sphere mapping.

Self-study	Qubit Touchdown- A quantum computing board game.
Text Book	Text Book 1 - 2.2, 2.3,2.4, Text book 2 -1.2, Reference book 1-1.1,1.2,1.6.

MODULE-2	**Quantum Mechanics for computing**	**22ECE824.2**	**8 Hours**

Linear algebra: matrices, vectors, tensor products, The postulates of quantum mechanics, density operator, Schmidt decomposition and purifications, Bell inequality.

Self-study	Super dense coding
Text Book	Text Book 2 :2.1, 2.2,2.4,2.5,2.6

MODULE-3	**Quantum Gates and Algorithms**	**22ECE824.3, 22ECE824.4**	**8 Hours**

Quantum gates: X, Y, Z, H, S, T, CNOT, SWAP.
Quantum circuit design, Universal gates and circuit equivalence, Deutsch's algorithm, Deutsch–Jozsa algorithm, Grover's Search Algorithm, Quantum Fourier Transform (QFT),

Self- Study	Quantum teleportation

Text Book	Text Book 1:2,6. Text Book 2: 1.3, 4.5.1,4.5.2, 6.1.2, 5.1		
MODULE-4	**Quantum Information Theory**	**22ECE824.1 22ECE824.5**	**8 Hours**
Quantum Error Correction: Decoherence, Bit flip code, Phase flip code, Shor code. Quantum Information Theory: Von Neumann entropy, Quantum information over noisy quantum channels. Quantum Key Distribution: Encryption, Classical solution, Quantum solution.			
Case Study	Quantum Information Theory in Secure Communication.		
Text Book	Text Book 1:4.7, 6.6 Text Book 2:11.3,12.4.		
MODULE-5	**AI in Quantum Computing**	**22ECE824.6**	**8 Hours**
Quantum machine learning, Types in QML, Quantum K means clustering, variational Quantum circuits, QSVM.			
Case Study	Quantum Machine Learning for Handwritten Digit Recognition		
Text Book	Text Book 3: Page no: 32,313, 332,368		

CIE Assessment Pattern (50 Marks – Theory)

RBT Levels		Marks Distribution		
		Test (s)	AAT1	AAT2
		25	15	10
L1	Remember	5	-	5
L2	Understand	5	-	5
L3	Apply	10	10	-
L4	Analyze	5	5	-
L5	Evaluate	-	-	-
L6	Create	-	-	-

SEE Assessment Pattern (50 Marks – Theory)

RBT Levels		Exam Marks Distribution (50)
L1	Remember	10
L2	Understand	10
L3	Apply	20
L4	Analyze	10
L5	Evaluate	-
L6	Create	--

Suggested Learning Resources:

Text Books:

1. Thomas G Wong, Introduction to classical and Quantum computing,1st edition, Rooted Grove publisher, 2022, ISBN: 979-8-9855931-1-2.
2. Michael A Nielsen, Isaac L Chuang, Quantum Computation and Quantum Information, 10th edition, Cambridge University Press, 2010, ISBN 978-1-107-00217-3.
3. Santanu Ganguly, Quantum Machine Learning: An Applied Approach: The Theory and Application of Quantum Machine Learning in Science and Industry, 1st edition, Apress Publisher,2021, ISBN: 978-1-4842-7097-4

Reference Books:

1. Jozef Gruska, Quantum Computing, McGraw-Hill Book Co Ltd, 2000, ISBN 978-0077095031.
2. Phillip Kaye, An Introduction to Quantum Computing, Oxford University Press, 2007, ISBN 978-0-19-857049-3.

Web links and Video Lectures (e-Resources):

- https://nptel.ac.in/courses/106106232
- https://nptel.ac.in/courses/106106241
- https://www.youtube.com/watch?v=30U2DTfIrOU

Activity-Based Learning (Suggested Activities in Class)/ Practical Based learning

- Simulated QKD using dice or coins to visualize key exchange and eavesdropping detection.
- Implement quantum algorithms using Qiskit.
- Contents related activities (Activity-based discussions)
  - Problem solving worksheets
  - Interactive Game: "Qubit Touchdown"
  - Group Discussion

MODULE 1

Introduction to Quantum computing

Need of Quantum computing, prehistory of Quantum computing, Quantum Principles, Superposition, entanglement, measurement, Quantum bits, multiple qubits, Blosch sphere mapping.

1.1 Need of Quantum Computing?

Quantum computing is a big and growing challenge, for both science and technology. Computations based on quantum world phenomena, processes and laws offer radically new and very powerful possibilities and lead to different constraints than computations based on the laws of classical physics. Moreover, quantum computing seems to have the potential to deepen our understanding of Nature as well as to provide more powerful information processing and communication tools. At the same time the main theoretical concepts and principles of quantum mechanics that are needed to grasp the basic ideas, models and theoretical methods of quantum computing, are simple, elegant and powerful.

Quantum computing is without doubt one of the hottest topics at the current frontiers of computing, or even of the whole science. It sounds very attractive and looks very promising.

There are several natural basic questions to ask before we start to explore the concepts and principles as well as the mystery and potentials of quantum computing.

1. Why to consider quantum computing at all?

The development of classical computers is still making enormous progress and no end of that seems to be in sight. Moreover, the design of quantum computers seems to be very questionable and almost surely enormously expensive. All this is true. However, there are at least four very good reasons for exploring quantum computing as much as possible.

Quantum computing is a **challeng**e. A very fundamental and very natural challenge. Indeed, according to our current knowledge, our physical world is fundamentally quantum mechanical. All computers are physical devices and all real computations are physical processes. It is therefore a fundamental challenge, and actually our duty, to explore the potentials, laws and limitations of quantum mechanics to perform information processing and communication.

All classical computers and models of computers are based on classical physics (even if this is rarely mentioned explicitly), and therefore they are not fully adequate. There is nothing wrong with them, but they do not seem to explore fully the potential of the physical world for information processing. They are good and powerful, but they should not be seen as reflecting our full view of information processing systems.

Quantum computing seems to be a must and actually our **destiny**. As miniaturization of

computing devices continues, we are rapidly approaching the microscopic level, where the laws of the quantum world dominate.

Quantum computing is a **potentia**l. There are already results convincingly demonstrating that for some important practical problems quantum computers are theoretically exponentially more powerful than classical computers. Such results, such as Shor's factorization algorithm, can be seen as apt killers for quantum computing and have enormously increased activity in this area. In addition, the laws of the quantum world,harvested through quantum cryptography, can offer, in view of our current knowledge, unconditional security of communication, unachievable by classical means.

Finally, the development of quantum computing is a drive and gives new impetus to explore in more detail and from new points of view concepts, potentials, laws and limitations of the quantum world and to improve our knowledge of the natural world. The study of information processing laws, limitations and potentials is nowadays in general a powerful methodology to extend our knowledge, and this seems to be particularly true for quantum mechanics. Information is being identified as one of the basic and powerful concepts of physics and quantum entanglement is an important communication resource.

2. Can quantum computers do what classical ones cannot?

The answer depends on the point of view. It can be YES. Indeed, the simplest example is generation of random numbers. Quantum algorithms can generate truly random numbers. Deterministic algorithms can generate only pseudo-random numbers. Other examples come from the simulation of quantum phenomena. On the other hand, the answer can also be NO. A classical computer can produce truly random numbers when attached to a proper physical source.

3. Where lie the differences between classical and quantum information processing?

Classical information can be read, transcribed (into any medium), duplicated at will, transmitted and broadcasted. Quantum information, on the other hand, cannot be in general read or duplicated without being disturbed, but it can be “teleported”. In classical randomized computing, a computer always selects one of the possible computation paths, according to a source of randomness, and “what-could-happen-but-did-not”has no influence whatsoever on the outcome of the computation. On the other hand, in quantum computing, exponentially many computational paths can be taken simultaneously in a single piece of hardware and in a special quantum way and “what-could-happen-but did-not” can really matter.

Acquiring information about a quantum system can inevitably disturb the state of the system. The tradeoff between acquiring quantum information and creating a disturbance of the

system is due to quantum randomness. The outcome of a quantum measurement has a random element and because of that we are unable to always faithfully infer the (initial) state of the system from the measurement outcome.

Perhaps the main difference between classical and quantum information processing lies in the fact that quantum information can be encoded in mutual correlations between remote parts of physical systems and quantum information processing can make essential use of this phenomena—called entanglement—not available for classical information processing.

Another big difference between the classical and quantum worlds that strongly influences quantum information processing stems from the fact that the relationship between a system and its subsystems is different in the quantum world than in the classical world. For example, the states of a quantum system composed of quantum subsystems cannot be in general decomposed into states of these subsystems.

4. Can quantum computers solve some practically important problems much more efficiently?

Yes. For example, integer factorization can be done in polynomial time on quantum computers which seems to be impossible on classical computers. Searching in an unordered database can be done probably with less queries on quantum computers.

5. Where does the power of quantum computing come from?

On one side, quantum computation offers enormous parallelism. The size of the computational state space is exponential in the physical size of the system and the energy available. A quantum bit can be in any of a potentially infinite number of states and quantum systems can be simultaneously in superposition of exponentially many of the basis states. A linear number of operations can create an exponentially large superposition of states and, in parallel, an exponentially large number of operations can be performed in one step.

Secondly, it is the branching and quantum interference that create parallel computation and constructive/destructive superpositions of states and can amplify or destroy the impacts of some computations. Due to this fact, we can, in spite of the peculiarities of quantum measurements, utilize quantum parallelism.

Thirdly, it is mainly the existence of so-called “entangled states” that makes quantum computing more powerful than classical and allows even very distant parts of systems to be strongly tied. This creates a base for developing and exploring quantum teleportation and other phenomena that are outside of the realm of the classical world.

6. Where are the drawbacks and bottlenecks of quantum computing?

Quantum computing can provide enormous parallelism. However, there are also enormous problems with harnessing the power of its parallelism. According to the basic principles of quantum mechanics, a (projection) measurement process can get out of (large) quantum superposition only one classical result, randomly chosen, and the remaining quantum information can be irreversibly destroyed.

An interaction of a quantum system with its environment can lead to the so-called decoherence effects and can greatly influence, or even completely destroy, subtle quantum interference mechanisms. This appears to make long reliable quantum computations practically impossible.

Differences between classical and quantum computing

Key points	classical computing	quantum computing
Basis of computing	Large scale multipurpose computer based on classical physics.	High speed computer based on quantum mechanics.
Information storage	Bit-based information storage using voltage/charge.	Quantum bit-based information storage using electron spin or polarization.
Bit values	Bits having a value of either 0 or 1 can have a single value at any instant.	Qubits have a value of 0, 1 or sometimes linear combination of both, (a property known as superposition).
Number of possible states	The number of possible states is 2 which is either 0 or 1.	The number of possible states is infinite since it can hold combinations of 0 or 1 along with some complex information.
Output	Deterministic (repetition of computation on the same input gives the same output)	Probabilistic (repetition of computation on superposed states gives probabilistic answer)
Gates used for processing	Logic gates (AND, OR, NOT, etc.)	Quantum gates (X, Y, Z, H, CNOT etc.)
Operations	Operations use Boolean Algebra.	Operations use linear algebra and are represented with unitary matrices.
Circuit implementation	Circuit implemented in macroscopic technologies	Circuits implemented in microscopic technologies.
Data processing	Data processing is carried out by logic and in sequential order.	Data processing is carried out by quantum logic at parallel instances.

1.2 Prehistory of Quantum computing

Since 1945 we have been witnessing a rapid growth of the raw performance of computers with respect to their speed and memory size. An important step in this development was the invention of transistors, which already use some quantum effects in their operation. However, it is clear that if such an increase in performance of computers continues, then after 50 years, our chips will have to contain 1016 gates and operate at a 1014 Hz clock rate (thus delivering 1030 logic operations per second)10. It seems that the only way to achieve that is to learn to build computers directly out of the laws of quantum physics.

In order to come up seriously with the idea of quantum information processing, and to develop it so far and so fast, it has been necessary to overcome several intellectual barriers.

The most basic one concerned an important feature of quantum physics—reversibility. None of the known models of universal computers was reversible. This barrier was overcome first by the existence of universal reversible Turing machines, and then by the existence of universal classical reversible gates.

The second intellectual barrier was overcome by showing that quantum mechanical computational processes can be at least as powerful as classical computational processes. He did that by showing how a quantum system can simulate actions of the classical reversible Turing machines. However, his “quantum computer” was not fully quantum yet and could not outperform classical ones. The overcoming of these basic intellectual barriers had significant and broad consequences. Relations between physics and computation started to be investigated on a more general and deeper level. This has also been due to the fact that reversibility results implied the theoretical possibility of zero-energy computations.

The third intellectual barrier that had to be overcome was a lack of a proper model for a universal quantum computing device capable of simulating effectively any other quantum computer. The first step to overcome this barrier was done by Deutsch (1985) who elaborated Feynman’s ideas and developed a (theoretically) physically realisable model of quantum computers, a quantum physical analogue of a probabilistic Turing machine, which makes full use of the quantum superposition principle, and on any given input produces a random sample from a probability distribution. Deutsch conjectured that it might be more efficient than a classical Turing machine for certain computations. He also showed the existence of a universal quantum Turing machine (that could consequently simulate any physical process and experiment) and also a model of quantum networks—a quantum analog of classical sequential logical circuits.

In parallel with the development of the basic models of quantum computing an effort was put into overcoming the fourth intellectual barrier. Can quantum computing be really more powerful

than classical computing? Are there some good reasons to assume that quantum computing could bring an essential (exponential) speed-up of computations for at least some important information processing problems? This was a very important issue because it was clear that any design of a quantum computer would require overcoming a number of large scientific and engineering barriers and therefore it was needed to know whether the proposed model of quantum computer offers, at least theoretically, any substantial benefit over the classical computers.

Historical Highlights

1980s

In the 1980s, physicists Richard Feynman and Paul Benioff observed that simulating quantum systems on classical computers did not work well. They had the idea that a different kind of a computer, based on quantum principles, could perform these simulations better. Further, they suggested that such a quantum computer could speed up computation for other problems standardly solved using classical computers.David Deutsch gave the first formal model of a quantum computer in 1985. After the development of the first formal model, researchers found input-output relational tasks for which they developed quantum algorithms obtaining polynomial and exponential speedups. However, these initial tasks were contrived and were not interesting problems in their own right. Even so, quantum computing's potential was demonstrated in cryptographic tasks. Secure cryptographic key distribution, where an eavesdropper cannot observe a message without irreversibly altering it, was shown to be possible in the quantum setting. This cannot be done securely in the classical setting.

1990s

In the 1990s, researchers began finding interesting and practical computational tasks with quantum speedup.In 1995, Lov Grover showed how to search for an element in an unordered list of n elements using $O(\sqrt{n})$ queries. 1 In the classical and randomized settings, we require $\Theta(n)$ queries in the worst case. Grover's algorithm uses a technique called amplitude amplification. As in classical computation, where there are important algorithmic paradigms like divide-and-conquer and dynamic programming, there are important quantum algorithmic paradigms, and amplitude amplification is such an example. Applying Grover's algorithm to exhaustive search for a satisfying assignment to an n-variable Boolean formula yields a $O(^{2n/2})$ algorithm for a satisfiability, while we don't know of any such algorithms in the classical setting. However, finding a subexponential algorithm for satisfiability is an open problem even on quantum computers. This makes Grover's

algorithm an example of quantum advantage rather than quantum supremacy; the quantum algorithm's runtime is still exponential, but it is a better exponential than the best classical algorithms' runtimes.

In 1994, Peter Shor used a technique called phase estimation to develop an $O(n^2)$ polynomial time quantum algorithm for factoring integers and for finding discrete logarithms. Today's best known classical algorithms for these problems have exponential running times, taking $O(^{2\Theta(\sqrt{3}\ n)})$ time. An important implication of this result is that public-key cryptosystems like RSA can theoretically be broken in polynomial time using a quantum computer, since breaking RSA only requires the prime factorization of a large integer. However, quantum computers breaking RSA is not an imminent threat since the largest integer that existing quantum computers can factored is 21.

2000s

In the 2000s, researchers developed a new paradigm for quantum computation called quantum walks, a variant of classical random walks. Quantum walks are closely related to the simulation of physical quantum systems, a problem known as Hamiltonian simulation Much research in quantum algorithms in the 2000s focused on efficient algorithms for NPintermediate problems, problems which are known to be in NP, but are not known to be NPcomplete and are not known to be in P. Examples of NP-intermediate problems are factoring, the discrete logarithm problem, the approximate shortest vector problem in lattices, and graph isomorphism. Note that lattice problems underlie many cryptosystems. However, whether there are efficient quantum algorithms to solve lattice problems is still an open question, so these cryptosystems are not immediately threatened by quantum computers. Another direction of research was post-quantum cryptography. Crucially, there are some classical cryptosystems that remain secure in quantum settings. This means that the cryptosystems can be run using classical computers but would still be safe against quantum adversaries.

2010s

Several important advancements in quantum algorithms came out of the 2010s. Using phase estimation (a technique from Shor's algorithm) and Hamiltonian simulation / quantum walks, Harrow, Hassidim, and Lloyd developed the HHL algorithm, an algorithm for solving well-conditioned, sparse systems of linear equations. Sparsity means that every row in the system of equations contains only a small number of non-zero coefficients. This is realistic in many engineering settings. Well-conditioned means that if you perturb the input system of equations slightly, the exact solution does not change significantly.

The best known classical algorithm for solving an N ×N system of equations with at most s non-zero entities per row takes $O(^{N2\ s})$. The HHL algorithm takes only $O(^{\log N)s})$ time. At first glance this seems impossible; if we can solve a system of equations in less than N time, then we have not even looked at all of the input coefficients. However, this seeming paradox is explained because – like we saw with Grover's algorithm – the HHL algorithm uses quantum queries rather than classical ones. When solving the equation Ax = b, this algorithm requires b to be in superposition and requires "quantum access" to A. Similarly, the output only provides "quantum access" to the solution x. The idea of "quantum access" will be formalized at a later point.

The area of quantum approximate optimization algorithms (QAOA) was inspired by the HHL algorithm. QAOAs are subject to the same caveats about quantum access to inputs and outputs. QAOAs have applications in machine learning. Several of the quantum machine learning algorithms have subsequently been dequantized, meaning that people came up with classical algorithms with comparable efficiencies.

Another area of interest is sampling problems, in which we want to sample an object from a specific distribution or a class of distributions. A simple example would be sampling a binary string from the uniform distribution over strings of length n. It's thought that it may be easier to establish quantum supremacy by looking at sampling problems because there may be very simple quantum algorithms without the need for heavy error correction that can generate distributions which are hard to generate classically. Indeed, Google's 2019 claim about quantum supremacy was based on random quantum circuit sampling, a problem which is likely classically hard.

1.3 Quantum Principles

In order to explain the basic principles of quantum computing, and to develop quantum algorithms and networks, it is not necessary to explore why the strange behaviour of quantum systems appears. It is sufficient to use some basic empirically- based principles of quantum behaviour.In order to formulate these principles we need to introduce some basic concepts: events, states, amplitudes of events, evolution, compound quantum systems and measurement.

Concept of bit and qubit and its properties of quibits:

Bit: A digital computer stores and processes information using bits which can be either 0 or 1. Physically, a bit can be anything that has two distinct configurations: one represented by "0", and the other represented by "1". It could be a system with two distinct and distinguishable possibilities. In modern computing and communications, bits are represented by the absence or presence of an electrical signal, encoding "0" and "1" respectively.

States and amplitudes

A **quantum state** is a complete description of a quantum system. By an event of a quantum experiment (system, process) we understand a pair

(final state, initial state).

For example, an event in the experiment with electrons, is that an electron leaves a source *s* and arrives at the position *x*.

One of the main goals of quantum mechanics is to predict whether events happen or not, and if they do, then with which probability.

The amplitude of an event with the initial state IS and a final state FS is usually denoted by using Dirac's bra-ket notation.

$$\langle FS \mid IS \rangle,$$

A quantum state represents complete information about a system and is written as $|\psi\rangle$.

$$\mathbf{|\psi\rangle = \alpha|0\rangle + \beta|1\rangle}$$

Probability Amplitude

The probability of an event is obtained from the square of the amplitude.

$$\mathbf{p = |\alpha|^2}$$

Superposition

A system can exist in multiple states simultaneously until measured.

$$\mathbf{|\psi\rangle = \alpha|0\rangle + \beta|1\rangle}$$

$|0\rangle$ ----

>--- $|\psi\rangle$

$|1\rangle$ ----

Measurement

Measurement collapses the quantum state into one of the basis states.

Probability of $|0\rangle = |\alpha|^2$, Probability of $|1\rangle = |\beta|^2$

Hilbert Space

Quantum states exist in a vector space called Hilbert space.

Evolution of Quantum Systems

Quantum evolution is governed by unitary transformations.

$$|\psi(t)\rangle = U(t)|\psi(0)\rangle$$

Schrödinger Equation

Describes how quantum states change over time.

$$i\hbar\,(\partial|\psi(t)\rangle/\partial t) = H|\psi(t)\rangle$$

Unitary Operators

Unitary operators preserve probability and are reversible.

$$U^{\dagger}U = I$$

Tensor Product (Compound Systems)

Multiple quantum systems combine using tensor products.

$$H = H_1 \otimes H_2$$

System A $\otimes$ System B $\rightarrow$ Combined System

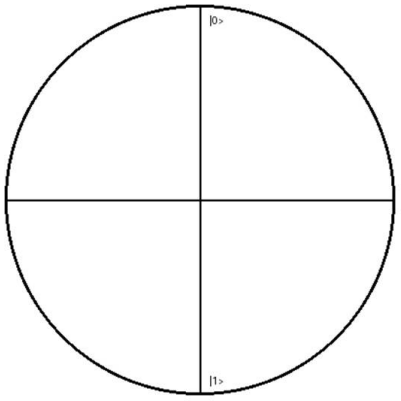
1.4 Superposition

A qubit can exist in multiple states at once. This is called superposition. Unlike classical bits, quantum systems allow linear combinations of basis states. This enables parallel computation and forms the foundation of quantum algorithms.

At the quantum level the particle experiences a difficult to understand and very peculiar phenomenon, like superposition. A quantum particle can exist in multiple states simultaneously

before the measurement. The feature of a quantum system is it exists in several separate quantum states at the same time. For example, electrons possess a type of intrinsic angular momentum called spin. In the presence of a magnetic field the electron can exist in two possible spin states, like spin up and spin down. Before the measurement the electron has a finite chance to exist in either up spin or down spin. Only after the measurement the electron can possess a specific spin state.Superposition plays a crucial role in quantum communication and quantum computing.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$



In superposition, α and β are complex amplitudes. Their squared magnitudes represent probabilities. The relative phase between α and β changes the state behavior and its position on the Bloch sphere.

$$|\alpha|^2 + |\beta|^2 = 1$$

The fundamental building block of classical computational devices is the two-state system. We typically think of these systems as being built from bits which can take values in $\{0, 1\}$. Quantum mechanics, on the other hand, tells us that any such system can exist in a superposition of states. A *qubit* is a quantum system in which the Boolean states 0 and 1 are represented[1] by a prescribed pair of normalized and mutually orthogonal quantum states labeled as $\{|0\rangle , |1\rangle\}$.

More specifically, a qubit is defined as a 2-dimensional Hilbert space H_2 and we label an orthonormal basis of H_2 by $\{|0\rangle , |1\rangle\}$. The state of the qubit is an associated unit length vector in H_2. If a state is a basis vector then we say it is a *pure* state or *superposition*. If a state is any other linear combination of states we say it is a *mixed* state[2]. Note that unlike pure states, mixed states cannot be written as a wave function and instead are described by a density matrix.

If a system can exist in multiple states, then a superposition of states is a sum of states with definite phase and amplitude, represented by complex coefficients. One the other hand, a mixed state is represented by a sum of probabilities for finding the system in each state. The key difference is that probabilities have no phase information and hence cannot interfere in the way that complex amplitudes can. Also the terminology is important. The probability of finding a system in a specific state requires calculating an expectation value, which is a measurement. On the other hand a superposition is a quantum state that evolves according to Schr¨odinger's equation

The key difference between a mixed state and a superposition of pure states lies in the latter being able to give rise to quantum interference . If you make a measurement of a mixed state, you just get the various probabilities. However, if the state is in a superposition, any measurement will give you apparent probabilities that depend on the phase of the quantum state, and so can change either with different points in time or space, which is characteristic of interference.

The state of a qubit bit, $|\psi\rangle$, is described by

$$|\psi\rangle = \alpha\,|0\rangle + \beta|\,|1\rangle$$

where $\alpha, \beta \in \mathbb{C}$, $|0\rangle =$

$$\begin{bmatrix}1\\0\end{bmatrix}, |1\rangle = \begin{bmatrix}0\\1\end{bmatrix} \text{ and } |\alpha|^2 + |\beta|^2 = 1.$$

Here $|x|^2 = xx^*$, where x^* is the complex conjugate of x [8]. The complex plane is shown in Figure

1.5 Measurement

When a qubit is measured, it collapses into one of the basis states. The probability of each outcome depends on amplitudes. Measurement is irreversible and destroys superposition.

$$P(0)=|\alpha|^2,\ P(1)=|\beta|^2$$

Measurement Bases

Measurement can be performed in different bases such as Z, X, and Y. Each basis gives different results for the same state, highlighting the probabilistic nature of quantum mechanics.

Suppose we measure state $|\varphi\rangle$. We cannot "see" a superposition itself, but only classical states. Accordingly, if we measure state $|\varphi\rangle$ we will see one and only one classical state $|j\rangle$. Which specific $|j\rangle$ will we see? This is not determined in advance; the only thing we can say is that we will see state $|j$ with probability $|\alpha_j|^2$, which is the squared norm of the corresponding amplitude α_j. This is known as "Born's rule." Accordingly, observing a quantum state induces a probability distribution on the classical states, given by the squared norms of the amplitudes.

1.6 Entanglement

Entanglement occurs when two qubits become correlated such that the state of one instantly determines the state of the other, regardless of distance. This is a key resource in quantum computing.

A pair or group of particles is entangled when the quantum state of each particle cannot be described independently of the quantum state of the other particle. The quantum state of the system as a whole can be described. For example, two particles are created in such a way that the total spin of the system is zero. If the spin of one of the particles is measured on a certain axis and found to be anticlockwise, then it is guaranteed that a measurement of the spin of the other particle along the same axis will show the spin to be clockwise. In classical circuit theory, the controlled-not gate, or XOR, creates correlations between the two input bits. A target input bit value of 0 will come out with the same value as the control bit, 0 or 1. What is unique for the corresponding quantum gate (CNOT) is that it also accepts superposition states, allowing to perform calculations on qubits. In quantum computation this effect can be used to create entanglement. Quantum entanglement is a crucial element in the establishment of the entangled network structure of the quantum Internet. In the quantum Internet, the quantum nodes share quantum entanglement among one another, which provides an entangled ground-base network structure for the various quantum networking protocols.

The important fact about the entanglement is that, once entangled, the particles preserve the quantum states even when they are separated wide apart. This allows a communication engineer to exploit the property well and use it to improve security and other aspects of communication. There are four important maximally entangled states in the two-qubit space, called the Bell-states,shown in Figure. which are created from a product state using the circuit.

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$
$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$
$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$$
$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

$$|\psi\rangle = 1/\sqrt{2}\ (|00\rangle + |11\rangle)$$

Properties of Entanglement

Entangled states cannot be separated into individual qubit states. Measurement of one qubit affects the other. This leads to non-classical correlations used in teleportation and cryptography.

1.7 Quantum bits

The bit is the fundamental concept of classical computation and classical information. Quantum computation and quantum information are built upon an analogous concept, the quantum bit,orqubit for short.

What is a qubit? Just as a classical bit has a state– either 0 or 1– a qubit also has a state. Two possible states for a qubit are the states |0> and |1> , which as you might guess corresponds to the states 0 and 1 for a classical bit. Notation like '| > ' is called the Dirac notation, and we'll be seeing it often, as it's the standard notation for states in quantum mechanics. The difference between bits and qubits is that a qubit can be in a state other than |0> or |1>.It is also possible to form linear

combinations of states, often called superpositions:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle .$$

The numbers α and β are complex numbers, although for many purposes not much is lost by thinking of them as real numbers.

The special states |0> and |1> are known as computational basis states, and form an orthonormal basis for this vector space.

We can examine a bit to determine whether it is in the state 0 or 1. For example, computers do this all the time when they retrieve the contents of their memory. Rather remarkably, we cannot examine a qubit to determine its quantum state, that is, the values of α and β. Instead, quantum mechanics tells us that we can only acquire much more restricted information about the quantum state. When we measure a qubit we get either the result 0, with probability |α|2, or the result 1, with probability |β|2. Naturally, |α|2 +|β|2 = 1, since the probabilities must sum to one. Geometrically, we can interpret this as the condition that the qubit's state be normalized to length 1. Thus, in general a qubit's state is a unit vector in a two-dimensional complex vector space.

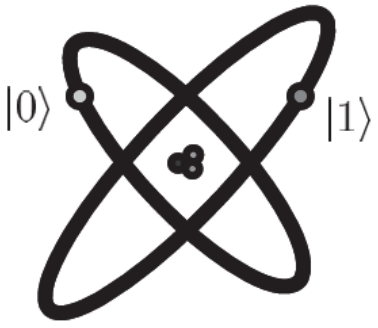


Figure: Qubit represented by two electronic levels in an atom

Qubit is the physical carrier of quantum information. It is the quantum version of a bit, and its quantum state can be written in terms of two levels, labelled $|0\rangle$ and $|1\rangle$.

$|\ \rangle$ this notation is known as 'ket' notation and $\langle\ |$ is known as 'brac' notation. Both are together called as Dirac notations 'Ket' are analogous to a column vector. They are also called basis vectors and represented by two-dimensional column vectors as follows

$$|\mathbf{0}\rangle = \begin{bmatrix}\mathbf{1}\\ \mathbf{0}\end{bmatrix} \text{ and } \quad |\mathbf{1}\rangle = \begin{bmatrix}\mathbf{0}\\ \mathbf{1}\end{bmatrix}$$

The qubit can be in any one of the two states as well as in the superposed state simultaneously. In quantum computation two distinguishable states of a system are needed to represent a bit of data.

For example, two states of an electron orbiting a single atom. Spin up is taken as $|1\rangle$ and spin down is taken as $|0\rangle$. Similarly ground state energy level is $|0\rangle$ and excited state level is $|1\rangle$

Superposition of two states:

The difference between qubits and classical bits is that a qubit can be in a linear combination (superposition) of the two states $|0\rangle$ and $|1\rangle$.

For ex, if $\propto$ and β are the probability amplitudes of electron in ground state (ie, in $|0\rangle$ state) and in excited state (ie, in $|1\rangle$ state) then the linear combination of two states is

$$|\psi\rangle = \alpha\,|0\rangle + \beta|1\rangle$$

The numbers α and β are complex but due to normalization conditions

$$|\propto|^2 + |\beta|^2 = 1.$$

Here $|\propto|^2$ is the probability of finding $|\psi\rangle$ in state $|0\rangle$ and

$|\beta|^2$ is the probability of finding $|\psi\rangle$ in state $|1\rangle$.

So, that when a qubit is measured, it only gives either '0' or '1' as the measurement result – probabilistically.

Consider the following example of qubit representation

$$|\Psi\rangle = \left(\frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\right)|0\rangle + \left(\frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\right)|1\rangle$$

$$\therefore \propto = \frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\ and\ \ \beta = \frac{\mathbf{1}}{\sqrt{\mathbf{2}}}$$

$$|\propto|^2 = |\beta|^2 = 1/2$$

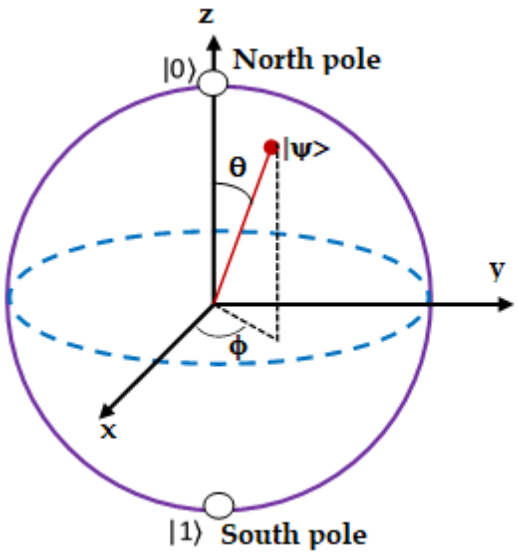
This means that with 50% probability the qubit will be found in $|0\rangle$ state as well as in $|1\rangle$ state. The superposed states are also called as space states where as $|0\rangle$ and $|1\rangle$ are called basis states.

Properties of qubits

1. Qubits make use of discrete energy state particles such as electrons and photons.
2. Qubits exists in two quantum state $|0\rangle$ and $|1\rangle$ or in a linear combination of both states. This is known as superposition.
3. Unlike classical bits, qubit can work with the overlap of both 0 & 1 states.
   For ex, a 4-bit register can store one number from 0 to 15 (because of $2^n = 2^4=16$), but 4-qubit register can store all 16 numbers.
4. When the qubit is measured, it collapses to one of the two basis states $|0\rangle$ or $|1\rangle$
5. Quantum entanglement and quantum tunnelling are two exclusive properties of qubit.
6. State of the qubits is represented using Bloch sphere.

Representation of Qubits by Bloch Sphere: (single qubit state)

Bloch sphere is an imaginary sphere which is used to represent pure single-qubit states as a point on its surface. It has unit radius. Its North Pole and South Pole are selected to represent the basis states namely $|0\rangle$ and $|1\rangle$. North Pole represents $|0\rangle$ (say spin up↑) and South Pole represents $|1\rangle$ (say spin down ↓). All other points on the sphere represent superposed states (ie, state space). Bloch sphere allows the state of a qubit to be represented in spherical coordinates (ie, r, θ and ϕ). It is as follows



The state qubit $|\psi\rangle$ on the Bloch sphere makes an angle θ with z-axis and its projection (azimuth) makes angle ϕ with x-axis as shown. It is clear from the fig that $0 < \theta < \pi$ and $0 < \phi < 2\pi$.

$|\psi\rangle$ is represented as

$$|\psi\rangle = \alpha\,|0\rangle + \beta|1\rangle$$

It can be proved that

$$|\Psi\rangle = cos\,(\theta/2)\,|0\rangle + e^{i\phi} sin\,(\theta/2)\,|1\rangle \quad ---(1)$$

Using this equation we can represent $|\psi\rangle$ for different θ and ϕ as follows

Case-1: let $\theta = 0$ and $\phi = 0$, then eq (1) becomes

$$|\Psi\rangle = cos\ 0|0\rangle + e^{i0}\ sin0|1\rangle = |0\rangle + 0$$

$$\therefore |\Psi\rangle = |0\rangle$$

Case-2: let $\theta = \pi$ and $\phi = 0$, then eq (1) becomes

$$|\Psi\rangle = cos\,(\pi/2)|0\rangle + e^{i0}\ sin\,(\pi/2)|1\rangle = 0 + |1\rangle$$

$$\therefore |\Psi\rangle = |1\rangle$$

Case-3: let $\theta = \pi/2$ and $\phi = 0$, then eq (1) becomes

$$|\Psi\rangle = cos\,(\pi/4)|0\rangle + e^{i0}\ sin\,(\pi/4)|1\rangle$$

$$|\Psi\rangle = \left(\frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\right)|0\rangle + \left(\frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\right)|1\rangle$$

$$|\Psi\rangle = \left(\frac{|0\rangle+|1\rangle}{\sqrt{2}}\right)$$

Case-4: let $\theta = \pi/2$ and $\phi = \pi$, then eq (1) becomes

$$|\Psi\rangle = cos\,(\pi/4)\,|0\rangle + e^{i\pi} sin\,(\pi/4)\,|1\rangle$$

$$|\Psi\rangle = \left(\frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\right)|0\rangle - \left(\frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\right)|1\rangle$$

$$|\Psi\rangle = \left(\frac{|0\rangle-|1\rangle}{\sqrt{2}}\right)$$

In the above discussion we have represented only single qubit state. Bloch sphere is a nice visualization of single qubit states.

Representation of Multiple Qubits (Two qubits and Extension to N qubits):

Two qubits:

Consider two qubits. They can be in any one of four possible states represented as $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$.

$$|00\rangle=\begin{bmatrix}1\\0\\0\\0\end{bmatrix} \quad |01\rangle=\begin{bmatrix}0\\1\\0\\0\end{bmatrix} \quad |10\rangle=\begin{bmatrix}0\\0\\1\\0\end{bmatrix} \quad |11\rangle=\begin{bmatrix}0\\0\\0\\1\end{bmatrix}$$

The state qubit is (ie, linear combination of these four)

$$|\Psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle$$

For 2 qubit system we have 4 complex amplitudes namely α_{00}, α_{01}, α_{10} and α_{11}. According to normalization condition

$$|\alpha_{00}|^2 + |\alpha_{01}|^2 + |\alpha_{10}|^2 + |\alpha_{11}|^2 = 1$$

Similarly if there are 3 qubits there will be 8 complex amplitudes.

In general for N qubits, there will be having 2^N complex amplitudes. This means that a basis state is represented by a number 0 to 2^{N-1}.

The superposition state is represented as

$$|\Psi\rangle = \sum_{x=0}^{2^{N-1}} \alpha_x |x\rangle$$

Qubit has two quantum states similar to the classical binary states. The qubit can be in any one of the two states as well as in the superposed state simultaneously.

Dirac representation and Matrix operations:

In Quantum mechanics, Bra-Ket notation is a standard notation for describing quantum states. The notation $|\ \rangle$ is known as 'ket' notation and $\langle\ |$ is known as 'bra' notation. Both are together called as Dirac notations.

Matrix representation of 0 and 1 states:

Consider a quantum state ψ in a vector space represented by $|\psi\rangle$. The notation $|\rangle$ indicates that the object is a vector and is called a ket vector. The examples of such ket vectors are like $|\psi\rangle$, $|\phi\rangle$ and $|u\rangle$ etc.

The wave function could be expressed in ket notation as $|\psi\rangle$ (ket Vector), ψ is the wave function.

Hence, any arbitrary state can be represented as

$$|\psi\rangle = \begin{bmatrix}\alpha\\\beta\end{bmatrix} \quad \text{or} \quad |\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

The 'ket' vector typically represented as a column vector and 'bra' vector typically represented as a row vector. The matrix for of the states $|0\rangle$ and $|1\rangle$ as follows;

$|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}$ $|1\rangle = \begin{bmatrix}0\\1\end{bmatrix}$ ket notations

$\langle 0| = [1\ 0]$ $\langle 1| = [0\ 1]$ – – – – – bra notations

Operators and matrices:

An operator is a mathematical rule that transform a given function into another function.

Consider an operator 'A' transforms the vector $|a\rangle$ to another vector $|b\rangle$, it can be written as

$$\hat{A}|a\rangle = |b\rangle$$

There are different types of operators like Linear operator, Identity operator, Null operator, Inverse operator, Singular & non-singular operator etc.

Identity operator I :

The identity operator is an operator which, operating on a function, leaves the function unchanged i.e. $I\ |a\rangle = |a\rangle$ It is given in matrix form by

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

This is also called as identity matrix. There will be no change when I operate on either $|0\rangle$ state or $|1\rangle$ state. It is explained as follows

$$I|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\therefore I|0\rangle = |0\rangle$$

Similarly

$$I|1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\therefore I|1\rangle = |1\rangle$$

Identity matrix acts as number 1. It is always a square matrix.

Conjugate matrices:

If the elements in a matrix A are complex numbers, then the matrix obtained by the corresponding conjugate complex elements is called the conjugate of A and is denoted by $A*$.

For example

if $A = \begin{bmatrix} 0 & i \\ -i & 0 \end{bmatrix}$ then $A^* = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$

if $A = \begin{bmatrix} 1 & i \\ -i & 1 \end{bmatrix}$ then $A^* = \begin{bmatrix} 1 & -i \\ i & 1 \end{bmatrix}$

Transpose matrices

If columns and rows of a matrix A are interchanged then the resultant matrix is transpose of A and represented as A^T .

For example ,

if $A = \begin{bmatrix} 0 & 1 \\ -i & 0 \end{bmatrix}$ then $A^* = \begin{bmatrix} 0 & -i \\ 1 & 0 \end{bmatrix}$

if $A = \begin{bmatrix} 1 & 2i \\ 4i+1 & 0 \end{bmatrix}$ then $A^* = \begin{bmatrix} 1 & 4i+1 \\ 2i & 0 \end{bmatrix}$

Hermitian matrices:

The transpose of complex conjugate of a matrix is known as Hermitian operator and the resultant matrix is known as Hermitian matrix. It is represented by $A^\dagger$

Let A be a matrix, A* be its complex conjugate and A^{*T} is its transpose then its Hermitian matrix is $A^\dagger = A^{*T}$

if $A = \begin{bmatrix} 1 & 2i \\ 4i+1 & 0 \end{bmatrix}$ $A^* = \begin{bmatrix} 1 & -2i \\ -4i+1 & 0 \end{bmatrix}$ then $A^\dagger = \begin{bmatrix} 1 & -4i+1 \\ -2i & 0 \end{bmatrix}$

Unitary matrices:

Matrix A is said to be unitary if it produces an identity matrix I when multiplied by its conjugate transpose $AA^\dagger = I$

In other words, A is a unitary matrix if its conjugate transpose is equal to its reciprocal, ie

$$A^\dagger = \frac{I}{A} = \frac{1}{A} = A^{-1}$$

we can show that $A = \frac{1}{2}\begin{bmatrix} 1+i & 1-i \\ 1-i & 1+i \end{bmatrix}$

Column and Row Matrices and their inner product:

The Column Vectors are called ket Vectors denoted by $|\psi\rangle$ and are represented by Column Matrices.

The Row Vectors are called Bra Vectors denoted by $\langle\phi|$ and are represented by Row Matrices.

Let us consider a ket vector represented in the form of a column matrix.

$$|\psi\rangle = \begin{bmatrix} \alpha_1 \\ \beta_1 \end{bmatrix}$$

The Row Matrix is represented as

$$\langle\psi| = [\alpha^* \quad \beta^*]$$

Here, $\begin{bmatrix} \alpha_1 \\ \beta_1 \end{bmatrix}^\dagger = [\alpha^* \quad \beta^*]$

Thus the Bra is the complex conjugate of ket and vice versa.

Inner product: The inner product of two vectors U and V in the complex space is a function that takes U and V as inputs and produces a complex number as output.

In terms of Dirac notation, the inner product is given as $\langle U|V\rangle$ = C

In matrix form U and V are written a

$$|\mathbf{U}\rangle = \begin{bmatrix}\mathbf{x_1}\\ \mathbf{y_1}\end{bmatrix} \text{and} \quad |\mathbf{V}\rangle = \begin{bmatrix}\mathbf{x_2}\\ \mathbf{y_2}\end{bmatrix}$$

Their inner product is written as $\langle U|V\rangle$, but $\langle U|$ is equal to conjugate transpose of $|U\rangle$

$ie,\ \langle U| = |U^*\rangle^{-1} = |U\rangle^{\dagger} = [x_1{}^*\ y_1{}^*]$

$\therefore \langle U|V\rangle = [x_1{}^*\ y_1{}^*]\begin{bmatrix}x_2\\ y_2\end{bmatrix} = x_1{}^*x_2 + y_1{}^*y_2$

The square root of the inner product of a vector with itself is also called as norm or the length of the vector. It is given by $|U| = \sqrt{\langle U|U\rangle}$

Ex: Find the inner product of

$$|\mathbf{U}\rangle = \begin{bmatrix}\mathbf{3+i}\\ \mathbf{4-i}\end{bmatrix} \text{and} \quad |\mathbf{V}\rangle = \begin{bmatrix}\mathbf{3i}\\ \mathbf{4}\end{bmatrix}$$

First we shall find the conjugate transpose of $|U\rangle$

$$|U^*\rangle = \begin{bmatrix}3-i\\ 4+i\end{bmatrix}$$

$$|U\rangle^{\dagger} = [3-i \quad 4+i]$$

$$\therefore \langle U| = |U\rangle^{\dagger} = [3-i \quad 4+i]$$

$$\langle U|V\rangle = [3-i \quad 4+i]\begin{bmatrix}3i\\ 4\end{bmatrix}$$

$$\langle U|V\rangle = (3-i) * 3i + * (4+i)4$$

$$\langle U|V\rangle = 9i + 3 + 16 + 4i$$

$$\langle U|V\rangle = 13i + 19$$

Orthogonality:

If the inner product of two vectors is equal to 0 then they are said to be orthogonal (or perpendicular) to each other.

If $\langle U|V\rangle$ = 0 then $|U\rangle$ and $|V\rangle$ are perpendicular.

Consider,

$$|\mathbf{0}\rangle = \begin{bmatrix}\mathbf{1}\\ \mathbf{0}\end{bmatrix} \text{ and } \quad |\mathbf{1}\rangle = \begin{bmatrix}\mathbf{0}\\ \mathbf{1}\end{bmatrix}$$

$$\langle 0|1\rangle = [\mathbf{1} \quad \mathbf{0}]\begin{bmatrix}\mathbf{0}\\ \mathbf{1}\end{bmatrix} = 0$$

Hence $|0\rangle$ is perpendicular to $|1\rangle$

The most important property of the inner product of a vector with itself is equal to one ie, $\langle\psi|\psi\rangle = 1$

This is known as normalization condition. The physical significance of normalization is that the "probability amplitude" of the quantum system

Orthonormality: If each element of a set of vectors is normalized and the elements are orthogonal with respect to each other, we say the set is orthonormal.

Consider the set $|\mathbf{0}\rangle = \begin{bmatrix}\mathbf{1}\\\mathbf{0}\end{bmatrix}$ and $|\mathbf{1}\rangle = \begin{bmatrix}\mathbf{0}\\\mathbf{1}\end{bmatrix}$

$\langle 0|0\rangle = [\mathbf{1} \quad \mathbf{0}]\begin{bmatrix}\mathbf{1}\\\mathbf{0}\end{bmatrix} = 1$ normalized

$\langle 0|1\rangle = [\mathbf{1} \quad \mathbf{0}]\begin{bmatrix}\mathbf{0}\\\mathbf{1}\end{bmatrix} = 0$ orthogonal

$\langle 1|1\rangle = [\mathbf{0} \quad \mathbf{1}]\begin{bmatrix}\mathbf{0}\\\mathbf{1}\end{bmatrix} = 1$ normalized

$\langle 1|0\rangle = [\mathbf{0} \quad \mathbf{1}]\begin{bmatrix}\mathbf{1}\\\mathbf{0}\end{bmatrix} = 0$ orthogonal

Hence set of $|0\rangle$ and $|1\rangle$ is orthonormal.

From the above relations, if states $|\psi\rangle$ and $|\phi\rangle$ are said to be orthonormal if

1. $|\psi\rangle$ and $|\phi\rangle$ are normalized.
2. $|\psi\rangle$ and $|\phi\rangle$ are orthogonal to each other.

Probability

Let us consider a Quantum State

$|\psi\rangle = \alpha\ |0\rangle + \beta\ |1\rangle$

$$|\psi\rangle = \alpha\begin{bmatrix}1\\0\end{bmatrix} + \beta\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}\alpha\\\beta\end{bmatrix}$$

The inner product $\langle\psi|\psi\rangle$ is given by

$$\langle\psi|\psi\rangle = [\alpha^* \quad \beta^*]\begin{bmatrix}\alpha\\\beta\end{bmatrix} = \alpha^*\alpha + \beta^*\beta$$

$$\alpha^*\alpha + \beta^*\beta = |\alpha|^2 + |\beta|^2$$

This could also be written as $|\psi|^2 = \psi\ \psi^*$

Thus the above equation represents Probability Density. As per the principle of Normalization

$$|\psi|^2 = \psi\ \psi^* = \langle\psi|\psi\rangle = 1 = |\alpha|^2 + |\beta|^2$$

Thus it implies $|\psi\rangle$ is normalized.

Pauli Matrices:

These are the 2 × 2 complex matrices introduced by Pauli in order to account for the interaction of the spin with an external electromagnetic field. They are given by

$$\sigma_1 = \sigma_x = X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\sigma_2 = \sigma_y = Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$\sigma_3 = \sigma_z = Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Properties of Pauli matrices:

- Square Pauli matrices gives identity matrix I

$$X^2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

  Similarly,

$$Y^2 = I \quad \text{and} \quad Z^2 = I$$

- Pauli matrices are unitary matrix.

$$X X^\dagger = 1, \; Y Y^\dagger = 1 \quad \text{and} \quad Z Z^\dagger = 1$$

- Pauli matrices are Hermitian:

  Let A be a matrix, A* be its complex conjugate and $A^\dagger$ is its transpose. If A = $A^\dagger$ then the matrix is Hermitian. $Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$

$$Y^* = \begin{bmatrix} 0 & i \\ -i & 0 \end{bmatrix}$$

$$Y^\dagger = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$Y = Y^\dagger$$

Operation of Pauli Matrices on 0 and 1 states

Three Pauli matrices X, Y and Z operates on states $|0\rangle$ and $|1\rangle$ as follows

- **X operating on $|0\rangle$ and $|1\rangle$**

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |1\rangle$$

$$X|0\rangle = |1\rangle$$

$$X|1\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

$$X|1\rangle = |0\rangle$$

Since X inverts each input (ie, $|0\rangle$ becomes $|1\rangle$ and $|1\rangle$ becomes $|0\rangle$).

It is also called as bit-flip gate.

If a superposed qubit goes through X gate, the result will be

$$X|\psi\rangle = \begin{bmatrix}0 & 1\\ 1 & 0\end{bmatrix}\begin{bmatrix}\alpha\\ \beta\end{bmatrix} = \begin{bmatrix}\beta\\ \alpha\end{bmatrix} = \alpha|1\rangle + \beta|0\rangle$$

$$X|\psi\rangle = \alpha|1\rangle + \beta|0\rangle$$

- **Y operating on $|0\rangle$ and $|1\rangle$**

$$Y|0\rangle = \begin{bmatrix}0 & -i\\ i & 0\end{bmatrix}\begin{bmatrix}1\\ 0\end{bmatrix} = \begin{bmatrix}0+0\\ i+0\end{bmatrix} = \begin{bmatrix}0\\ i\end{bmatrix} = i\begin{bmatrix}0\\ 1\end{bmatrix} = i|1\rangle$$

$$Y|0\rangle = i|1\rangle$$

$$Y|1\rangle = \begin{bmatrix}0 & -i\\ i & 0\end{bmatrix}\begin{bmatrix}0\\ 1\end{bmatrix} = \begin{bmatrix}0-i\\ 0+0\end{bmatrix} = \begin{bmatrix}-i\\ 0\end{bmatrix} = -i\begin{bmatrix}1\\ 0\end{bmatrix} = -i|0\rangle$$

$$Y|1\rangle = -i|0\rangle$$

If a superposed qubit goes through Y gate, the result will be

$$Y|\psi\rangle = \begin{bmatrix}0 & -i\\ i & 0\end{bmatrix}\begin{bmatrix}\alpha\\ \beta\end{bmatrix} = \begin{bmatrix}-i\beta\\ i\alpha\end{bmatrix} = -i\beta|0\rangle + i\alpha|1\rangle$$

$$Y|\psi\rangle = -i\beta|0\rangle + i\alpha|1\rangle$$

- **Z operating on $|0\rangle$ and $|1\rangle$**

$$Z|0\rangle = \begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}\begin{bmatrix}1\\ 0\end{bmatrix} = \begin{bmatrix}1+0\\ 0+0\end{bmatrix} = \begin{bmatrix}1\\ 0\end{bmatrix} = \begin{bmatrix}1\\ 0\end{bmatrix} = |0\rangle$$

$$Z|0\rangle = |0\rangle$$

$$Z|1\rangle = \begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}\begin{bmatrix}0\\ 1\end{bmatrix} = \begin{bmatrix}0+0\\ 0-1\end{bmatrix} = \begin{bmatrix}0\\ -1\end{bmatrix} = -1\begin{bmatrix}0\\ 1\end{bmatrix} = -|1\rangle$$

$$Z|1\rangle = -|1\rangle$$

If a superposed qubit goes through Y gate, the result will be

$$Z|\psi\rangle = \begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}\begin{bmatrix}\alpha\\ \beta\end{bmatrix} = \begin{bmatrix}\alpha\\ -\beta\end{bmatrix} = \alpha|0\rangle - \beta|1\rangle$$

$$Z|\psi\rangle = \alpha|0\rangle - \beta|1\rangle$$

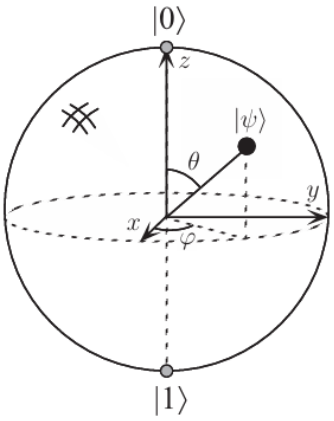


Figure: Bloch sphere representation of a qubit.

1.8 Multiple qubits

Consider we have two qubits. If these were two classical bits, then there would be four possible states, 00, 01, 10, and 11. Correspondingly, a two qubit system has four computational basis states denoted |00> ,|01> ,|10> ,|11> . A pair of qubits can also exist in superpositions of these four states, so the quantum state of two qubits involves associating a complex coefficient– sometimes called an amplitude– with each computational basis state, such that the state vector describing the two qubits is

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle.$$

1.9 Bloch Sphere Mapping

The Bloch sphere represents a qubit as a point on a sphere. The north pole is |0⟩ and south pole is |1⟩. Superposition states lie on the surface.

and

$$
\begin{aligned}
&z_- : \theta = \pi, \phi = 0 \longrightarrow \\
&\cos\frac{\pi}{2}\,|0\rangle + e^{i0}\sin\frac{\pi}{2}\,|1\rangle \\
&= \cos 0\,|0\rangle + (1)\sin\frac{\pi}{2}\,|1\rangle \\
&= 0\,|0\rangle + 1\,|1\rangle \\
&= |1\rangle
\end{aligned}
$$

So we can write the state of a qubit in many different ways. Perhaps the most generic way to write $|\psi\rangle$ is

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

The maximally superimposed state is

$$|\psi\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle$$

Here the probability of measuring $|0\rangle$ is $|a|^2 = aa^* = \left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2}$. Similarly, the probability of measuring $|1\rangle$ is $\frac{1}{2}$.

Now, note that

$$|\psi\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{e^{i\theta}}{\sqrt{2}}\,|1\rangle$$

Why does the second equality in Equation 3 hold? Well, consider $\left|e^{i\theta}\right|^2$. By Euler's formula[3] we can see that

$$
\begin{aligned}
\left|e^{i\theta}\right|^2 &= |(\cos\theta + i\sin\theta)|^2 \\
&= (\cos\theta + i\sin\theta)(\cos\theta - i\sin\theta) \\
&= \cos^2\theta - \cos\theta\; i\sin\theta + i\sin\theta\cos\theta - i^2\sin^2\theta \\
&= \cos^2\theta - i^2\sin^2\theta = \cos^2\theta + \sin^2\theta = 1
\end{aligned}
$$

So $\left|e^{i\theta}\right|^2 = 1$. Now, dividing both sides by 2 and taking the square roots gives

$$\sqrt{\frac{\left|e^{i\theta}\right|^2}{2}} = \sqrt{\frac{1}{2}} \implies \frac{e^{i\theta}}{\sqrt{2}} = \frac{1}{\sqrt{2}} \tag{4}$$

So $|\psi\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{e^{i\theta}}{\sqrt{2}}\,|1\rangle$. This implies that the statistics of any measurements we could perform on the state $e^{i\theta}\,|\psi\rangle$ would be exactly the same as they would be for the state $|\psi\rangle$.

We have seen that measuring $|\psi\rangle$ in the standard basis ($\{|0\rangle, |1\rangle\}$) yields a probability of $\frac{1}{2}$ for measuring either $|0\rangle$ or $|1\rangle$. Is there any measurement that yields information about the phase θ?

Consider a measurement in a different basis, the $\{|+\rangle, |-\rangle\}$ basis. As we saw above, $|+\rangle \equiv \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|-\rangle \equiv \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. What does $|\psi\rangle$ look like in this new basis? The basic approach is to first represent $|0\rangle$ and $|1\rangle$ in the new basis. Here $|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$ and $|1\rangle = \frac{1}{\sqrt{2}}(|+\rangle - |-\rangle)$. Then

$$\begin{aligned}
|\psi\rangle &= \frac{1}{\sqrt{2}}|0\rangle + \frac{e^{i\theta}}{\sqrt{2}}|1\rangle \\
&= \frac{1}{\sqrt{2}}\Big(\frac{1}{\sqrt{2}}\big(|+\rangle + |-\rangle\big) + \frac{e^{i\theta}}{\sqrt{2}}\big(|+\rangle - |-\rangle\big)\Big) \\
&= \frac{1}{2}\big(|+\rangle + |-\rangle\big) + \frac{e^{i\theta}}{2}\big(|+\rangle - |-\rangle\big) \\
&= \frac{1+e^{i\theta}}{2}|+\rangle + \frac{1-e^{i\theta}}{2}|-\rangle
\end{aligned}$$

Since $e^{i\theta} = \cos\theta + i\sin\theta$ we can write

$$= \frac{\big(1+\cos\theta+i\sin\theta\big)}{2}|+\rangle + \frac{\big(1-(\cos\theta+i\sin\theta)\big)}{2}|-\rangle$$

What then is the probability of measuring, say, $|+\rangle$, in the $\{|+\rangle, |-\rangle\}$ basis? Well, we know that the probability is the amplitude squared, so,

$$\begin{aligned}
P(|+\rangle) &= \left|\frac{\big(1+\cos\theta+i\sin\theta\big)}{2}\right|^2 \\
&= \frac{1}{4}\big((1+\cos\theta+i\sin\theta)(1+\cos\theta-i\sin\theta)\big) \\
&= \frac{1}{4}\big(1+\cos\theta-i\sin\theta+\cos\theta+\cos^2\theta-\cos\theta i\sin\theta+i\sin\theta+\cos\theta i\sin\theta-i^2\sin^2\theta\big) \\
&= \frac{1}{4}\big(1+2\cos\theta+\cos^2\theta+\sin^2\theta\big) && \#\ \cos^2\theta+\sin^2\theta = 1 \\
&= \frac{1}{4}\big(1+2\cos\theta+1\big) \\
&= \frac{1}{4}\big(2+2\cos\theta\big) \\
&= \frac{1}{2}\big(1+\cos\theta\big) && \#\ \cos\frac{\theta}{2} = \pm\sqrt{\frac{1+\cos\theta}{2}} \implies 1+\cos\theta = 2\cdot\cos^2\frac{\theta}{2} \\
&= \frac{1}{2}\Big(2\cdot\cos^2\frac{\theta}{2}\Big) \\
&= \cos^2\frac{\theta}{2} && \#\ P(|+\rangle) = \cos^2\frac{\theta}{2}
\end{aligned}$$

Similarly, we know that the probability of measuring $|-\rangle$, $P(|-\rangle) = \sin^2\frac{\theta}{2}$. So measuring the qubit in the $\{|+\rangle, |-\rangle\}$ basis does reveal some information about the phase θ.

$$
\begin{aligned}
&x_+ : \theta = \frac{\pi}{2}, \phi = 0 \longrightarrow \\
&\cos\frac{\pi}{4}\,|0\rangle + e^{i\cdot 0}\sin\frac{\pi}{4}\,|1\rangle && \#\ e^{i\cdot 0} = e^0 = 1 \\
&= \frac{1}{\sqrt{2}}\,|0\rangle + (1)\frac{1}{\sqrt{2}}\,|1\rangle \\
&= \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle && \#\ H\,|0\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle \\
&\equiv |+\rangle
\end{aligned}
$$

$$
\begin{aligned}
&x_- : \theta = \frac{\pi}{2}, \phi = \pi \longrightarrow \\
&\cos\frac{\pi}{4}\,|0\rangle + e^{i\pi}\sin\frac{\pi}{4}\,|1\rangle && \#\ \text{Euler's Identity [11]: } e^{i\pi} + 1 = 0,\ e^{i\pi} = -1 \\
&= \frac{1}{\sqrt{2}}\,|0\rangle + (-1)\frac{1}{\sqrt{2}}\,|1\rangle \\
&= \frac{1}{\sqrt{2}}\,|0\rangle - \frac{1}{\sqrt{2}}\,|1\rangle && \#\ H\,|1\rangle = \frac{1}{\sqrt{2}}\,|0\rangle - \frac{1}{\sqrt{2}}\,|1\rangle \\
&\equiv |-\rangle
\end{aligned}
$$

On the y axis of the Bloch Sphere we can see that

$$
\begin{aligned}
&y_+ : \theta = \frac{\pi}{2}, \phi = \frac{\pi}{2} \longrightarrow \\
&\cos\frac{\pi}{4}\,|0\rangle + e^{i\frac{\pi}{2}}\sin\frac{\pi}{4}\,|1\rangle && \#\ \text{Euler's formula} \quad : e^{i\frac{\pi}{2}} = \cos\frac{\pi}{2} + i\sin\frac{\pi}{2} = 0 + i\cdot 1 = i \\
&= \frac{1}{\sqrt{2}}\,|0\rangle + i\frac{1}{\sqrt{2}}\,|1\rangle \\
&= \frac{1}{\sqrt{2}}\,|0\rangle + \frac{i}{\sqrt{2}}\,|1\rangle
\end{aligned}
$$

$$
\begin{aligned}
&y_- : \theta = \frac{\pi}{2}, \phi = \frac{3\pi}{2} \longrightarrow \\
&\cos\frac{\pi}{4}\,|0\rangle + e^{i\frac{3\pi}{2}}\sin\frac{\pi}{4}\,|1\rangle && \#\ e^{i\frac{3\pi}{2}} = \cos\frac{3\pi}{2} + i\sin\frac{3\pi}{2} = 0 - i\cdot 1 = -i \\
&= \frac{1}{\sqrt{2}}\,|0\rangle - i\frac{1}{\sqrt{2}}\,|1\rangle \\
&= \frac{1}{\sqrt{2}}\,|0\rangle - \frac{i}{\sqrt{2}}\,|1\rangle
\end{aligned}
$$

Finally, on the z axis of the Bloch Sphere we can see that

$$
\begin{aligned}
&z_+ : \theta = 0, \phi = 0 \longrightarrow \\
&\cos\frac{0}{2}\,|0\rangle + e^{i0}\sin\frac{0}{2}\,|1\rangle \\
&= \cos 0\,|0\rangle + (1)\sin 0\,|1\rangle \\
&= 1\,|0\rangle + 0\,|1\rangle \\
&= |0\rangle
\end{aligned}
$$

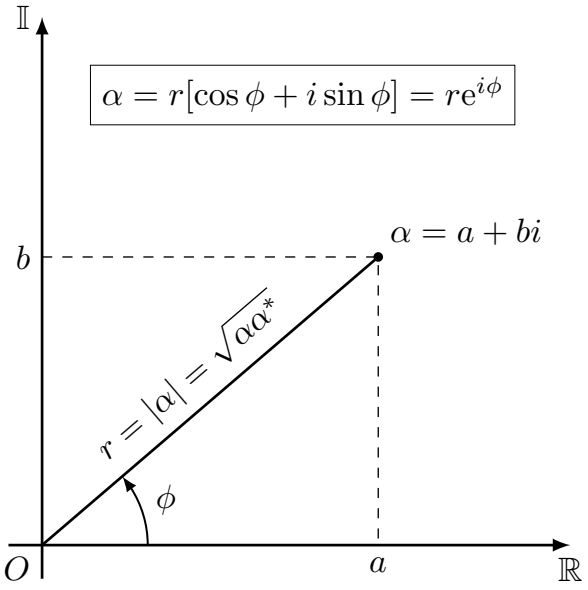


Figure : α n the Complex Plane

Why does θ vary between 0 and π in Equation 1?

In the complex plane representation of $|\psi\rangle$, the basis vectors $|0\rangle$ and $|1\rangle$ are orthogonal, that is, there is a $\frac{\pi}{2}$ angle between them and hence their inner product $\langle 0|1\rangle = 0$. In the Bloch Sphere the angle θ between $|0\rangle$ and $|1\rangle$ is π. That is, in the Bloch Sphere $|0\rangle$ and $|1\rangle$ are *antipodal* rather than orthogonal.

Further, in the general form of $|\psi\rangle$, , α and β are known as *probability amplitudes* [3] and thus $0 \leq |\alpha| \leq 1$ (likewise for β). However, in the Bloch Sphere we have $0 \leq \theta \leq \pi$. So to keep the values of the probability amplitudes in the right range ($0 \leq |\alpha| \leq 1$) we need to divide θ by 2 (I'll just note here that there are many other ways to think about this). This is one reason that we see the angle $\frac{\theta}{2}$ in Equation 1. Having $|0\rangle$ and $|1\rangle$ represented on the same axis (z) also saves a dimension in the Bloch Sphere representation. We will see further implications of $|0\rangle$ and $|1\rangle$ being antipodal rather than orthogonal in Section 3.

Deriving Equation :

Let's take a look at why Equation looks the way it does. First, remember that the general state of a qubit is

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

where $\alpha, \beta \in \mathbb{C}$ and $\langle\psi|\psi\rangle = |\alpha|^2 + |\beta|^2 = 1$. $|\alpha|^2 + |\beta|^2 = 1$ is known as a *normalization constraint*. Here $\langle\psi|\psi\rangle$ is the inner product of ψ with itself, namely $\begin{bmatrix}\alpha & \beta\end{bmatrix}\begin{bmatrix}\alpha \\ \beta\end{bmatrix} = |\alpha|^2 + |\beta|^2$. In general for two $n \times 1$ vectors $\mathbf{u}$ and $\mathbf{v}$, the inner product $\langle\mathbf{u}|\mathbf{v}\rangle$ is defined as

$$\langle\mathbf{u}|\mathbf{v}\rangle = \mathbf{u}^{\mathrm{T}}\mathbf{v}$$

What can we say about α and β?

Since α and β are complex numbers, we can write α (or β, with different a and b) as $\alpha = a + ib$. This is depicted in Figure . There are a few things we can notice about α. First, $a = r\cos\phi$; similarly, $b = r\sin\phi$. So we can write

Consider some physical system that can be in N different, mutually exclusive classical states. Because we will typically start counting from 0 in these notes, we call these states $|0\rangle, |1\rangle, \ldots, |N{-}1\rangle$. Roughly, by a "classical" state we mean a state in which the system can be found if we observe it. A *pure quantum state* (usually just called *state*) $|\phi\rangle$ is a *superposition* of classical states, written

$$|\phi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle + \cdots + \alpha_{N-1}|N-1\rangle.$$

Here α_i is a complex number that is called the *amplitude* of $|i\rangle$ in $|\phi\rangle$. Intuitively, a system in quantum state $|\phi\rangle$ is "in *all* classical states *at the same time*," each state having a certain amplitude. It is in state $|0\rangle$ with amplitude α_0, in state $|1\rangle$ with amplitude α_1, and so on. Mathematically, the states $|0\rangle, \ldots, |N-1\rangle$ form an orthonormal basis of an N-dimensional *Hilbert space* (i.e., an N-dimensional vector space equipped with an inner product). A quantum state $|\phi\rangle$ is a vector in this space, usually written as an N-dimensional column vector of its amplitudes:

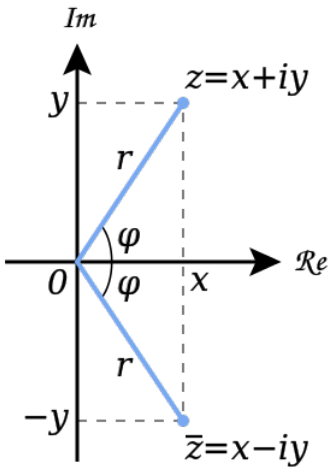
$$|\phi\rangle = \begin{pmatrix} \alpha_0 \\ \vdots \\ \alpha_{N-1} \end{pmatrix}.$$

Such a vector is sometimes called a "ket." It conjugate transpose is the following row vector, sometimes called a "bra":

$$\langle\phi| = \left(\alpha_0^*, \ldots, \alpha_{N-1}^*\right).$$

The reason for this terminology (often called "Dirac notation" after Paul Dirac) is that an inner product $\langle\phi|\psi\rangle$ between two states corresponds to the dot product between a bra and a ket vector ("bracket"): $\langle\phi|\psi\rangle = \langle\phi| \cdot |\psi\rangle$.

We can combine different Hilbert spaces using tensor product: if $|0\rangle, \ldots, |N-1\rangle$ are an orthonormal basis of space $\mathcal{H}_A$ and $|0\rangle, \ldots, |M-1\rangle$ are an orthonormal basis of space $\mathcal{H}_B$, then the tensor product space $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B$ is an NM-dimensional space spanned by the set of states $\{|i\rangle \otimes |j\rangle \mid i \in \{0, \ldots, N-1\}, j \in \{0, \ldots, M-1\}\}$. An arbitrary state in $\mathcal{H}$ is of the form $\sum_{i=0}^{N-1} \sum_{j=0}^{M-1} \alpha_{ij}|i\rangle \otimes |j\rangle$. Such a state is called *bipartite*. Similarly we can have *tripartite* states that "live" in a Hilbert space that is the tensor product of three smaller Hilbert spaces, etc.



The Complex Plane

More generally, a *quantum system* is an n-dimensional Hilbert Space H_n. We label an orthonormal basis on H_n by $\{|x_1\rangle, |x_2\rangle, \ldots, |x_n\rangle\}$ such that $x_i \in X$ for some finite X. The associated state of the system is a unit length vector in H_n. If the state S is $|\phi\rangle$ at some moment in time, then we say that S has state $|\phi\rangle$ or S is in state $|\phi\rangle$.

We can also see that a general form for the state of a quantum system is

$$\sum_{i=1}^{n} \alpha_i |x_i\rangle \text{ where } \alpha_i \in \mathbb{C} \text{ and } \sum_{i=1}^{n} |\alpha_i|^2 = 1$$

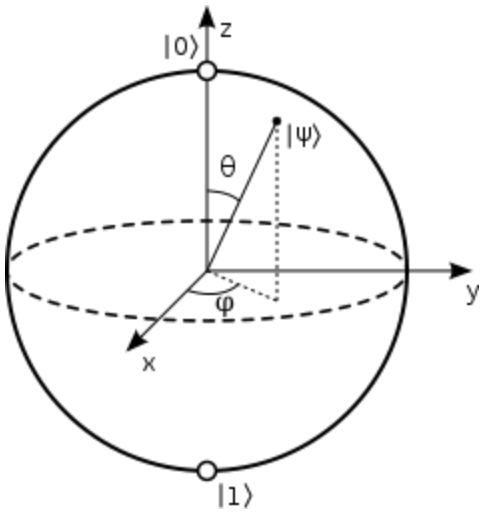
Remark: We can already see an important technical problem that must be solved if quantum computers are ever to be practical. A superposition of states $|0\rangle$ and $|1\rangle$ is known as a *coherent* state. A coherent state is extremely "unstable"; it inevitably interacts with its environment and collapses into a *pure* state (see discussion below). This process is known as *decoherence*. Algorithms that exploit quantum effects such as superposition are known as quantum algorithms. To apply these quantum algorithms in the real world, the *decoherence time* must be longer than the time to run the algorithm. As a result methods for increasing the decoherence time are of great interest (see, for examp

The Bloch Sphere

One question that we might ask, given the complex plane representation of the state of a qubit, is "how does the real, 3-dimensional space in which we live correspond to the 2-dimensional representation of a qubit's state (as represented in the complex plane)"? The *Bloch Sphere* gives us an elegant way to think about this question.

The Bloch Sphere, shown in Figure , is named for the physicist Felix Bloch and gives us a beautiful way to think about and visualize the state of a qubit in 3-dimensional space.

In the Bloch Sphere the *pure* state of a qubit $|\psi\rangle$ is represented as a point on the surface of the sphere. *Mixed* states are represented as points in the interior of the sphere. Somewhat surprisingly, specification of a point on the Bloch Sphere requires only two parameters, θ and ϕ. This itself is remarkable; you might imagine that describing a vector in 3-dimensional space might take three or more parameters.



The Bloch Sphere

The representation of a classical bits on the Bloch Sphere is given by the poles of the sphere. The representation of the probabilistic classical bit, that is, a bit that is 0 with probability p and 1 with probability $1-p$, is given by the point in z-axis with coordinate $2p-1$. The interior of the Bloch Sphere is used to describe the states of a qubit in the presence of decoherence.

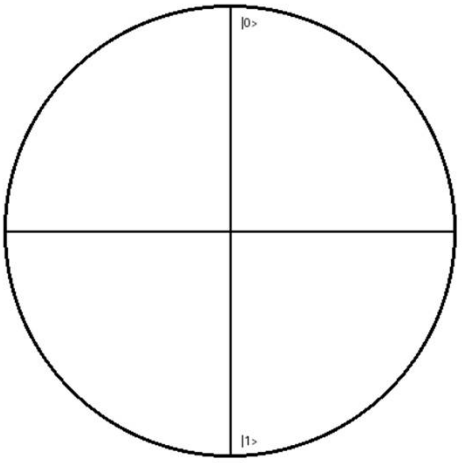
The standard way to write the state of a qubit using the Bloch Sphere is

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle$$

where $0 \leq \theta \leq \pi$ and $0 \leq \phi \leq 2\pi$. See Section 2.3 for a derivation of this equation.

Bloch Sphere Coordinate System

Figure _ shows the coordinate system of the Bloch Sphere. Consider the x axis. Here we can see that



Bloch Sphere Angles

A general qubit can be written using spherical coordinates θ and φ. These determine the position of the qubit on the sphere.

$$|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\varphi} \sin(\theta/2)|1\rangle$$

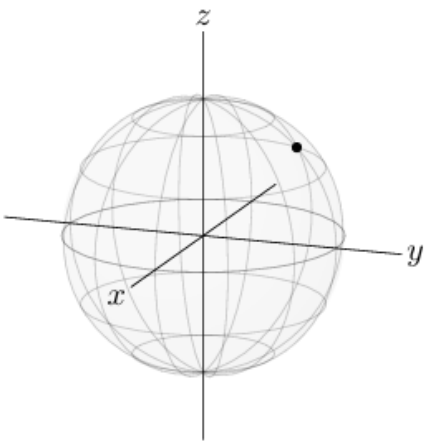
Coordinate Mapping

The Cartesian coordinates of the Bloch sphere are derived from θ and φ. These help visualize quantum states geometrically.

$$x = \sin\theta \cos\varphi,\ y = \sin\theta \sin\varphi,\ z = \cos\theta$$

Example :

$$\frac{3+i\sqrt{3}}{4}|0\rangle - \frac{1}{2}|1\rangle$$



$$
\begin{array}{rcll}
\alpha & = & a + ib & \text{\# definition of complex coordinates (Figure 3)} \\
& = & r\cos\phi + ir\sin\phi & \text{\# definitions: } a = r\cos\phi \text{ and } b = r\sin\phi \\
& = & r[\cos\phi + i\sin\phi] & \text{\# factor out } r \\
& = & re^{i\phi} & \text{\# Euler's formula: } e^{i\phi} = \cos\phi + i\sin\phi
\end{array}
$$

So now let $\alpha = r_0 e^{i\phi_0}$ and $\beta = r_1 e^{i\phi_1}$. Then we have

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = r_0 e^{i\phi_0}\,|0\rangle + r_1 e^{i\phi_1}\,|1\rangle$$

If we factor out $e^{i\phi_0}$, we get

$$|\psi\rangle = e^{i\phi_0}[r_0\,|0\rangle + r_1 e^{i(\phi_1-\phi_0)}\,|1\rangle]$$

It turns out that the term $e^{i\phi_0}$, the global phase, has no physical significance [2]; you can rotate the axes any way you like with no effect so long as the x, y and z axes remain perpendicular to one another. So we can drop the $e^{i\phi_0}$ term and write

$$|\psi\rangle = r_0\,|0\rangle + r_1 e^{i(\phi_1-\phi_0)}\,|1\rangle$$

Next let $\phi = \phi_1 - \phi_0$. Then we have

$$|\psi\rangle = r_0\,|0\rangle + r_1 e^{i\phi}\,|1\rangle \tag{6}$$

3 Normalization Constraints

One more observation we can make: Equation 6 tells us that we can write our normalization constraint $|\alpha|^2 + |\beta|^2 = 1$ as $|r_0|^2 + |r_1 e^{i\phi}|^2 = 1$.

Since r_0 has no complex term $|r_0|^2 = r_0^2$ and we can write $r_0^2 + |r_1 e^{i\phi}|^2 = 1$. The term $|r_1 e^{i\phi}|^2$ can be written as $r_1^2|e^{i\phi}|^2$. So now we have $r_0^2 + r_1^2|e^{i\phi}|^2 = 1$.

But we can learn a bit more. Recall that $e^{i\phi} = \cos\phi + i\sin\phi$ (Euler's formula), so

$$
\begin{aligned}
|e^{i\phi}|^2 &= |(\cos\phi + i\sin\phi)|^2 \\
&= (\cos\phi + i\sin\phi)(\cos\phi - i\sin\phi) \\
&= \cos^2\phi - \cos\phi\, i\sin\phi + i\sin\phi\cos\phi - i^2\sin^2\phi \\
&= \cos^2\phi - i^2\sin^2\phi \\
&= \cos^2\phi + \sin^2\phi \\
&= 1
\end{aligned}
$$

The result is that the constraint $|\alpha|^2 + |\beta|^2 = 1$ can be written as $r_0^2 + r_1^2 = 1$. If we choose $r_0 = \cos x$ and $r_1 = \sin x$ then we'll have $\cos^2 x + \sin^2 x = 1$ (by the Pythagorean trigonometric identity [14]), which satisfies the normalization constraint.

If we now let $x = \frac{\theta}{2}$ (see Section 2.2), we get Equation 1, namely $|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\phi}\sin\frac{\theta}{2}\,|1\rangle$.

Question Bank

1.Define a quantum bit and list its basic states.

2.What are computational basis states in quantum computing?

3.Write the four computational basis states of a two-qubit system.

4.State the general form of a two-qubit quantum state.

5.Explain the concept of a qubit and how it differs from a classical bit.

6.Explain the principle of superposition using the qubit state equation.

7.Describe how measurement affects a qubit state.

8.Discuss how a qubit state can be represented as a vector in a two-dimensional complex vector space.

9.Explain the Bloch sphere representation of a qubit with a diagram.

10.Describe the concept of multiple qubits and the associated state space.

11.Explain the difference between a bit and a qubit.

12.Summarize the fundamental differences between classical and quantum computational models.

13..Apply the concept of superposition to explain how quantum computers perform parallel computation.

14.Demonstrate how two classical bits and two qubits differ by applying superposition to a two-qubit system.

15.Construct the general quantum state of a two-qubit system and apply the normalization condition.

16.Apply Bloch sphere representation to describe the state of a qubit.

17.Show how Bloch sphere mapping helps in visualizing qubit rotations.

MODULE 2

Quantum Mechanics for computing

Linear algebra: matrices, vectors, tensor products, The postulates of quantum mechanics, density operator, Schmidt decomposition and purifications, Bell inequality.

2.1 Linear algebra

Linear algebra is the study of vector spaces and of linear operations on those vector spaces. A good understanding of quantum mechanics is based upon a solid grasp of elementary linear algebra. In this section we review some basic concepts from linear algebra, and describe the standard notations which are used for these concepts in the study of quantum mechanics. These notations are summarized in Figure 2.1, with the quantum notation in the left column, and the linear-algebraic description in the right column. You may like to glance at the table, and see how many of the concepts in the right column you recognize.

In our opinion the chief obstacle to assimilation of the postulates of quantum mechanics is not the postulates themselves, but rather the large body of linear algebraic notions required to understand them. Coupled with the unusual Dirac notation adopted by physi- cists for quantum mechanics, it can appear (falsely) quite fearsome. For these reasons, we advise the reader not familiar with quantum mechanics to quickly read through the material which follows, pausing mainly to concentrate on understanding the absolute ba- sics of the notation being used. Then proceed to a careful study of the main topic of the chapter – the postulates of quantum mechanics – returning to study the necessary linear algebraic notions and notations in more depth, as required.

The basic objects of linear algebra are *vector spaces*. The vector space of most interest to us is C^n, the space of all n-tuples of complex numbers, $(z_1,\ldots, z_n)$. The elements of a vector space are called *vectors*, and we will sometimes use the column matrix notation

$$\begin{bmatrix} z_1 \\ \vdots \\ z_n \end{bmatrix}$$

to indicate a vector. There is an *addition* operation defined which takes pairs of vectors to other vectors. In C^n the addition operation for vectors is defined by

$$\begin{bmatrix} z_1 \\ \vdots \\ z_n \end{bmatrix} + \begin{bmatrix} z'_1 \\ \vdots \\ z'_n \end{bmatrix} \equiv \begin{bmatrix} z_1 + z'_1 \\ \vdots \\ z_n + z'_n \end{bmatrix}$$

where the addition operations on the right are just ordinary additions of complex numbers. Furthermore, in a vector space there is a *multiplication by a scalar* operation. In C^n this operation is defined by

$$z \begin{bmatrix} z_1 \\ \vdots \\ z_n \end{bmatrix} \equiv \begin{bmatrix} zz_1 \\ \vdots \\ zz_n \end{bmatrix},$$

where z is a *scalar*, that is, a complex number, and the multiplications on the right are ordinary multiplications of complex numbers. Physicists sometimes refer to complex numbers as *c-numbers*.

Quantum mechanics is our main motivation for studying linear algebra, so we will use the standard notation of quantum mechanics for linear algebraic concepts. The standard quantum mechanical notation for a vector in a vector space is the following:

$$|\psi\rangle.$$

Notation	Description
z^*	Complex conjugate of the complex number z. $(1+i)^* = 1-i$
$\|\psi\rangle$	Vector. Also known as a *ket*.
$\langle\psi\|$	Vector dual to $\|\psi\rangle$. Also known as a *bra*.
$\langle\varphi\|\psi\rangle$	Inner product between the vectors $\|\varphi\rangle$ and $\|\psi\rangle$.
$\|\varphi\rangle \otimes \|\psi\rangle$	Tensor product of $\|\varphi\rangle$ and $\|\psi\rangle$.
$\|\varphi\rangle\|\psi\rangle$	Abbreviated notation for tensor product of $\|\varphi\rangle$ and $\|\psi\rangle$.
A^*	Complex conjugate of the A matrix.
A^T	Transpose of the A matrix.
$A^\dagger$	Hermitian conjugate or adjoint of the A matrix, $A^\dagger = (A^T)^*$. $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^\dagger = \begin{bmatrix} a^* & c^* \\ b^* & d^* \end{bmatrix}$.
$\langle\varphi\|A\|\psi\rangle$	Inner product between $\|\varphi\rangle$ and $A\|\psi\rangle$. Equivalently, inner product between $A^\dagger\|\varphi\rangle$ and $\|\psi\rangle$.

Figure 2.1 Summary of some standard quantum mechanical notation for notions from linear algebra. This style of notation is known as the Dirac notation.

Bases and linear independence

A spanning set for a vector space is a set of vectors |v1> ,...,|vn> such that any vector |v> in the vector space can be written as a linear combination $|v\rangle = \Sigma_i\, ai|vi\rangle$ of vectors in that set. For example, a spanning set for the vector space C^2 is the set

$$|v_1\rangle \equiv \begin{bmatrix} 1 \\ 0 \end{bmatrix}; \quad |v_2\rangle \equiv \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

since any vector

$$|v\rangle = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$$

in C^2 can be written as a linear combination |v> = a1|v1> +a2|v2> of the vectors |v1> and |v2> . We say that the vectors |v1> and |v2> span the vector space C^2. Generally, a vector space may have many different spanning sets. A second spanning set for the vector space C^2 is the set

$$|v_1\rangle \equiv \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}; \quad |v_2\rangle \equiv \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \tag{2.7}$$

since an arbitrary vector $|v\rangle = (a_1, a_2)$ can be written as a linear combination of $|v_1\rangle$ and $|v_2\rangle$,

$$|v\rangle = \frac{a_1 + a_2}{\sqrt{2}}|v_1\rangle + \frac{a_1 - a_2}{\sqrt{2}}|v_2\rangle. \tag{2.8}$$

A set of non-zero vectors $|v_1\rangle, \ldots, |v_n\rangle$ are *linearly dependent* if there exists a set of complex numbers $a_1, \ldots, a_n$ with $a_i \neq 0$ for at least one value of i, such that

$$a_1|v_1\rangle + a_2|v_2\rangle + \cdots + a_n|v_n\rangle = 0. \tag{2.9}$$

A set of vectors is linearly independent if it is not linearly dependent. It can be shown that any two sets of linearly independent vectors which span a vector space V contain the same number of elements. We call such a set a basis for V. Furthermore, such a basis set always exists. The number of elements in the basis is defined to be the dimension of V. Here we will only be interested in finite dimensional vector spaces. There are many interesting and often difficult questions associated with infinite dimensional vector spaces. We won't need to worry about these questions.

Linear Algebra is strikingly similar to the algebra you learned in high school, except that in the place of ordinary single numbers, it deals with vectors. Many of the same algebraic operations you're used to performing on ordinary numbers (a.k.a. scalars), such as addition, subtraction and multiplication, can be generalized to be performed on vectors. We'll better start by defining what we mean by *scalars* and *vectors*.

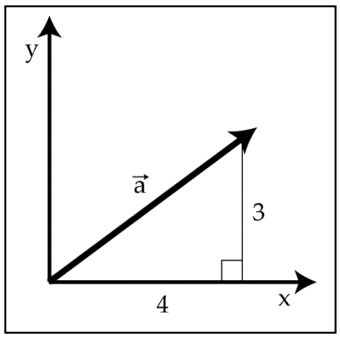
Definition: A scalar is a number. Examples of scalars are temperature, distance, speed, or mass – all quantities that have a magnitude but no "direction", other than perhaps positive or negative.

Okay, so scalars are what you're used to. In fact we could go so far as to describe the algebra you learned in grade school as *scalar* algebra, or the calculus many of us learned in high school as *scalar* calculus, be cause both dealt almost exclusively with scalars. This is to be contrasted with *vector* calculus or *vector* algebra, that most of us either only got in college if at all. So what *is* a vector?

Definition: A vector is *a list of numbers*. There are (at least) two ways to interpret what this list of numbers mean: One way to think of the vector as being *a point in a space*. Then this list of numbers is a way of identifying that point in space, where each number represents the vector's component that dimension. Another way to think of a vector is *a magnitude and a direction*, e.g. a quantity like velocity ("the fighter jet's velocity is 250 mph north-by-northwest"). In this way of think of it, a vector is a directed arrow pointing from the origin to the end point given by the list of numbers.

In this class we'll denote vectors by including a small arrow overtop of the symbol like so: $\vec{a}$. Another common convention you might encounter in other texts and papers is to denote vectors by use of a boldface font ($\boldsymbol{a}$). An example of a vector is $\vec{a} = [4,3]$. Graphically, you can think of this vector as an arrow in the *x*-*y* plane, pointing from the origin to the point at *x*=3, *y*=4 (see illustration.)

In this example, the list of numbers was only two elements long, but in principle it could be any length. The dimensionality of a vector is the length of the list. So, our example $\vec{a}$ is 2-dimensional because it is a list of two numbers. Not surprisingly all 2-dimentional vectors live in a plane. A 3-dimensional vector would be a list of three numbers, and they live in a 3-D volume. A 27-dimensional vector would be a list of twenty-seven numbers, and would live in a space only Ilana's dad could visualize.



Magnitudes and direction

The "magnitude" of a vector is the distance from the endpoint of the vector to the origin – in a word, it's length. Suppose we want to calculate the magnitude of the vector $\vec{a} = [4,3]$. This vector extends 4 units along the x-axis, and 3 units along the y-axis. To calculate the magnitude $\|\vec{a}\|$ of the vector we can use the Pythagorean theorem ($x^2 + y^2 = z^2$).

$$\|\vec{a}\| = \sqrt{x^2 + y^2} = \sqrt{4^2 + 3^2} = 5$$

The magnitude of a vector is a scalar value – a number representing the length of the vector independent of the direction. There are a lot of examples were the magnitudes of vectors are important to us: velocities are vectors, speeds are their magnitudes; displacements are vectors, distances are their magnitudes.

To complement the magnitude, which represents the length independent of direction, one might wish for a way of representing the direction of a vector independent of its length. For this purpose, we use "unit vectors," which are quite simply vectors with a magnitude of 1. A unit vector is denoted by a small "carrot" or "hat" above the symbol. For example, $\hat{a}$ represents the unit vector associated with the vector $\vec{a}$. To calculate the unit vector associated with a particular vector, we take the original vector and divide it by its magnitude. In mathematical terms, this process is written as:

$$\hat{a} = \frac{\vec{a}}{\|\vec{a}\|}$$

Definition: A unit vector is a vector of magnitude 1. Unit vectors can be used to express the direction of a vector independent of its magnitude.

Returning to the previous example of $\vec{a} = [4,3]$, recall $\|\vec{a}\| = 5$. When dividing a vector ($\vec{a}$) by a scalar ($\|\vec{a}\|$), we divide each component of the vector individually by the scalar. In the same way, when multiplying a vector by a scalar we will proceed component by component. Note that this will be very different when multiplying a vector by another vector, as discussed below. But for now, in the case of dividing a vector by a scalar we arrive at:

$$\hat{a} = \frac{[4,3]}{5}$$

$$\hat{a} = \left[\frac{4}{5}, \frac{3}{5}\right]$$

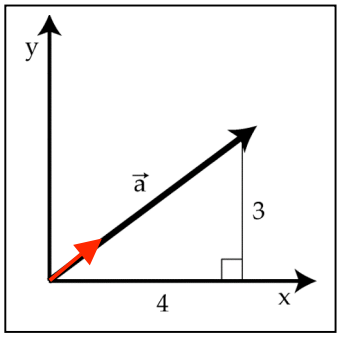
As shown in red in the figure, by dividing each component of the vector by the same number, we leave the direction of the vector unchanged, while we change the magnitude. If we have done this correctly, then the magnitude of the unit vector must be equal to 1 (otherwise it would not be a unit vector). We can verify this by calculating the magnitude of the unit vector $\|\hat{a}\|$.

$$\|\hat{a}\|^2 = \left(\frac{4}{5}\right)^2 + \left(\frac{3}{5}\right)^2$$

$$\|\hat{a}\|^2 = \left(\frac{16}{25}\right) + \left(\frac{9}{25}\right)$$

$$\|\hat{a}\|^2 = \left(\frac{25}{25}\right) = 1$$

$$\|\hat{a}\| = 1$$



So we have demonstrated how to create a unit vector $\hat{a}$ that has a magnitude of 1 but a direction identical to the vector $\vec{a}$. Taking together the magnitude $\|\vec{a}\|$ and the unit vector $\hat{a}$ we have all of the information contained in the vector $\vec{a}$, but neatly separated into its magnitude and direction components. We can use these two components to re-create the vector $\vec{a}$ by multiplying the vector $\hat{a}$ by the scalar $\|\vec{a}\|$ like so:

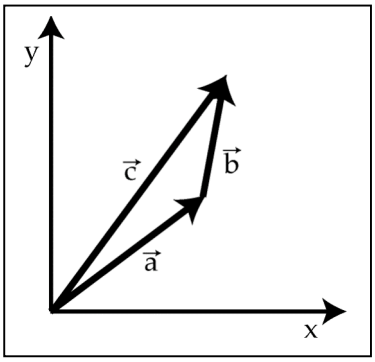
$$\vec{a} = \hat{a} * \|\vec{a}\|$$

Vector addition and subtraction

Vectors can be added and subtracted. Graphically, we can think of adding two vectors together as placing two line segments end-to-end, maintaining distance and direction. An example of this is shown in the illustration, showing the addition of two vectors $\vec{a}$ and $\vec{b}$ to create a third vector $\vec{c}$.

$$\vec{a} + \vec{b} = \vec{c}$$

Numerically, we add vectors component-by-component. That is to say, we add the x components together, and then separately we add the y components together. For example, if $\vec{a} = [4,3]$ and $\vec{b} = [1,2]$, then:



$$\vec{c} = \vec{a} + \vec{b}$$

$$\vec{c} = [4,3] + [1,2]$$

$$\vec{c} = [4 + 1, 3 + 2]$$

$$\vec{c} = [5,5]$$

Similarly, in vector subtraction:

$$\vec{c} = \vec{a} - \vec{b}$$

$$\vec{c} = [4,3] - [1,2]$$

$$\vec{c} = [3,1]$$

Vector addition has a very simple interpretation in the case of things like displacement. If in the morning a ship sailed 4 miles east and 3 miles north, and then in the afternoon it sailed a further 1 mile east and 2 miles north, what was the total displacement for the whole day? 5 miles east and 5 miles north – vector addition at work.

Linear independence

If two vectors point in different directions, even if they are not very different directions, then the two vectors are said to be *linearly independent*. If vectors $\vec{a}$ and $\vec{b}$ point in the same direction, then you can multiply vector $\vec{a}$ by a constant, scalar value and get vector $\vec{b}$, and vice versa to get from $\vec{b}$ to $\vec{a}$. If the two vectors point in different directions, then this is not possible to make one out of the other because multiplying a vector by a scalar will never change the direction of the vector, it will only change the magnitude. This concept generalizes to families of more than two vectors. Three vectors are said to be linearly independent if there is no way to construct one vector by combining scaled versions of the other two. The same definition applies to families of four or more vectors by applying the same rules.

The vectors in the previous figure provide a graphical example of linear independence. Vectors $\vec{a}$ and $\vec{c}$ point in slightly different directions. There is no way to change the length of vector $\vec{a}$ and generate vector $\vec{c}$, nor vice-versa to get from $\vec{c}$ to $\vec{a}$. If, on the other hand, we consider the family of vectors that contains $\vec{a}$, $\vec{b}$ and $\vec{c}$, it is now possible, as shown, to add vectors $\vec{a}$ and $\vec{b}$ to generate vector $\vec{c}$. So the family of vectors $\vec{a}$, $\vec{b}$ and $\vec{c}$ is *not* linearly independent, but is instead said to be linearly dependent. Incidentally, you could change the length of any or all of these three vectors and they would still be linearly dependent.

Definition: A family of vectors is linearly independent if no one of the vectors can be created by any linear combination of the other vectors in the family. For example, $\vec{c}$ is linearly independent of $\vec{a}$ and $\vec{b}$ if and only if it is *impossible* to find scalar values of α and β such that $\vec{c} = \alpha\vec{a} + \beta\vec{b}$

Vector multiplication: dot products

Next we move into the world of vector multiplication. There are two principal ways of multiplying vectors, called *dot products* (a.k.a. *scalar products*) and *cross products*. The dot product:

$$d = \vec{a} \cdot \vec{b}$$

generates a scalar value from the product of two vectors and will be discussed in greater detail below. Do not confuse the dot product with the cross product:

$$\vec{c} = \vec{a} \times \vec{b}$$

which is an entirely different beast. The cross product generates a vector from the product of two vectors. Cross products so up in physics sometimes, such as when describing the interaction between electrical and magnetic fields (ask your local fMRI expert), but we'll set those aside for now and just focus on dot products in this course. The dot product is calculated by multiplying the x components, then separately multiplying the y components (and so on for z, etc… for products in more than 2 dimensions) and then adding these products together. To do an example using the vectors above:

$$\vec{a} \cdot \vec{b} = [4,3] \cdot [1,2]$$

$$\vec{a} \cdot \vec{b} = (4 * 1) + (3 * 2)$$

$$\vec{a} \cdot \vec{b} = 11$$

Another way of calculating the dot product of two vectors is to use a geometric means. The dot product can be expressed geometrically as:

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos\theta$$

where θ represents the angle between the two vectors. Believe it or not, calculation of the dot product by either procedure will yield exactly the same result. Recall, again from high school geometry, that $\cos 0° = 1$, and that $\cos 90° = 0$. If the angle between $\vec{a}$ and $\vec{b}$ is nearly $0°$ (i.e. if the vectors point in nearly the same direction), then the dot product of the two vectors will be nearly $\|\vec{a}\| \|\vec{b}\|$.

Definition: A dot product (or scalar product) is the numerical product of the lengths of two vectors, multiplied by the cosine of the angle between them.

Orthogonality

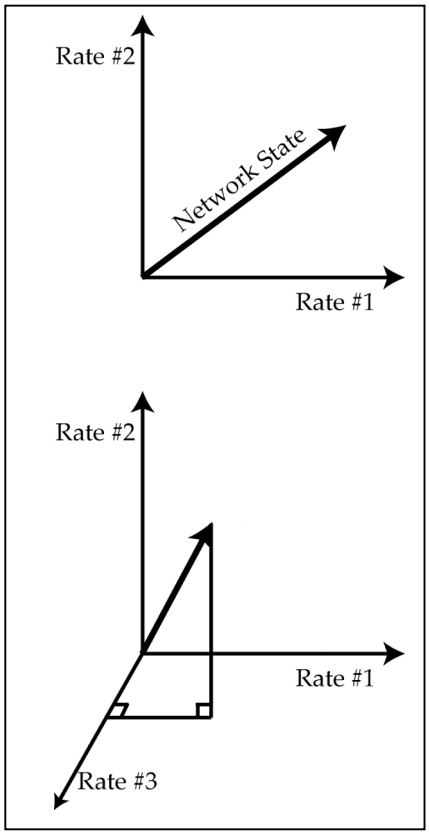
As the angle between the two vectors opens up to approach $90°$, the dot product of the two vectors will approach 0, regardless of the vector magnitudes $\|\vec{a}\|$ and $\|\vec{b}\|$. In the special case that the angle between the two vectors is exactly $90°$, the dot product of the two vectors will be 0 regardless of the magnitude of the vectors. In this case, the two vectors are said to be *orthogonal*.

Definition: Two vectors are orthogonal to one another if the dot product of those two vectors is equal to zero.

Orthogonality is an important and general concept, and is a more mathematically precise way of saying "perpendicular." In two- or three-dimensional space, orthogonality is identical to perpendicularity and the two ideas can be thought of interchangeably. Whereas perpendicularity is restricted to spatial representations of things, orthogonality is a more general term. In the context of neural networks, neuroscientists will often talk in terms of two patterns of neuronal firing being orthogonal to one another. Orthogonality can also apply to functions as well as to things like vectors and firing rates. As we will discuss later in this class in the context of *fourier transforms*, the sin and cos functions can be said to be orthogonal functions. In any of these contexts, orthogonality will always mean something akin to "totally independent" and is specifically refering to two things having a dot product of zero.

Vector spaces

All vectors live within a *vector space*. A vector space is exactly what it sounds like – the space in which vectors live. When talking about spatial vectors, for instance the direction and speed with which a person is walking through a room, the vector space is intuitively spatial since all available directions of motion can be plotted directly onto a spatial map of the room.



A less spatially intuitive example of a vector space might be all available states of a neural network. Imagine a very simple network, consisting of only five neurons which we will call n_1, n_2, n_3, n_4, and n_5. At each point in time, each neuron might not fire any action potentials at all, in which case we write $n_i = 0$, where i denotes the neuron number. Alternatively, the neuron might be firing action potentials at a rate of up to 100 Hz, in which case we write that $n_i = x$, where $0 \le x \le 100$. The state of this network at any moment in time can be depicted by a vector that describes the firing rates of all five neurons:

$s = [n_1, n_2, n_3, n_4, n_5]$

The set of all possible firing rates for all five neurons represents a vector space that is every bit as real as the vector space represented by a person walking through a room. The vector space represented by the neurons, however, is a 5-dimensional vector space. The math is identical to the two dimensional situation, but in this case we must trust the math because our graphical intuition fails us.

If we call state 1 the state in which neuron #1 is firing at a rate of 1 Hz and all others are silent, we can write this as:

$$s_1 = [1, 0, 0, 0, 0]$$

We may further define states 2, 3, 4, and 5 as follows:

$$s_2 = [0, 1, 0, 0, 0]$$

$$s_3 = [0, 0, 1, 0, 0]$$

$$s_4 = [0, 0, 0, 1, 0]$$

$$s_5 = [0, 0, 0, 0, 1]$$

By taking combinations of these five *basis vectors*, and multiplying them by scalar constants, we can describe any state of the network in the entire vector space. For example, to generate the network state [0, 3, 0, 9, 0] we could write:

$$(3 * s_2) + (9 * s_4) = [0, 3, 0, 9, 0]$$

If any one of the basis vectors is removed from the set, however, there will be some states of the network we will be unable to describe. For example, no combination of the vectors s_1, s_2, s_3, and s_4 can describe the network state [1, 0, 5, 3, 2] without also making use of s_5. Every vector space has a set of basis vectors. The definition of a set of basis vectors is twofold: (1) linear combinations (meaning addition, subtraction and multiplication by scalars) of the basis vectors can describe any vector in the vector space, and (2) every one of the basis vectors must be required in order to be able to describe all of the vectors in the vector space. It is also worth noting that the vectors s_1, s_2, s_3, s_4, and s_5 are all orthogonal to one another. You can test this for yourself by calculating the dot product of any two of these five basis vectors and verifying that it is zero. Basis vectors are not always orthogonal to one another, but they must always be linearly independent. The vector space that is defined by the set of all vectors you can possibly generate with different combinations of the basis vectors is called the *span* of the basis vectors.

Definition: A basis set is a linearly independent set of vectors that, when used in linear combination, can represent every vector in a given vector space.

It's worth taking a moment to consider what vector space is actually defined by these basis vectors. It's not all of 5-dimensional space, because we don't allow negative firing rates. So it's sort of the positive "quadrant" of 5-dimensional space. But we also said that we don't allow firing rates over 100Hz,

so it's really only the part of that quadrant that fits between 0 and 100. This is a 5-dimensional hypercube. Doesn't that just sound so cool.

Matrices

A matrix, like a vector, is also a collection of numbers. The difference is that a matrix is a table of numbers rather than a list. Many of the same rules we just outlined for vectors above apply equally well to matrices. (In fact, you can think of vectors as matrices that happen to only have one column or one row.) First, let's consider matrix addition and subtraction. This part is uncomplicated. You can add and subtract matrices the same way you add vectors – element by element:

$$A + B = C$$

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{pmatrix}$$

Matrix multiplication gets a bit more complicated, since multiple elements in the first matrix interact with multiple elements in the second to generate each element in the product matrix. This means that matrix multiplication can be a tedious task to carry out by hand, and can be time consuming on a computer for very large matrices. But as we shall see, this also means that, depending on the matrices we are multiplying, we will be able to perform a vast array of computations. Shown here is a simple example of multiplying a pair of 2×2 matrices, but this same procedure can generalize to matrices of arbitrarily large size, as will be discussed below:

$$AB = D$$

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

If you look at what's going on here, you may notice that what's going on it that we're taking dot products between rows of the ***A*** matrix with columns of the ***B*** matrix. for example to find the entry in our new matrix that's in the first-row-second-column position, we take the first row of ***A*** and compute it's dot product with the second column of ***B***. Matrix multiplication and dot products are intimately linked.

When multiplying two matrices together, the two matrices do not need to be the same size. For example, the vector $\vec{a} = (4,3)$ from earlier in this chapter represents a very simple matrix. If we multiply this matrix by another simple matrix the result is:

$$\begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 3 \\ -4 \end{pmatrix}$$

Linear Transformations

Linear transformations are functions (functions that take vectors as inputs), but their a special subset of functions. Linear transformations are the set of all functions that can be written as a matrix multiplication:

$$\vec{y} = A\vec{x}$$

This sees like a pretty restrictive class of function, and indeed it is. But as it turns out, these simple functions can accomplish some surprisingly interesting things.

Consider the matrix we used for matrix multiplication in our last example.

$$A = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

When multiplied the vector [4,3] by this matrix, we got the vector [-3,4]. If you draw those to vectors you'll notice that the two vectors have the same length, but the second vector is rotated 90º counter clockwise from the first vector. Coincidence? Pick another vector, say [100, 0]; that vector gets transformed to the vector [0, 100], rotating 90º from the 3 o'clock position to the 12 o'clock position. As it turns out this is true for any vector $\vec{x}$ you pick: $\vec{y}$ will always be rotated 90º counter clockwise.

As it turns out our matrix $\boldsymbol{A}$ is a very special matrix. It's a *rotation matrix* – it never changes the length of a vector, but changes every vector's direction by a certain fixed amount – in this case 90º. This is just one special case of a rotation matrix. We can generate a rotation matrix that will allow us to rotate a vector by any angle we want. The general rotation matrix to rotate a vector by an angle θ looks like this:

$$R_\theta = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

First you plug in the desired value for θ to calculate the different elements of the matrix, then you multiply the matrix by the vector and you have a rotated vector.

This may sound like an awful lot to go through when a protractor would do just as well. The reason this is worth all the trouble is that it doesn't stop at rotations, it doesn't even stop at two dimensions. Above we defined a rotation matrix $\boldsymbol{R}$ for two dimensions. We can just as well define a rotation matrix in three dimensions, it would just be a 3×3 matrix rather than the 2×2 one above. In the case above, we think of the rotation matrix as "operating on" the vector to perform a rotation. Following this terminology, we call the rotation matrix the *operator*. We can make all sorts of other linear operators (matrixes) to do all sorts of other transformations. A few other useful linear transformations include identity $(\boldsymbol{I})$, Stretch and squash $(\boldsymbol{S})$, Skew $(\boldsymbol{W})$ and Flip $(\boldsymbol{F})$.

You may notice that since a linear operator is anything that can be expressed as a matrix, that means that you can stick any series of linear operators together to make a new function, and that function

will also be a linear operator. For example if we want to rotate a vector and then stretch it we could, premultiply those to matrices together, and the resulting matrix operator will still be linear and will still follow all the same rules. So if $\boldsymbol{S} \cdot \boldsymbol{R} = \boldsymbol{X}$, then we can apply $\boldsymbol{X}$ as its own operator. Extending this, any set of linear transformations, no mater how long can be multiplied together and condensed into a single operator. For instance, if we wanted to stretch-skew-rotate-skew-stretch-flip-rotate ($\boldsymbol{S} \cdot \boldsymbol{W} \cdot \boldsymbol{R} \cdot \boldsymbol{W} \cdot \boldsymbol{S} \cdot \boldsymbol{F} \cdot \boldsymbol{R}$), we can define all of that into a single new operator $\boldsymbol{Y}$.

Tensor Products

Basic Definition: Let R be a commutative ring with 1. A (*unital*) R-*module* is an abelian group M together with a operation $R \times M \to M$, usually just written as rv when $r \in R$ and $v \in M$. This operation is called *scaling*. The scaling operation satisfies the following conditions.

1. $1v = v$ for all $v \in M$.
2. $(rs)v = r(sv)$ for all $r, s \in R$ and all $v \in M$.
3. $(r + s)v = rv + sv$ for all $r, s \in R$ and all $v \in M$.
4. $r(v + w) = rv + rw$ for all $r \in R$ and $v, w \in M$.

Technically, an R-module just satisfies properties 2, 3, 4. However, without the first property, the module is pretty pathological. So, we'll always work with unital modules and just call them modules. When R is understood, we'll just say *module* when we mean *unital* R-*module*.

Submodules and Quotient Modules: A *submodule* $N \subset M$ is an abelian group which is closed under the scaling operation. So, $rv \in N$ provided that $v \in N$. A submodule of a module is very much like an ideal of a ring. One defines M/N to be the set of (additive) cosets of N in M, and one has the scaling operation $r(v + N) = (rv) + N$. This makes M/N into another R-module.

Examples: Here are some examples of R-modules.

- When R is a field, an R-module is just a vector space over R.
- The direct product $M_1 \times M_2$ is a module. The addition operation is done coordinate-wise, and the scaling operation is given by

  $$r(v_1, v_2) = (rv_1, rv_2).$$

  More generally, $M_1 \times ... \times M_n$ is another R-module when $M_1, ..., M_n$ are.
- If M is a module, so is the set of finite formal linear combinations $L(M)$ of elements of M. A typical element of $L(M)$ is $r_1(v_1) + ... +$

  $$r_1(v_n), \qquad r_1, ..., r_n \in R, \quad v_1, ..., v_n \in M.$$

  This definition is subtle. The operations in M allow you to simplify these expressions, but in $L(M)$ you are not allowed to simplify. Thus, for instance, $r(v)$ and $1(rv)$ are considered distinct elements if $r \neq 1$.
- If $S \subset M$ is some subset, then $R(S)$ is the set of all finite linear combinations of elements of S, where simplification is allowed. With this definition, $R(S)$ is a submodule of M. In fact, $R(S)$ is the smallest submodule that contains S. Any other submodule containing S also

The Tensor Product

The tensor product of two R-modules is built out of the examples given above. Let M and N be two R-modules. Here is the formula for $M \otimes N$:

$$M \otimes N = Y/Y(S), \qquad Y = L(M \times N), \tag{1}$$

and S is the set of all formal sums of the following type:

1. $(rv, w) - r(v, w)$.
2. $(w, rv) - r(v, w)$.
3. $(v_1 + v_2, w) - (v_1, w) - (v_2, w)$.
4. $(v, w_1 + w_2) - (v, w_1) - (v, w_2)$.

Our convention is that (v, w) stands for $1(v, w)$, which really is an element of $L(M \times N)$. Being the quotient of an R-module by a submodule, $M \otimes N$ is another R-module. It is called the *tensor product* of M and N.

There is a map $B : M \times N \to M \otimes N$ given by the formula

$$B(m, n) = [(m, n)] = (m, n) + Y(S), \tag{2}$$

namely, the $Y(S)$-coset of (m, n). The traditional notation is to write

$$m \otimes n = B(m, n). \tag{3}$$

The operation $m \otimes n$ is called the *tensor product of elements*.

Given the nature of the set S in the definition of the tensor product, we have the following rules:

1. $(rv) \otimes w = r(v \otimes w)$.
2. $r \otimes (rw) = r(v \otimes w)$.
3. $(v_1 + v_2) \otimes w = v_1 \otimes w + v_2 \otimes w$.
4. $v \otimes (w_1 + w_2) = v \otimes w_1 + v \otimes w_2$.

These equations make sense because $M \otimes N$ is another R-module. They can be summarised by saying that the map B is *bilinear*. We will elaborate below.

An Example: Sometimes it is possible to figure out $M \otimes N$ just from using the rules above. Here is a classic example. Let $R = \boldsymbol{Z}$, the integers. Any finite abelian group is a module over $\boldsymbol{Z}$. The scaling rule is just $mg = g + ... + g$ (m times). In particular, this is true for $\boldsymbol{Z}/n$. Let's show that $\boldsymbol{Z}/2 \otimes \boldsymbol{Z}/3$ is the trivial module.

Consider the element $1 \otimes 1$. We have

$$2(1 \otimes 1) = 2 \otimes 1 = 0 \otimes 1 = 0(1 \otimes 1) = 0.$$

At the same time

$$2(1 \otimes 1) = 1 \otimes 3 = 1 \otimes 0 = 0(1 \otimes 1) = 0.$$

But then

$$1(1 \otimes 1) = (3 - 2)(1 \otimes 1) = 0 - 0 = 0.$$

Hence $1 \otimes 1$ is trivial. From here it is easy to see that $a \otimes b$ is trivial for all $a \in \boldsymbol{Z}/2$ and $b \in \boldsymbol{Z}/3$. There really aren't many choices. But $\boldsymbol{Z}/2 \otimes \boldsymbol{Z}/3$ is the span of the image of $M \times N$ under the tensor map. Hence $\boldsymbol{Z}/2 \otimes \boldsymbol{Z}/3$ is trivial.

The Universal Property

Linear and Bilinear Maps: Let M and N be R-modules. A map $\phi : M \to N$ is *R-linear* (or just *linear* for short) provided that

1. $\phi(rv) = r\phi(v)$.
2. $\phi(v_1 + v_2) = \phi(v_1) + \phi(v_2)$.

A map $\phi : M \times N \to P$ is *R-bilinear* if

1. For any $m \in M$, the map $n \to \phi(m, n)$ is a linear map from N to P.
2. For any $n \in N$, the map $m \to \phi(m, n)$ is a linear map from M to P.

Existence of the Universal Property: The tensor product has what is called a *universal property*. the name comes from the fact that the construction to follow works for all maps of the given type.

Lemma 3.1 *Suppose that $\phi : M \times N \to P$ is a bilinear map. Then there is a linear map $\phi : M \otimes N \to P$ such that $\phi(m, n) = \widehat{\phi}(m \otimes n)$. Equivalently, $\phi = \widehat{\phi} \circ B$, where $B : M \times N \to M \otimes N$ is as above.*

Proof: First of all, there is a linear map $\psi : Y(M \times N) \to P$. The map is given by

$$\psi(r_1(v_1, w_1) + ... + r_n(v_n, w_n)) = r_1\psi(v_1, w_1) + ... + r_n\psi(v_n, w_n). \quad (4)$$

That is, we do the obvious map, and then simplify the sum in P. Since ϕ is bilinear, we see that $\psi(s) = 0$ for all $s \in S$. Therefore, $\psi = 0$ on $Y(S)$. But then ψ gives rise to a map from $M \otimes N = Y/Y(S)$ into P, just using the formula

$$\widehat{\phi}(a + Y(S)) = \psi(a). \quad (5)$$

Since ψ vanishes on $Y(S)$, this definition is the same no matter what coset representative is chosen. By construction $\widehat{\phi}$ is linear and satisfies $\widehat{\phi}(m \otimes n) = \phi(m, n)$. ♠

Uniqueness of the Universal Property: Not only does $(B, M \otimes N)$ have the universal property, but any other pair $(B', (M \otimes N)')$ with the same property is essentially identical to $(B, M \otimes N)$. The next result says this precisely.

Lemma 3.2 *Suppose that $(B', (M \otimes N)')$ is a pair satisfying the following axioms:*

- *$(M \otimes N)'$ is an R-module.*
- *$B' : M \times N \to (M \otimes N)'$ is a bilinear map.*
- *$(M \otimes N)'$ is spanned by the image $B'(M \times N)$.*
- *For any bilinear map $T : M \times N \to P$ there is a linear map $L : (M \otimes N)' \to P$ such that $T = L \circ B'$.*

Then there is an isomorphism $I : M \otimes N \to (M \otimes N)'$ and $B' = I \circ B$.

Proof: Since $(B, M \otimes N)$ has the universal property, and we know that $B' : M \times N \to (M \otimes N)'$ is a bilinear map, there is a linear map $I : M \otimes N \to (M \otimes N)'$ such that

$$B' = I \circ B.$$

We just have to show that I is an isomorphism. Reversing the roles of the two pairs, we also have a linear map $J : (M \otimes N)' \to M \otimes N$ such that

$$B = J \circ B'.$$

Combining these equations, we see that

$$B = J \circ I \circ B.$$

But then $J \circ I$ is the identity on the set $B(M \times N)$. But this set spans $M \otimes N$. Hence $J \circ I$ is the identity on $M \otimes N$. The same argument shows that $I \circ J$ is the identity on $(M \otimes N)'$. But this situation is only possible if both I and J are isomorphisms. ♠

Vector Spaces

The tensor product of two vectors spaces is much more concrete. We will change notation so that F is a field and V, W are vector spaces over F . Just to make the exposition clean, we will assume that V and W are finite

dimensional vector spaces. Let $v_1, ..., v_m$ be a basis for V and let $w_1, ..., w_n$ be a basis for W. We define $V \otimes W$ to be the set of formal linear combinations of the mn symbols $v_i \otimes w_j$. That is, a typical element of $V \otimes W$ is

$$\sum_{i,j} c_{ij}(v_i \otimes w_j). \tag{6}$$

The space $V \otimes W$ is clearly a finite dimensional vector space of dimension mn. it is important to note that we are not giving a circular definition. This time $v_i \otimes w_j$ is just a formal symbol.

However, now we would like to define the bilinear map

$$B : V \times W \to V \otimes W.$$

Here is the formula

$$B\Big(\sum a_i v_i, \sum b_j w_j\Big) = \sum_{i,j} a_i b_j (v_i \otimes w_j). \tag{7}$$

This gives a complete definition because every element of V is a unique linear combination of the $\{v_i\}$ and every element of W is a unique linear combination of the $\{w_j\}$. A routine check shows that B is a bilinear map.

Finally, if $T : V \times W \to P$ is some bilinear map, we define $L : V \otimes W \to P$ using the formula

$$L\Big(\sum_{i,j} c_{ij}(v_i \otimes w_j)\Big) = \sum_{i,j} c_{ij} T(v_i, w_j). \tag{8}$$

It is an easy matter to check that L is linear and that $T = L \circ B$.

Since our definition here of B and $V \otimes W$ satisfies the universal property, it must coincide with the more abstract definition given above.

Properties of the Tensor Product

Going back to the general case, here I'll work out some properties of the tensor product. As usual, all modules are unital R-modules over the ring R.

Lemma 5.1 *$M \otimes N$ is isomorphic to $N \otimes M$.*

Proof: This is obvious from the construction. The map $(v, w) \to (w, v)$ extends to give an isomorphism from $Y_{M,N} = L(M \times N)$ to $Y_{N,M} = L(N \times N)$, and this isomorphism maps the set $S_{M,N} \subset Y_{M,N}$ of bilinear relations set $S_{N,M} \subset Y_{N,M}$ and therefore gives an isomorphism between the ideals $Y_{M,N}S_{M,N}$ and $Y_{N,M}S_{N,M}$. So, the obvious map induces an isomorphism on the quotients. ♠

Lemma 5.2 *$R \otimes M$ is isomorphic to M.*

Proof: The module axioms give us a surjective bilinear map $T : R \times M \to M$ given by $T(r, m) = rm$. By the universal property, there is a linear map $L : R \otimes M \to M$ such that $T = L \circ B$. Since T is surjective, L is also surjective. At the same time, we have a map $L^* : M \to R \otimes M$ given by the formula

$$L^*(v) = B(1, v) = 1 \otimes v. \tag{9}$$

The map L^* is linear because B is bilinear. We compute

$$L^* \circ L(r \otimes v) = L^*(rv) = 1 \otimes rv = r \otimes v. \tag{10}$$

So $L^* \circ L$ is the identity on the image $B(R \times M)$. But this image spans $R \otimes M$. Hence $L^* \circ L$ is the identity. But this is only possible if L is injective. Hence L is an isomorphism. ♠

Lemma 5.3 *$M \otimes (N_1 \times N_2)$ is isomorphic to $(M \otimes N_1) \times (N \otimes N_2)$.*

Proof: Let $N = N_1 \times N_2$. There is an obvious isomorphism ϕ from $Y = Y_{M,N}$ to $Y_1 \times Y_2$, where $Y_j = Y_{M,N_j}$, and $\phi(S) = S_1 \times S_2$. Here $S_j = S_{M,N_j}$. Therefore, ϕ induces an isomorphism from Y/YS to $(Y_1/Y_1S_1) \times (Y_2/Y_2S_2)$. ♠

Finally, we can prove something (slghtly) nontrivial.

Lemma 5.4 *$M \otimes R^n$ is isomorphic to M^n.*

Proof: By repeated applications of the previous result, $M \otimes R^n$ is isomorphic to $(M \otimes R)^n$, which is in turn isomorphic to M^n. ♠

As a special case,

Corollary 5.5 *$R^m \otimes R^n$ is isomorphic to R^{mn}.*

This is a reassurance that we got things right for vector spaces.

For our next result we need a technical lemma.

Lemma 5.6 *Suppose that Y is a module and $Y' \subset Y$ and $I \subset Y$ are both submodules. Let $I' = I \cap Y'$. Then there is an injective linear map from Y'/I' into Y/I.*

Proof: We have a linear map $\phi : Y' \to Y/I$ induced by the inclusion from Y' into Y. Suppose that $\phi(a) = 0$. Then $a \in I$. But, at the same time $a \in Y'$. Hence $a \in I'$. Conversely, if $a \in I'$ then $\phi(a) = 0$. In short, the kernel of ϕ is I'. But then the usual isomorphism theorem shows that ϕ induces an injective linear map from Y'/I' into Y/I. ♠

Now we deduce the corollary we care about.

Lemma 5.7 *Suppose that $M' \subset M$ and $N' \subset N$ are submodules. Then there is an injective linear map from $M' \otimes N'$ into $M \otimes N$. This map is the identity on elements of the form $a \otimes b$, where $a \in M'$ and $b \in N'$.*

Proof: We apply the previous result to the module $Y = Y_{M,N}$ and the submodules $I = S_{M,N}$ and $M' = Y_{M',N'}$. ♠

In view of the previous result, we can think of $M' \otimes N'$ as a submodule of $M \otimes N$ when $M' \subset N$ and $N' \subset N$ are submodules.

This last result says something about vector spaces. Let's take an example where the field is $\boldsymbol{Q}$ and the vector spaces are $\boldsymbol{R}$ and $\boldsymbol{R}/\boldsymbol{Q}$. These two vector spaces are infinite dimensional. It follows from Zorn's lemma that they both have bases. However, You might want to see that $\boldsymbol{R} \otimes \boldsymbol{R}/\boldsymbol{Q}$ is nontrivial even without using a basis for both. If we take any finite dimensional subspaces $V \subset \boldsymbol{R}$ and $W \subset \boldsymbol{R}/\boldsymbol{Q}$, then we know $V \otimes W$ is a submodule of $\boldsymbol{R} \otimes \boldsymbol{R}/\boldsymbol{Q}$. Hence $\boldsymbol{R} \otimes \boldsymbol{R}/\boldsymbol{Q}$ is nontrivial. In particular, we can use this to show that the element $1 \otimes [\alpha]$ is nontrivial when α is irrational.

Postulates of Quantum Mechanics - First Postulate

At each instant the state of a physical system is represented by a **ket** $|\psi\rangle$ *in the space of states.*

Comments

- Thespace of states is a vector space. This postulate is already radical because it implies that the *superposition* of two states is again a state of the system. If $|\psi_1\rangle$ and $|\ \psi_2\rangle$ are possible states of a system, then so is

$$|\psi\rangle = a_1|\psi_1\rangle + a_2|\psi_2\rangle, \tag{1}$$

  where a_1 and a_2 are complex numbers. Imagine that $|\psi_1\rangle$ is a particle with one value for some property like location and $|\psi_2\rangle$ is the same particle with a different value. In quantum mechanics we must allow ourselves to consider which superpose a particle in different locations. We were forced to do this by the results of experiments like the double-slit diffraction of electrons.

- Thespace of states comes equipped with the concept of an *inner product* which we abstract from wave mechanics. The inner product associates a complex number to any two states

$$(|\psi\rangle, |\phi\rangle) \equiv \langle\psi|\phi\rangle = \int dx \psi^*(x)\phi(x). \tag{2}$$

  Here we have used two different notations. The first defines the inner product as an operation acting on two states in the ket space. The second introduces another copy of the space of states called the "bra space", and defines the inner product as an operation involving one element of the bra space and one element of the ket space. Either way, the inner product reduces to the integral overlap of the two states when evaluated in terms of wavefunctions — $\int \psi^*\phi$. From (2) we see that

$$\langle\psi|\phi\rangle^* = \langle\phi|\psi\rangle. \tag{3}$$

1.2 Second Postulate

Every observable attribute of a physical system is described by an operator that acts on the kets that describe the system.

Comments

- By *convention*, an operator $\hat{\mathcal{A}}$ acting on a ket $|\psi\rangle$ is denoted by left multiplication,

$$\hat{\mathcal{A}} : |\psi\rangle \rightarrow |\psi'\rangle = \hat{\mathcal{A}}|\psi\rangle. \tag{4}$$

You are used to this concept in the context of wave-mechanics, where the concept of a *state* is replaced by that of a *wavefunction*. A system (a particle in a potential, for example) is described by a wavefunction $\psi(x)$ in wave-mechanics. Some simple observable attributes of such a system are its *position*, its *momentum* and its *energy*. These are represented in wave mechanics by *differential operators*, $\hat{X} = x$, $\hat{P} = -i\hbar\frac{d}{dx}$ and $\hat{\mathcal{H}} = -\frac{\hbar^2}{2m}\frac{d^2}{dx^2} + V(x)$ respectively. These operators act on a wavefunction by left-multiplication, like

$$\hat{P}\psi(x) = -i\hbar\frac{d\psi}{dx} \tag{5}$$

- It is important to recognize that acting with an operator on a state in general changes the state. Again think back to wave mechanics. The lowest energy eigenfunction in a square well ($0 \leq x \leq L$) is

$$\psi(x) = \sqrt{2/L}\sin\pi x/L \quad \text{for} \quad 0 \leq x \leq L. \tag{6}$$

When we act on this wavefunction with $\hat{P}$, for example, we get

$$\hat{P}\psi(x) = -i\hbar\pi/L\sqrt{2/L}\cos\pi x/L \tag{7}$$

which is no longer an energy eigenfunction at all. So the operator changed the state of the particle.

- For every operator, there are special states that are not changed (except for being multiplied by a constant) by the action of an operator,

$$\hat{\mathcal{A}}|\psi_a\rangle = a|\psi_a\rangle. \tag{8}$$

These are the *eigenstates* and the numbers a are the *eigenvalues* of the operator. You have encountered them in wave mechanics, now they show up in the abstract space of states.

1.3 Third Postulate

The only possible result of the measurement of an observable $\mathcal{A}$ is one of the eigenvalues of the corresponding operator $\hat{\mathcal{A}}$.

Comments

- This is, of course, the origin of the word "quantum" in quantum mechanics. If the observable has a continuous spectrum of eigenvalues, like the position x or the momentum p, then the statement is not surprising. If it has a discrete spectrum, like the Hamiltonian for an electron bound to a proton (the hydrogen atom), then the statement is shocking. A measurement of the energy of the hydrogen atom will yield

only one of a discrete set of values. Needless to say, this postulate reflects mountains of experimental evidence such as the discrete *spectral lines* observed in the radiation from a tube of hot hydrogen gas.

- Since we measure only real numbers, the eigenvalues of operators corresponding to observables had better be real. Operators with real eigenvalues are *hermitian*.

  The eigenstates of a hermitian operator have some important properties.

  - They are orthogonal

$$\langle a_j|a_k\rangle \equiv (a_j, a_k) = \int dx \psi^*_{a_j}(x)\psi_{a_k}(x) = \delta_{jk}. \tag{9}$$

  - They span the space of states, so they form a *basis*. This means than an arbitrary state can be expanded as a sum (with complex coefficients) of the eigenstates of a hermitian operator. For this reason we say that the set of states is "complete".

1.4 Fourth Postulate

When a measurement of an observable $\mathcal{A}$ is made on a generic state $|\psi\rangle$, the probability of obtaining an eigenvalue a_n is given by the square of the inner product of $|\psi\rangle$ with the eigenstate $|a_n\rangle$, $|\langle a_n|\psi\rangle|^2$.

Comments

- The states are assumed to be normalized. Usually we normalize our states to unity,

$$\begin{aligned} \langle\psi|\psi\rangle &= 1 \\ \langle a_j|a_k\rangle &= \delta_{jk}. \end{aligned} \tag{10}$$

  Sometimes this is not possible. The case of momentum eigenstates, $\psi_p(x) = \frac{1}{\sqrt{2\pi\hbar}} \exp ipx/\hbar$, is the classic example. In this case we must use "δ-function" or "continuum" normalization as discussed in Section 2.

- The complex number, $\langle a_n|\psi\rangle$ is known as the "probability amplitude" or "amplitude", for short, to measure a_n as the value for $\mathcal{A}$ in the state $|\psi\rangle$.

- Here is the algebraic exercise suggested by this postulate. First, any state can be expanded as a superposition of $\mathcal{A}$-eigenstates (see Post. 3),

$$|\psi\rangle = \sum_n c_n |a_n\rangle. \tag{11}$$

Next use the orthonormality of the $\mathcal{A}$ eigenstates to find an expression for the expansion coefficients c_n,

$$\begin{aligned} \langle a_j|\psi\rangle &= \sum_n c_n \langle a_j|a_n\rangle \\ &= c_j. \end{aligned} \tag{12}$$

So,

$$|\psi\rangle = \sum_n \langle a_n|\psi\rangle \cdot |a_n\rangle. \tag{13}$$

The $\cdot$ is added just to make clear the separation between the complex number $\langle a_n|\psi\rangle$ and the state $|a_n\rangle$. So, the component of $|\psi\rangle$ along the "direction" of the n^{th} eigenstate of $\mathcal{A}$ is given by $\langle a_n|\psi\rangle$. The measurement operation yields the result a_n with a probability proportional to the square of this component, $|\langle a_n|\psi\rangle|^2$.

- The probability of obtaining *some result* is unity. For states normalized to unity,

$$|\langle\psi|\psi\rangle|^2 = \sum_m \sum_n c_m^* c_n \langle a_m|a_n\rangle. \tag{14}$$

Using $|\langle\psi|\psi\rangle| = 1$ and $\langle a_m|a_n\rangle = \delta_{mn}$, we get

$$\sum_n |c_n|^2 = 1 \tag{15}$$

- According to the usual rules of probability, we can compute the "expected value" of the observable $\mathcal{A}$. If the probability to observe a_n is $|c_n|^2$ then the expected value (denoted $\langle\mathcal{A}\rangle$) is

$$\langle\mathcal{A}\rangle = \sum_n a_n |c_n|^2. \tag{16}$$

- When there is more than one eigenstate with the same eigenvalue, then this discussion needs a little bit of refinement. We'll let this go until we need to confront it.

1.5 Fifth Postulate

Immediately after the measurement of an observable $\mathcal{A}$ has yielded a value a_n, the state of the system is the normalized eigenstate $|a_n\rangle$.

Comments

- Known picturesquely as the "collapse of the wavepacket", this is the most controversial of the postulates of quantum mechanics, and the most difficult to get comfortable with. It is motivated by experience with repeated measurements. If an experimental sample is prepared in a state $|\psi\rangle$ then it is observed that a measurement of $\mathcal{A}$ can

yield a variety of results a_n with probabilities $|\langle a_n|\psi\rangle|^2$. Identically prepared systems can yield different experimental outcomes. This is encompassed by the fourth postulate. However, if $\mathcal{A}$ is measured with outcome a_n on a given system, and then is immediately *remeasured*, the results of the second measurement *are not statistically distributed,* the result is always a_n again. Hence this postulate.

- The collapse of the wavepacket preserves the normalization of the state. If $|\psi\rangle$ and $|a_n\rangle$ are normalized, then

$$|\langle a_n|\psi\rangle|^2 \cdot |a_n\rangle.$$

Sixth Postulate

The time evolution of a quantum system preserves the normalization of the associated ket. The time evolution of the state of a quantum system is described by

$$|\psi(t)\rangle = \hat{U}(t, t_0)|\psi(t_0)\rangle,$$

for some unitary operator $\hat{U}$.

Comments

- We have not got to this subject yet. I include it for completeness.
- Under time evolution, a state $|\psi\rangle$ moves through the space of states on a trajectory we can define as $|\psi(t)\rangle$. The preservation of the norm of the state is associated with conservation of probability. If the observable A is energy, for example, then the statement (15) says that the probability to find the system with *some value for the energy* is unity when summed over all possible values. For this to remain true as time goes on, it is necessary for the norm of the state to stay unity.
- Soon we will show that this postulate requires $|\psi\rangle$ to obey a differential equation of the form

$$i\hbar\frac{d}{dt}|\psi(t)\rangle = \mathcal{H}|\psi(t)\rangle \tag{17}$$

where $\mathcal{H}$ is a hermitian operator (we know it as the Hamiltonian). This is Schroedinger's equation written as an operator equation in the space of states (as opposed to a differential equation in the space of wavefunction).

2.2 Purifications

Last week we learned about the partial trace operation, which provides a way to describe the state of a subsystem when given a description of the state on a larger composite system. Even if the state of the larger system is pure, the reduced state can sometimes be mixed, and this is a signature of entanglement in the larger state.

Is it possible to reverse this process? Suppose given a density matrix ρ_A describing a quantum state on system A. Is it always possible to find a pure state $\rho_{AB} = |\Psi\rangle\langle\Psi|_{AB}$ such that $\mathrm{tr}_B(\rho_{AB}) = \rho_A$? Such a state is called a *purification* of ρ_A.

Definition 2.2.1 — Purification. Given any density matrix ρ_A, a pure state $|\Psi_{AB}\rangle$ is a *purification* of A if $\mathrm{tr}_B(|\Psi\rangle\langle\Psi|_{AB}) = \rho_A$.

Let's see how an arbitrary density matrix ρ_A can be purified. As a first step, diagonalize ρ_A, expressing it as a mixture

$$\rho_A = \sum_{j=1}^{d_A} \lambda_j |\phi_j\rangle\langle\phi_j| \,, \tag{2.14}$$

where λ_j are the (necessarily non-negative) eigenvalues of ρ_A and $|\phi_j\rangle$ the eigenstates. Since ρ_A is a density matrix the λ_j are non-negative and sum to 1. We've seen an interpretation of density matrices before: here we would say that ρ_A describes a quantum system that is in a probabilistic mixture of being in state $|\phi_j\rangle$ with probability λ_j. But who "controls" which part of the mixture A is in?

Let's introduce an imaginary system B which achieves just this. Let $\{|j\rangle_B\}_{j\in\{1,\ldots,d_B\}}$ be the standard basis for a system B of dimension $d_B = d_A$, and consider the pure state

$$|\Psi\rangle_{AB} = \sum_{j=1}^{d_A} \sqrt{\lambda_j} |\phi_j\rangle_A \otimes |j\rangle_B \,, \tag{2.15}$$

where $\{|j\rangle_B\}_j$ is the standard basis on system B. Suppose we were to measure the B system of $|\Psi\rangle_{AB}$ in the standard basis. We know what would happen: we will obtain outcome j with probability $\langle\Psi|_{AB} M_j |\Psi\rangle_{AB}$, where $M_j = \mathbb{I}_A \otimes |j\rangle\langle j|_B$, and a short calculation will convince you this equals λ_j. Since we're using a projective measurement, we can describe the post-measurement state easily as being proportional to $M_j|\Psi\rangle\langle\Psi|_{AB}M_j$, and looking at the A system only we find that it is $|\phi_j\rangle\langle\phi_j|_A$.

To summarize, a measurement of system B gives outcome j with probability λ_j, and the post-measurement state on A is precisely $|\phi_j\rangle\langle\phi_j|$. This implies that $\mathrm{Tr}_B(|\Psi\rangle\langle\Psi|_{AB}) = \rho_A$, a fact which can be verified directly using the mathematical definition of the partial trace operation.

Are purifications unique? You'll notice that in the above construction we made the choice of the standard basis for system B, but any other basis would have worked just as well. So it seems like we at least have a choice of basis on system B: there is a "unitary degree of freedom". To see that this is the only freedom that we have in choosing a purification, we first need to learn about a very convenient representation of bipartite pure states, the *Schmidt decomposition*.

2.2.1 The Schmidt decomposition

The purification that we constructed in (2.15) has a special form: it is expressed as a sum, with non-negative coefficients whose squares sum to 1, of tensor products of basis states for the A and B systems respectively. As we saw, this particular form is convenient because it lets us compute the reduced states in A and B very easily. Unfortunately, not every state is always given in this way: for example, if we write $|\Psi\rangle_{AB} = \frac{1}{\sqrt{2}}(|0\rangle_A|0\rangle_B + |+\rangle_A|1\rangle_B)$ then the two states $|0\rangle_A, |+\rangle_A$ on A are not orthogonal. But maybe the same state can be written in a more convenient form? The answer is yes, and it is given by the Schmidt decomposition.

Theorem — Schmidt decomposition. Consider quantum systems A and B with dimensions d_A,d_B respectively, and let $d = \min(d_A,d_B)$. Any pure bipartite state $|\Psi\rangle_{AB}$ has a Schmidt decomposition

$$|\Psi\rangle_{AB} = \sum_{i=1}^{d} \sqrt{\lambda_i}|u_i\rangle_A|v_i\rangle_B, \tag{2.16}$$

where $\lambda_i \geq 0$ and $\{|u_i\rangle_A\}_i, \{|v_i\rangle_B\}_i$ are orthonormal vector sets. The coefficients $\sqrt{\lambda_i}$ are called the *Schmidt coefficients* and $|u_i\rangle_A, |v_i\rangle_B$ the *Schmidt vectors*.

We discussed the proof of the theorem in the video module; you can also find a detailed proof in Section 2.5 of [NC01]. The main idea is to start by expressing $|\Psi\rangle_{AB} = \sum_{j,k} \alpha_{j,k}|j\rangle_A|k\rangle_B$ using the standard bases of A and B, and then write the singular value decomposition of the $d_A \times d_B$ matrix with coefficients $\alpha_{j,k}$ to recover the $\sqrt{\lambda_i}$ (the singular values) and the $|u_i\rangle_A$ (the left eigenvectors) and the $|v_i\rangle_B$ (the right eigenvectors).

The Schmidt decomposition has many interesting consequences. A first consequence is that it provides a simple recipe for computing the reduced density matrices: given a state of the form (2.16), we immediately get $\rho_A = \sum_i \lambda_i |u_i\rangle\langle u_i|_A$, and $\rho_B = \sum_i \lambda_i |v_i\rangle\langle v_i|_B$. An important observation is that ρ_A and ρ_B have the same eigenvalues, which are precisely the squares of the Schmidt coefficients. As a consequence, given any two density matrices ρ_A and ρ_B, there exists a pure bipartite state $|\Psi\rangle_{AB}$ such that $\rho_A = \mathrm{Tr}_B(|\Psi\rangle\langle\Psi|_{AB})$ and $\rho_B = \mathrm{Tr}_A(|\Psi\rangle\langle\Psi|_{AB})$ if and only if ρ_A and ρ_B have the same spectrum! Without the Schmidt decomposition this is not at all an obvious fact to prove.

The same observation also implies that the Schmidt coefficients are uniquely defined: they are the square roots of the eigenvalues of the reduced density matrix. The Schmidt vectors are also unique, up to degeneracy and choice of phase: if an eigenvalue has an associated eigenspace of dimension 1 only then the associated Schmidt vector must be the corresponding eigenvector. If the eigenspace has dimension more than 1 we can choose as Schmidt vectors any basis for the subspace. And note that in (2.16) we can always multiply $|u_i\rangle$ by $e^{i\theta_i}$, and $|v_i\rangle$ by $e^{-i\theta_i}$, so there is a phase degree of freedom.

Another important consequence of the Schmidt decomposition is that it provides us with a way to measure entanglement between the A and B systems in a pure state $|\Psi_{AB}\rangle$. A first, rather rough but convenient such measure is given by the number of non-zero coefficients $\sqrt{\lambda_j}$. This measure is the so-called *Schmidt rank*. If the Schmidt rank is 1 then the state is a product state, and if it is strictly larger than 1 then the state is entangled.

Definition 2.2.2 — Schmidt rank. For any bipartite pure state with Schmidt decomposition $|\Psi\rangle_{AB} = \sum_{i=1}^{d} \sqrt{\lambda_i}|a_i\rangle_A|b_i\rangle_B$, the *Schmidt rank* is defined as the number of non-zero coefficients $\sqrt{\lambda_i}$. It is also equal to $\mathrm{rank}(\rho_A)$ and $\mathrm{rank}(\rho_B)$.

The Schmidt coefficients provide a finer way to measure entanglement than the Schmidt rank. A natural measure, called "entropy of entanglement", consists in taking the entropy of the distribution specified by the squares of the coefficients. If the entropy is 0 then there is only a single coefficient equal to 1, and the state is not entangled. But as soon as the entropy is positive the state is entangled. This measure is finer than the Schmidt rank. For example, it distinguishes the entanglement in the two states

$$|\Psi\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle \qquad \text{and} \qquad |\phi\rangle = \sqrt{1-\varepsilon}|00\rangle + \sqrt{\varepsilon}|11\rangle.$$

Bell-Inequality

A very important one is that it allows correlations between two particles — two qubits — that cannot be replicated classically. The very first example of such correlations was demonstrated in, where Bell proved that the predictions of quantum theory are incompatible with those of any classical theory satisfying a natural notion of locality.

The modern way to understand Bell non-locality is by means of so-called non-local games. Let's imagine that we play a game with two players, which we'll again call Alice and Bob. Alice has a system A, and Bob has some system B. In this game, we will ask Alice and Bob questions, and collect answers. Let us denote the possible questions to Alice and Bob x and y, and label the answers a and b. We will play this game many times, and in each round choose the questions to ask with some probability p(xy). As you might have guessed our little game has some rules. We denote these rules using a predicate V(a,b|x,y), which takes the value "1" if a and b are winning answers for questions x and y. To be fair, Alice and Bob know the rules of the game given by V(a,b|x,y), and also the distribution p(xy). They can agree on any strategy before the game starts. However, once we start asking questions they are no longer allowed to communicate. Of interest to us will be the probability that Alice and Bob win the game, maximized over all possible strategies.

MODULE 3

Quantum Gates and Algorithms

Quantum gates: X, Y, Z, H, S, T, CNOT, SWAP. Quantum circuit design, Universal gates and circuit equivalence, Deutsch's algorithm, Deutsch–Jozsa algorithm, Grover's Search Algorithm, Quantum Fourier Transform (QFT)

Quantum Computing is the area of study focused on developing computing methods based on the principle of quantum theory. Quantum Physics explains the nature and behaviour of energy and matter on the quantum (atomic and subatomic) scale. Elementary particles such as protons, neutrons and electrons can exist in two or more states at a time. This fundamental behaviour is utilized in designing the quantum computation processing units. Quantum computing uses a combination of bits of 1's, 0's and both 1 and 0 at a time to perform computational tasks with greater efficiency. In Quantum computing, the information is encoded in quantum systems such as atoms, ions or quantum dots.

Quantum gates

A quantum gate is a very simple computing device that performs quantum operation on qubits. Quantum gates are one of the essential parts of a quantum computer and are the building blocks of all quantum algorithms. Quantum gates are mathematically represented as transformation matrices which operate on inputs to give outputs.

Quantum gates act on qubits, like logic gates act on bits. In this section, we will explore what quantum gates are.

A quantum gate transforms the state of a qubit into other states. As we will see later, we often use the capital letter U to denote a quantum gate.

Classical Reversible Gates

A classical logic gate is reversible if its outputs are unique. Any classical reversible logic gate simply permutes (shuffles) the amplitudes around. This is not just on a single qubit, but jumping ahead to multiple qubits, if there were more than two amplitudes, a classical reversible logic gate would just permute them, so the state stays normalized. Thus, Classical reversible logic gates are valid quantum gates.

There are different types of quantum gates. Single-qubit gates and multiple qubit gates. These gates can flip a qubit from 0 to 1 as well as allowing superposition states to be created.

Single-Qubit Gates:

Single qubit inputs are $|0\rangle$ to $|1\rangle$ and can be represented by matrix forms as

$$|0\rangle = \begin{bmatrix}1\\0\end{bmatrix} \text{ and } |1\rangle = \begin{bmatrix}0\\1\end{bmatrix}$$

Single Qubit Gates are X- gate, Y-gate, Z- gate, H –gate, S-gate, T-gate.

1. X – Gate or Quantum Not Gate

X-gate is the single qubit input gate and is also called as Pauli X – gate or quantum NOT gate.

The Matrix form of X is given by

$$X = \begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}$$

Action of the X-gate on inputs: When X gate operates of inputs $|0\rangle$, $|1\rangle$, $|\psi\rangle$;

When X operates on $|0\rangle$ and $|1\rangle$ the output will be inverted (ie,$|0\rangle$ becomes $|1\rangle$ and $|1\rangle$ becomes$|0\rangle$)

$$X|0\rangle = \begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}0\\1\end{bmatrix} = |1\rangle$$

$$X|0\rangle = |1\rangle$$

$$X|1\rangle = \begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix} = |0\rangle$$

$$X|1\rangle = |0\rangle$$

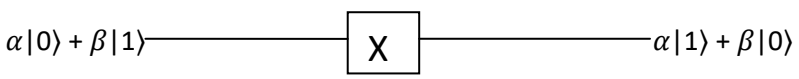
Since X inverts each input it is also called as bit-flip gate.

If a superposed qubit goes through X gate, the result will be

$$X|\psi\rangle = \begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}\begin{bmatrix}\alpha\\\beta\end{bmatrix} = \begin{bmatrix}\beta\\\alpha\end{bmatrix} = \alpha|1\rangle + \beta|0\rangle$$

$$X|\psi\rangle = \alpha|1\rangle + \beta|0\rangle$$

Gate representation is



Truth table is

X -gate	
Input	Output
$\|0\rangle$	$\|1\rangle$
$\|1\rangle$	$\|0\rangle$
$\alpha\|0\rangle + \beta\|1\rangle$	$\alpha\|1\rangle + \beta\|0\rangle$

2. **Y – Gate** : Y-gate is the single qubit input gate. This is also called as Pauli Y – gate.

The matrix form of Y gate is $\mathrm{Y} = \begin{bmatrix} 0 & -\mathrm{i} \\ \mathrm{i} & 0 \end{bmatrix}$

Action of Y gate on inputs : When Y operates on $|0\rangle$ and $|1\rangle$

$$\mathrm{Y}|0\rangle = \begin{bmatrix} 0 & -\mathrm{i} \\ \mathrm{i} & 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0+0 \\ \mathrm{i}+0 \end{bmatrix} = \begin{bmatrix} 0 \\ \mathrm{i} \end{bmatrix} = \mathrm{i}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \mathrm{i}|1\rangle$$

$$\mathrm{Y}|0\rangle = \mathrm{i}|1\rangle$$

$$\mathrm{Y}|1\rangle = \begin{bmatrix} 0 & -\mathrm{i} \\ \mathrm{i} & 0 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0-\mathrm{i} \\ 0+0 \end{bmatrix} = \begin{bmatrix} -\mathrm{i} \\ 0 \end{bmatrix} = -\mathrm{i}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = -\mathrm{i}|0\rangle$$

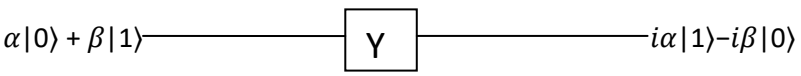
$$\mathrm{Y}|1\rangle = -\mathrm{i}|0\rangle$$

If a superposed qubit goes through Y gate, the result will be

$$\mathrm{Y}|\psi\rangle = \begin{bmatrix} 0 & -\mathrm{i} \\ \mathrm{i} & 0 \end{bmatrix}\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} -\mathrm{i}\beta \\ \mathrm{i}\alpha \end{bmatrix} = -\mathrm{i}\beta|0\rangle + \mathrm{i}\alpha|1\rangle$$

$$\mathrm{Y}|\psi\rangle = -\mathrm{i}\beta|0\rangle + \mathrm{i}\alpha|1\rangle$$

Gate representation is



Truth table is

Y -gate	
Input	Output
$\|0\rangle$	$i\|1\rangle$
$\|1\rangle$	$-i\|0\rangle$
$\alpha\|0\rangle + \beta\|1\rangle$	$i\alpha\|1\rangle - i\beta\|0\rangle$

3. **Z – Gate** : Z-gate is the single qubit input gate. This is also called as Pauli Z – gate.

   The matrix form of Z gate is

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

   Action of Z gate on inputs: When Z operates on $|0\rangle$ and $|1\rangle$ the phase will change. Hence this is also called as phase-flip gate

$$Z|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1+0 \\ 0+0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

$$Z|0\rangle = |0\rangle$$

$$Z|1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0+0 \\ 0-1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -1\begin{bmatrix} 0 \\ 1 \end{bmatrix} = -|1\rangle$$

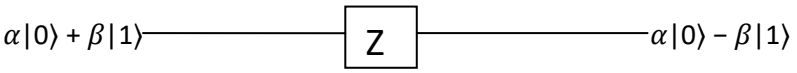
$$Z|1\rangle = -|1\rangle$$

   If a superposed qubit goes through Y gate, the result will be

$$Z|\psi\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha \\ -\beta \end{bmatrix} = \alpha|0\rangle - \beta|1\rangle$$

$$Z|\psi\rangle = \alpha|0\rangle - \beta|1\rangle$$

   Gate representation is

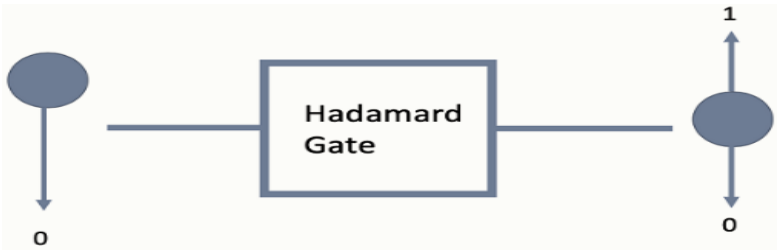


The truth tables for X, Y and Z gates are as follows;

Z-gate	
Input	Output
$\|0\rangle$	$\|0\rangle$
$\|1\rangle$	**-**$\|1\rangle$
$\alpha\|0\rangle + \beta\|1\rangle$	$\alpha\|0\rangle - \beta\|1\rangle$

4. **Hadamard Gate (H-gate)** – It is a single qubit gate and is also gate to superposition. H gate acts on single qubit input and produce superposition state output. Matrix form of H gate and its symbol is

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$



Action of H gate on inputs:

Let us find out what happens when Hadamard gate operates on a qubit that is in the $|0\rangle$ state

$$H|0\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}\begin{bmatrix}1\\ 0\end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\ 1\end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Let us find out what happens when Hadamard gate operates on a qubit that is in the $|1\rangle$ state.

$$H|1\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}\begin{bmatrix}0\\ 1\end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\ -1\end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

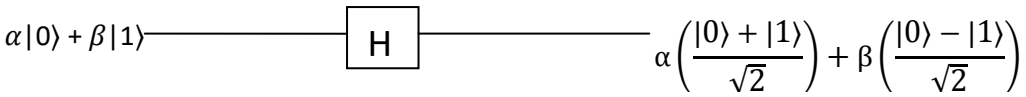
$$H|0\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

If a superposed qubit goes through H gate, the result will be

$$H|\psi\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}\begin{bmatrix}\alpha\\ \beta\end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix}\alpha+\beta\\ \alpha-\beta\end{bmatrix} = \left(\frac{\alpha+\beta}{\sqrt{2}}\right)|0\rangle + \left(\frac{\alpha-\beta}{\sqrt{2}}\right)|1\rangle$$

$$H|\psi\rangle = \alpha\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) + \beta\left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right)$$

Gate representation is



The truth table is as follows

Input	**Output**
$\|0\rangle$	$\frac{\|0\rangle + \|1\rangle}{\sqrt{2}}$
$\|1\rangle$	$\frac{\|0\rangle - \|1\rangle}{\sqrt{2}}$
$\alpha\|0\rangle + \beta\|1\rangle$	$\alpha\left(\frac{\|0\rangle + \|1\rangle}{\sqrt{2}}\right) + \beta\left(\frac{\|0\rangle - \|1\rangle}{\sqrt{2}}\right)$

Note: Difference between X, Y, Z and H gates is that in X, Y and Z gates, output is in single state whereas in H gate output is superposed state.

5. **Phase Gate (S Gate):**

It is a single qubit gate. The Phase gate or S gate is a gate that transfers $|0\rangle$ into $|0\rangle$ and $|1\rangle$ into $i|1\rangle$. The matrix form of S gate is

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$$

Action of S gate on inputs:

Consider S gate apply to a state $|0\rangle$ it will remain same

$$S|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

$$S|0\rangle = |0\rangle$$

If S gate apply to a state $|1\rangle$ it will be transformed into $i|1\rangle$

$$S|1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = i \begin{bmatrix} 0 \\ 1 \end{bmatrix} = i|1\rangle$$

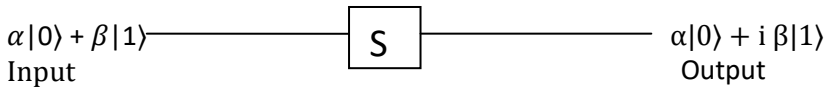
$$S|1\rangle = i|1\rangle$$

S gate apply to the state $\alpha|0\rangle + \beta|1\rangle$ it transforms to the state $\alpha|0\rangle + i\beta|1\rangle$

$$S|\psi\rangle = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha \\ i\beta \end{bmatrix}$$

$$S|\psi\rangle = \alpha|0\rangle + i\,\beta|1\rangle$$

The S gate representation is as follows



The truth table is as follows

Input	**Output**
$\|0\rangle$	$\|0\rangle$
$\|1\rangle$	$i\|1\rangle$
$\alpha\|0\rangle + \beta\|1\rangle$	$\alpha\|0\rangle + i\,\beta\|1\rangle$

6. **T- Gate** : It is a single qubit gate and it is also called π/8 gate.

Its matrix form is

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$$

Action of T gate on inputs:

If T gate operates on input is $|0\rangle$ then the output is also $|0\rangle$

$$T|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

$$T|0\rangle = |0\rangle$$

If T gate operates on input is $|1\rangle$ then the output is also $e^{i\pi/4}|1\rangle$

$$T|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = e^{i\pi/4} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = e^{i\pi/4}|1\rangle$$

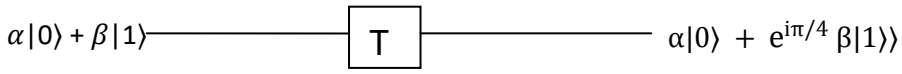
$$T|0\rangle = e^{i\pi/4}|1\rangle$$

If T gate operates on superposed state $\alpha|0\rangle + \beta|1\rangle$, It transforms to $\alpha|0\rangle + e^{i\pi/4}\beta|1\rangle$

$$T|\psi\rangle = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{i\pi}{4}} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta e^{\frac{i\pi}{4}} \end{bmatrix} = \alpha|0\rangle + e^{i\pi/4}\beta|1\rangle$$

$$T|\psi\rangle = \alpha|0\rangle + e^{i\pi/4}\beta|1\rangle$$

T Gate representation is



The truth table is as follows

Input	**Output**
$\|0\rangle$	$\|0\rangle$
$\|1\rangle$	$e^{i\pi/4}\|1\rangle$
$\alpha\|0\rangle + \beta\|1\rangle$	$\alpha\|0\rangle + e^{i\pi/4}\beta\|1\rangle$

Multiple Qubit gates: Quantum gates operating on multiple qubits are called as multiple qubit gates. Multiple Qubit Gates operate on Two or More input Qubits. Multiple qubit consists of control gate and target gate. The action of gate as follows;

i) The Target qubit is altered only when the control qubit is $|1\rangle$, and

ii) The control qubit remains unaltered during the transformations.

For two qubits, inputs qubits are $|00\rangle, |01\rangle, |10\rangle$ and $|11\rangle$

For three qubits, inputs qubits are $|000\rangle$, $|001\rangle$, $|010\rangle$, $|011\rangle$, $|100\rangle$, $|101\rangle$, $|110\rangle$ and $|111\rangle$

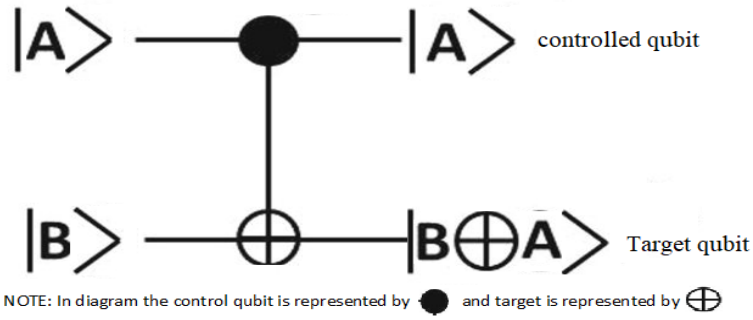
In multiple qubit gate, the input qubit applied in the form like $|AB\rangle$, first term $|A\rangle$ goes to control qubit and the second term $|B\rangle$ goes to target qubit.

Some of the multiple qubit gate are as follows; Controlled gate (CNOT gate), Swap gate, Controlled Z gate and Toffoli gate (CCNOT gate)

1. Controlled Gate (CNOT)

The CNOT gate is a two-qubit operation, where the first qubit is referred as the control qubit $|A\rangle$ and the second qubit as the target qubit $|B\rangle$. If the control qubit is $|1\rangle$ then it will flip the target qubit state from $|0\rangle$ to $|1\rangle$ or from $|1\rangle$ to $|0\rangle$. When the control qubit is in state $|0\rangle$ then the target qubit remains unchanged.

The symbolic representation is as follows. The upper line represents control qubit and bottom line represents target qubit.



In the combined qubit, first term is control qubit and the second term is target qubit. For ex, in $|AB\rangle$, A is control qubit and B is target qubit

Matrix form of **CNOT** Gate is given by

$$\mathbf{CNOT} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} \end{bmatrix}$$

The Transformation could be expressed as $|A, B\rangle \rightarrow |A, B \oplus A\rangle$

Action of the gate: Consider the operations of CNOT gate on the four inputs $|00\rangle, |01\rangle, |10\rangle$ and $|11\rangle$.

i) Operation of CNOT Gate for input $|00\rangle$: $|00\rangle \rightarrow |00\rangle$

Here, control qubit is $|0\rangle$. Hence no change in the state of Target qubit $|0\rangle$.

ii) Operation of CNOT Gate for input |01⟩: |01⟩ → |01⟩

Here, control qubit is |0⟩. Hence no change in the state of Target qubit |1⟩

iii) Operation of CNOT Gate for input |10⟩: |10⟩ → |11⟩

Here, control qubit is |1⟩. Hence the state of Target qubit flips from |0⟩ to |1⟩.

iv) Operation of CNOT Gate for input |11⟩: |11⟩ → |10⟩

Here control qubit is |1⟩. Hence the state of Target qubit flips from |1⟩ to |0⟩.

Input		Output	
$\|A\rangle$	$\|B\rangle$	$\|A\rangle$	$\|B \oplus A\rangle$
$\|0\rangle$	$\|0\rangle$	$\|0\rangle$	$\|0\rangle$
$\|0\rangle$	$\|1\rangle$	$\|0\rangle$	$\|1\rangle$
$\|1\rangle$	$\|0\rangle$	$\|1\rangle$	$\|1\rangle$
$\|1\rangle$	$\|1\rangle$	$\|1\rangle$	$\|0\rangle$

The Truth Table of operation of CNOT gate is as follows.

Input	**Output**
$\|00\rangle$	$\|00\rangle$
$\|01\rangle$	$\|01\rangle$
$\|10\rangle$	$\|11\rangle$
$\|11\rangle$	$\|10\rangle$

2. Swap Gate:

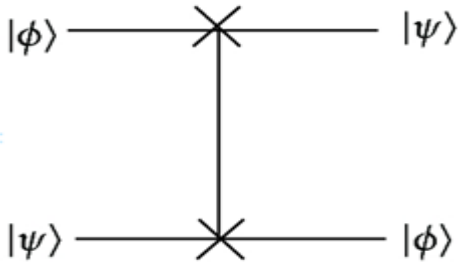
SWAP gate is a two qubit operation gate and swaps the state of the two qubits involved in the operation. It contains 3 CNOT gates.

The Matrix representation of the Swap Gate is as follows

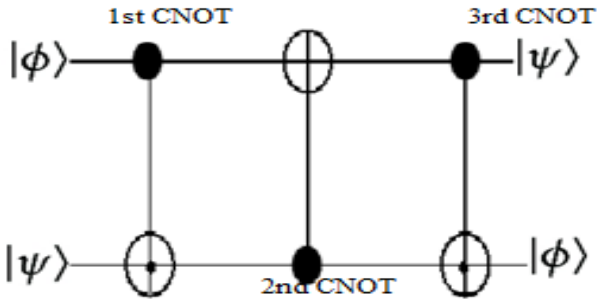
$$\mathbf{SWAP} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix}$$

The schematic symbol of swap gate circuit and its equivalent is shown in figure below.

The swap gate is a combined circuit of 3 CNOT gates (1st CNOT gate, 2nd CONT gate and 3rd CNOT gate) and the overall effect is that two input qubits are swapped at the output. Gate representation is



The Action of the swap gate is as follows.



v) Operation of SWAP Gate for input |00⟩: |00⟩ → |00⟩

vi) Operation of SWAP Gate for input |01⟩: |01⟩ → |01⟩

vii) Operation of SWAP Gate for input |10⟩: |10⟩ → |11⟩

viii) Operation of SWAP Gate for input |11⟩: |11⟩ → |10⟩

Gate	Input to the gate	Output of the gate
1	$\|a, b\rangle$	$\|a, a \oplus b\rangle$
2	$\|a, a \oplus b\rangle$	$\|b, a \oplus b\rangle$
3	$\|b, a \oplus b\rangle$	$\|b, a\rangle$

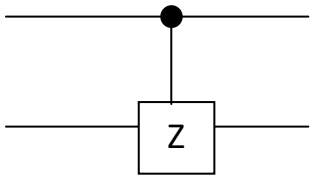
Truth table of the swap gate

Input	Output
$\|00\rangle$	$\|00\rangle$
$\|01\rangle$	$\|10\rangle$
$\|10\rangle$	$\|01\rangle$
$\|11\rangle$	$\|11\rangle$

3. **Controlled Z Gate**: In Controlled Z Gate, The operation of Z Gate is controlled by a Control Qubit. If the control Qubit is $|A\rangle = |1\rangle$ then only the Z gate transforms the Target Qubit $|B\rangle$ as per the Pauli-Z operation. The action of Controlled Z-Gate could is specified by a matrix as follows.

$$\textbf{Controlled Z gate} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{-1} \end{bmatrix}$$

The controlled Z gate representation as follows.



Action of controlled Z gate is

ix) Operation of the Gate for input $|00\rangle$: $|00\rangle \rightarrow |00\rangle$

Control qubit is $|0\rangle$, Z gate remains unchanged.

x) Operation of the Gate for input $|01\rangle$: $|01\rangle \rightarrow |01\rangle$

Control qubit is $|0\rangle$, Z gate remains unchanged.

xi) Operation of the Gate for input $|10\rangle$: $|10\rangle \rightarrow |10\rangle$

Control qubit is $|1\rangle$, Z gate remains unchanged.

xii) Operation of the Gate for input $|11\rangle$: $|11\rangle \rightarrow -|11\rangle$

Control qubit is $|1\rangle$, Z gate flips the state from $|1\rangle$ to $-|1\rangle$.

Truth table of the controlled Z gate

Input	**Output**
$\|00\rangle$	$\|00\rangle$
$\|01\rangle$	$\|01\rangle$
$\|10\rangle$	$\|10\rangle$
$\|11\rangle$	$-\|11\rangle$

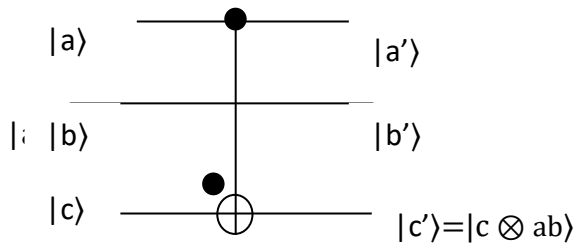
4. Toffoli Gate:

The Toffoli Gate is also known as CCNOT Gate (Controlled-Controlled-Not). It has three inputs out of which two are Control Qubits and one is the Target Qubit. The Target Qubit flips only when both the Control Qubits are $|1\rangle$. The two Control Qubits are not altered during the operation.

The matrix representation the Gate is,.

$$\textbf{Toffoli Gate} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Gate representation:



Truth Table of Toffoli Gate.

Input to the gate $\|abc\rangle$	**Output of the gate** $\|a'b'c'\rangle$
$\|100\rangle$	$\|100\rangle$
$\|001\rangle$	$\|001\rangle$
$\|010\rangle$	$\|010\rangle$
$\|011\rangle$	$\|011\rangle$
$\|100\rangle$	$\|100\rangle$
$\|101\rangle$	$\|101\rangle$
$\|110\rangle$	$\|111\rangle$
$\|111\rangle$	$\|110\rangle$

Quantum circuit design

A simple quantum circuit containing three quantum gates is shown in Figure. The circuit is to be read from left-to-right. Each line in the circuit represents a wire in the quantum circuit. This wire does not necessarily correspond to a physical wire; it may correspond instead to the passage of time, or perhaps to a physical particle such as a photon– a particle of light– moving from one location to another through space. It is conventional to assume that the state input to the circuit is a computational basis state, usually the state consisting of all |0s. This rule is broken frequently in the literature on quantum computation and quantum information, but it is considered polite to inform the reader when this is the case. The circuit in Figure accomplishes a simple but useful task– it swaps the states of the two qubits. To see that this circuit accomplishes the swap operation, note that the sequence of gates has the following sequence of effects on a computational basis state |a,b >,

$$\begin{aligned} |a,b\rangle &\longrightarrow |a, a \oplus b\rangle \\ &\longrightarrow |a \oplus (a \oplus b), a \oplus b\rangle = |b, a \oplus b\rangle \\ &\longrightarrow |b, (a \oplus b) \oplus b\rangle = |b, a\rangle \, , \end{aligned}$$

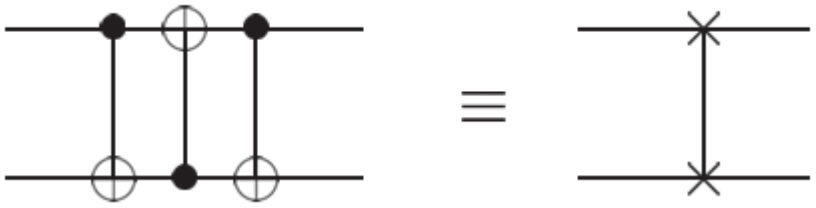


Figure Circuit swapping two qubits, and an equivalent schematic symbol notation for this common and useful circuit.

There are a few features allowed in classical circuits that are not usually present in quantum circuits. First of all, we don't allow 'loops', that is, feedback from one part of the quantum circuit to another; we say the circuit is acyclic. Second, classical circuits allow wires to be 'joined' together, an operation known as , with the resulting single wire containing the bitwise of the inputs. Obviously this operation is not reversible and therefore not unitary, so we don't allow it in our quantum circuits. Third, the inverse operation , whereby several copies of a bit are produced is also not allowed in quantum circuits. In fact, it turns out that quantum mechanics forbids the copying of a qubit, making the operation impossible! We'll see an example of this in the next section when we attempt to design a circuit to copy a qubit.

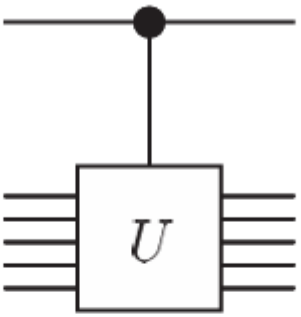


Figure 1.8. Controlled-U gate.

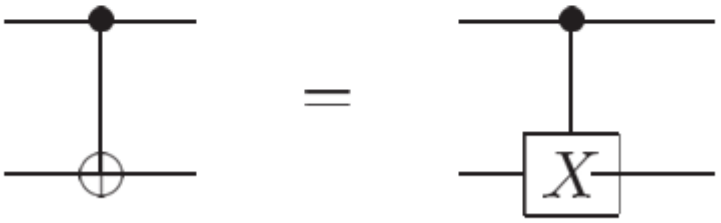


Figure 1.9. Two different representations for the controlled-NOT.

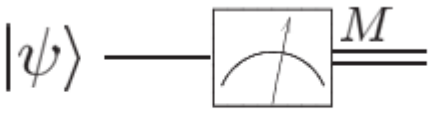
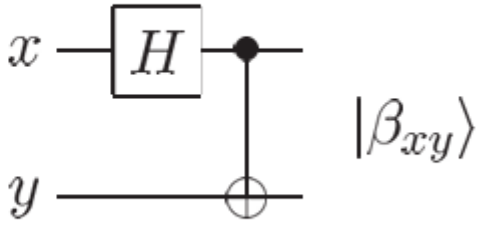


Figure 1.10. Quantum circuit symbol for measurement.

Example: Bell states

Let's consider a slightly more complicated circuit, shown in Figure below, which has a Hadamard gate followed by a , and transforms the four computational basis states according to the table given.

In	Out
$\|00\rangle$	$(\|00\rangle + \|11\rangle)/\sqrt{2} \equiv \|\beta_{00}\rangle$
$\|01\rangle$	$(\|01\rangle + \|10\rangle)/\sqrt{2} \equiv \|\beta_{01}\rangle$
$\|10\rangle$	$(\|00\rangle - \|11\rangle)/\sqrt{2} \equiv \|\beta_{10}\rangle$
$\|11\rangle$	$(\|01\rangle - \|10\rangle)/\sqrt{2} \equiv \|\beta_{11}\rangle$



As an explicit example, the Hadamard gate takes the input |00 to (|0 + |1)|0/√2, and then gives the output state (|00 + |11)/√2. Note how this works: first, the Hadamard transform puts the top qubit in a superposition; this then acts as a control input to the , and the target gets inverted only when the control is 1. The output states are known as the Bell states, or sometimes the EPR states or EPR pairs.

$$|\beta_{00}\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}};$$
$$|\beta_{01}\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}};$$
$$|\beta_{10}\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}; \text{ and}$$
$$|\beta_{11}\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}},$$

The mnemonic notation |β00> ,|β01> ,|β10> ,|β11> may be understood via the equations

$$|\beta_{xy}\rangle \equiv \frac{|0, y\rangle + (-1)^x |1, \bar{y}\rangle}{\sqrt{2}}.$$

Example :

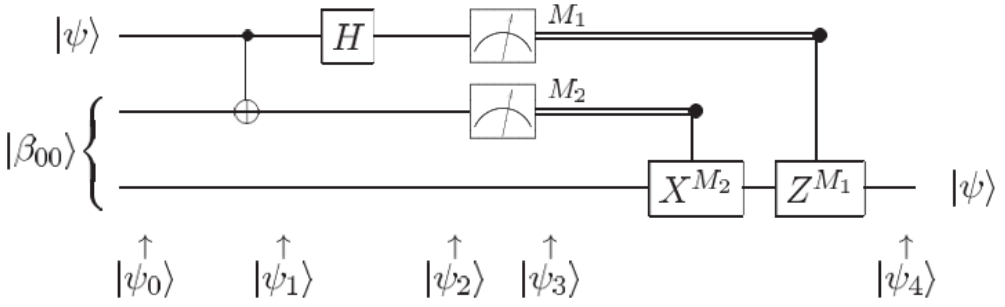


Figure. Quantum circuit for teleporting a qubit.

Deutsch-Jozsa algorithm

In the previous lecture we discussed Deutsch's Algorithm, which gives a simple example of how quantum algorithms can give some advantages over classical algorithms in certain restricted settings. We will start this lecture by discussing another simple example, and then move on to the Deutsch-Jozsa Algorithm, which generalizes Deutsch's Algorithm to functions from n bits to 1 bit.

A simple searching problem

Suppose that we are given a function of the form $f : \{0,1\}^2 \rightarrow \{0,1\}$, but this time we are *promised* that it is one of these four functions:

f_{00}

input	output
00	1
01	0
10	0
11	0

f_{01}

input	output
00	0
01	1
10	0
11	0

f_{10}

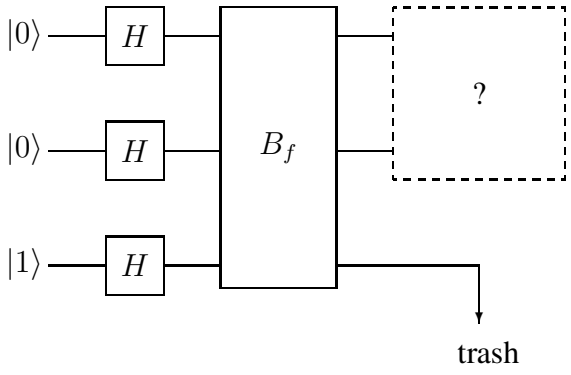
input	output
00	0
01	0
10	1
11	0

f_{11}

input	output
00	0
01	0
10	0
11	1

The goal is to determine which one it is. In other words, the function takes value 1 on one input and value 0 on all others, and the goal is to find the input on which the function takes value 1. Thus, we can think of this problem as representing a simple searching problem. As before, we assume that access to the function is restricted to evaluations of the transformation B_f. Now is a good time to mention some terminology: we say that an evaluation of B_f on some input (quantum or classical) is a *query*.

Classically, three queries are necessary and sufficient to solve the problem. In the quantum case, one query will be sufficient to solve the problem. Here is a circuit diagram describing part of the procedure. (The missing part will be filled in later.)



Let us analyze the state of the above circuit at various instants to try and determine what (if anything) would work for the omitted part of the circuit. The state after the Hadamard transforms can be written

$$\left(\frac{1}{2}|00\rangle + \frac{1}{2}|01\rangle + \frac{1}{2}|10\rangle + \frac{1}{2}|11\rangle\right)\left(\frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle\right).$$

After the transformation B_f is performed, I claim that the state is

$$\left(\frac{1}{2}(-1)^{f(00)}|00\rangle + \frac{1}{2}(-1)^{f(01)}|01\rangle + \frac{1}{2}(-1)^{f(10)}|10\rangle + \frac{1}{2}(-1)^{f(11)}|11\rangle\right)\left(\frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle\right).$$

The reasoning is the same as in the analysis of Deutsch's Algorithm—we are again using the *phase kickback* effect.

The last qubit is independent of the first two, so when it goes in the trash we are left with one of these four possibilities for the state of the first two qubits:

$$\begin{aligned}
f = f_{00} &\Rightarrow |\phi_{00}\rangle = -\frac{1}{2}|00\rangle + \frac{1}{2}|01\rangle + \frac{1}{2}|10\rangle + \frac{1}{2}|11\rangle \\
f = f_{01} &\Rightarrow |\phi_{01}\rangle = \frac{1}{2}|00\rangle - \frac{1}{2}|01\rangle + \frac{1}{2}|10\rangle + \frac{1}{2}|11\rangle \\
f = f_{10} &\Rightarrow |\phi_{10}\rangle = \frac{1}{2}|00\rangle + \frac{1}{2}|01\rangle - \frac{1}{2}|10\rangle + \frac{1}{2}|11\rangle \\
f = f_{11} &\Rightarrow |\phi_{11}\rangle = \frac{1}{2}|00\rangle + \frac{1}{2}|01\rangle + \frac{1}{2}|10\rangle - \frac{1}{2}|11\rangle
\end{aligned}$$

There is something special about these four states: they form an *orthonormal set*. This means that they are unit vectors and are *pairwise orthogonal*:

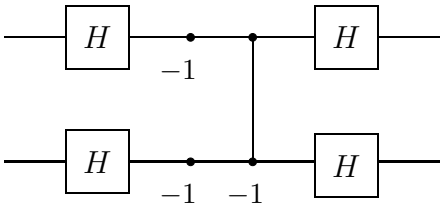
$$\langle\phi_{ab}|\phi_{cd}\rangle = \begin{cases} 1 & \text{if } a = c \text{ and } b = d \\ 0 & \text{otherwise.} \end{cases}$$

Whenever you have an orthonormal set like this, it is always possible to build a quantum circuit that exactly distinguishes the states. In fact, the corresponding unitary transformation is easy to obtain: you let the vectors form the columns of a matrix and then take the conjugate transform. In this case we want this matrix:

$$U = \begin{pmatrix} -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \end{pmatrix}.$$

You can check that $U|\phi_{ab}\rangle = |ab\rangle$ for all four choices of $a, b \in \{0, 1\}$.

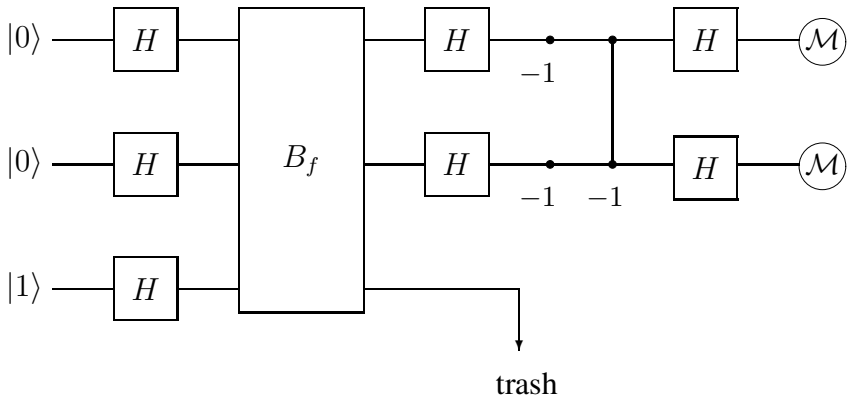
In case you are interested, it is possible to build a small circuit that uses gates that we have already seen to perform the unitary transformation U:



The gate consisting of a single circle labeled -1 corresponds to the matrix

$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

which is also called a *phase flip* or σ_z gate. The complete circuit looks like this:



The Deutsch-Jozsa Algorithm

Our next algorithm is a generalization of Deutsch's Algorithm called the Deutsch-Jozsa Algorithm. Although the computational problem being solved is still artificial and perhaps not particularly impressive, we are moving toward more interesting problems.

This time we assume that we are given a function $f : \{0,1\}^n \to \{0,1\}$, where n is some arbitrary positive integer, and we are promised that one of two possibilities holds:

1. f is **constant**. In other words, either $f(x) = 0$ for all $x \in \{0,1\}^n$ or $f(x) = 1$ for all $x \in \{0,1\}^n$.
2. f is **balanced**. This means that the number of inputs $x \in \{0,1\}^n$ for which the function takes values 0 and 1 are the same:

$$|\{x \in \{0,1\}^n : f(x) = 0\}| = |\{x \in \{0,1\}^n : f(x) = 1\}| = 2^{n-1}.$$

The goal is to determine which of the two possibilities holds.

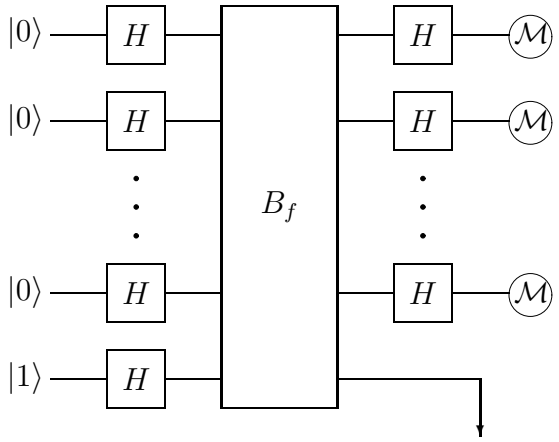
As for the previous two algorithms, we assume that access to the function f is restricted to queries to a device corresponding to the transformation B_f defined similarly to before:

$$B_f |x\rangle |b\rangle = |x\rangle |b \oplus f(x)\rangle$$

for all $x \in \{0,1\}^n$ and $b \in \{0,1\}$.

It turns out that classically this problem is pretty easy given a small number of queries if we allow randomness and accept that there may be a small probability of error. Specifically, we can randomly choose say k inputs $x_1, \ldots, x_k \in \{0,1\}^n$, evaluate $f(x_i)$ for $i = 1, \ldots, k$, and answer "constant" if $f(x_1) = \cdots = f(x_k)$ and "balanced" otherwise. If the function really was constant this method will be correct every time, and if the function was balanced, the algorithm will be wrong (and answer "constant") with probability $2^{-(k-1)}$. Taking $k = 11$, say, we get that the probability of error is smaller than $1/1000$. However, if you demand that the algorithm is correct every time, then $2^{n-1} + 1$ queries are needed in the worst case.

In the quantum case, 1 query will be sufficient to determine with certainty whether the function is constant or balanced. Here is the algorithm, which is called the *Deutsch-Jozsa Algorithm*:



There are n bits resulting from the measurements. If all n measurement results are 0, we conclude that the function was constant. Otherwise, if at least one of the measurement outcomes is 1, we conclude that the function was balanced.

Before we analyze the algorithm, it will be helpful to think more about Hadamard transforms. We have already observed that for $a \in \{0,1\}$ we have

$$H |a\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} (-1)^a |1\rangle ,$$

which we can also write as

$$H |a\rangle = \frac{1}{\sqrt{2}} \sum_{b \in \{0,1\}} (-1)^{ab} |b\rangle .$$

If we instead had two qubits, starting in state $|x\rangle$ for $x = x_1x_2 \in \{0,1\}^2$, and applied Hadamard transforms to both, we would obtain

$$\begin{aligned}(H \otimes H)|x\rangle &= \left(\frac{1}{\sqrt{2}} \sum_{y_1 \in \{0,1\}} (-1)^{x_1 y_1} |y_1\rangle\right) \left(\frac{1}{\sqrt{2}} \sum_{y_2 \in \{0,1\}} (-1)^{x_2 y_2} |y_2\rangle\right) \\ &= \frac{1}{2} \sum_{y \in \{0,1\}^2} (-1)^{x_1 y_1 + x_2 y_2} |y\rangle .\end{aligned}$$

The pattern generalizes to any number of qubits. Writing $H^{\otimes n}$ to mean $H \otimes \cdots \otimes H$ (n times), we have

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x_1 y_1 + \cdots + x_n y_n} |y\rangle$$

for every $x \in \{0,1\}^n$. We also use the shorthand

$$x \cdot y = \sum_{i=1}^{n} x_i y_i \pmod 2$$

so that we may write

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle .$$

The fact that $x \cdot y$ is defined modulo 2 is irrelevant for the previous expression (because $x \cdot y$ appears as an exponent of -1), but it will be convenient later to use this shorthand again and have the quantity defined modulo 2.

Using these formulas, the Deutsch-Jozsa Algorithm becomes easier to analyze. The state after the first layer of Hadamard transforms is performed is

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \left(\frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle\right).$$

When the B_f transformation is performed, the state will change to

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \left(\frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle\right).$$

Once again we are seeing the *phase kick-back* effect. Now, the last qubit is discarded and n Hadamard transforms are applied. The resulting state is

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \left(\frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle\right),$$

which simplifies to

$$\sum_{y\in\{0,1\}^n} \left(\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{f(x)+x\cdot y} \right) |y\rangle \, .$$

Now, all that we really care about is the probability that the measurements all give outcome 0. The amplitude associated with the classical state $|0^n\rangle$ is

$$\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}.$$

Thus, the probability that the measurements all give outcome 0 is

$$\left| \frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{f(x)} \right|^2 = \begin{cases} 1 & \text{if } f \text{ is constant} \\ 0 & \text{if } f \text{ is balanced,} \end{cases}$$

so the algorithm works as claimed.

Grover's Algorithm

In the previous algor we discussed how a quantum state may be transferred via quantum teleportation and how quantum mechanics is, in a sense, more powerful than classical mechanics in the CHSH game. Today, we are going to move on to discussing how we can apply quantum mechanics to computer science. We will see our first example of a quantum algorithm: Grover's algorithm, described in a paper published by Lov Grover in 1996 .[Gro96] Much of the excitement over quantum computation comes from how quantum algorithms can provide improvements over classical algorithms, and Grover's algorithm exhibits a quadratic speedup.

What it solves

Grover's algorithm solves the problem of *unstructured search*.

Definition 2.1. In an *unstructured search* problem, given a set of N elements forming a set $X = \{x_1, x_2, \ldots, x_N\}$ and given a boolean function $f : X \to \{0, 1\}$, the goal is to find an element x^* in X such that $f(x^*) = 1$.

Unstructured search is often alternatively formulated as a database search problem in which we are given a database and we want to find an item that meets some specification. For example, given a database of N names, we might want to find where your name is located in the database.

The search is called "unstructured" because we are given no guarantees as to how the database is ordered. If we were given a sorted database, for instance, then we could perform binary search to find an element in logarithmic time. Instead, we have no prior knowledge about the contents of the database. With classical circuits, we cannot do better than performing a linear number of queries to find the target element.

Grover's algorithm leads to the following theorem:

Theorem 2.2. *The unstructured search problem can be solved in* $\mathcal{O}($

—

In fact, this is within a constant factor of the best we can do for this problem under the quantum computation model, i.e. the query complexity is $\Theta(\sqrt{N})$ [BBBV97]. In particular, since we cannot solve the unstructured search problem in logarithmic time, this problem cannot be used as a way to solve NP problems in polynomial time; if we did have a logarithmic time algorithm, we could, given n variables arranged in clauses, solve the SAT problem by searching all 2^n possibilities in $\mathcal{O}(\log(2^n)) = \mathcal{O}(n)$ queries after a few manipulations. Nonetheless, solving the search problem in $\Theta(\sqrt{N})$ queries is still significantly better than what we can do in the classical case.

In addition, Grover's algorithm uses only $\mathcal{O}(\sqrt{N}\log N)$ gates. If we think of the number of gates as being roughly running time, then we have running time almost proportional to $\sqrt{N}$.

There is a caveat: the algorithm only gets the correct answer with high probability, say with probability greater than $\frac{2}{3}$. Of course, we can then make the probability of correctness an arbitrarily large constant. We do this by running the algorithm multiple times to get many different outputs. Then we can run each of our inputs through f to see if any match the criterion desired. This strategy only fails if all outputs fail, and since these are all independent outputs, our probability of failure is $\left(\frac{1}{3}\right)^m$.

Note that even with classical algorithms that only need to output the correct answer with two-thirds probability, we still need a linear number of queries. Therefore, this caveat does not provide an unfair advantage to the quantum algorithm.

The classical case

We need some sort of model for how we are allowed to interact with our database. In a classical algorithm, our medium of interaction is an *oracle* O_f that *implements* a function f.
The function f encodes information about the element of interest, and we query the oracle to fetch results from the function. Think of the oracle as a quantum circuit or a big gate. This oracle, given an input of i, outputs $f(i)$ in constant time. We do not care about the details of how it works, and instead treat it as a black box.

As we do with any good algorithm, we want to limit resources used, which in this case are running time and number of queries to the oracle. Of course, in any classical algorithm, the best we can do is $\Theta(N)$ queries and $\Theta(N)$ running time – because the data is unstructured, we cannot help but look at every element in the worst case no matter how cleverly we choose our queries.

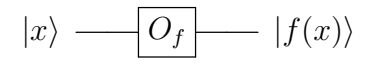
(Note that, in general, number of queries and running time are not the same, but in this case they are both linear. We care more about queries because they are simpler to reason about, plus they represent a lower bound to running time. Running time by itself involves many other factors that we do not know much about. In this course, we will be more concerned about number of queries as a measure of resources used.)

This is the end of the story in the classical situation; we have tight upper and lower bounds, and not much more can be done. Now we shall switch to the quantum setting.

Modifications in the quantum case

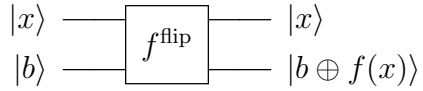
Right off the bat, we're going make some simplifications. First, we are going to assume that the element x^* that satisfies $f(x^*) = 1$ is unique. Second, we are going to assume the size of our database is is a power of two, letting $N = 2^n$ where N is the size of the database. Lastly, we are also going assume that the data is labeled as n-bit boolean strings in $\{0, 1\}^n$ rather than being indexed from 1 to N. In turn, our boolean function f that we are studying will map $\{0, 1\}^n$ to $\{0, 1\}$. These conditions will be more convenient for us, and we do not lose much; the algorithm can be extended to relax the uniqueness assumption, we can always
add garbage entries to the database to make the size of the database a power of two, and the naming of the elements is arbitrary.

Our interface with our database is not much different compared to the interface in the classical case. We have an oracle O_f that we give $|x\rangle$, and it will output $|f(x)\rangle$, as shown below:



We immediately see a problem: this oracle gate is not a valid quantum gate. It takes an n-bit input and gives a 1-bit output, and thus is not unitary or even reversible. There are a couple of ways we can fix this.

Our first possible fix is to give the gate an extra bit $|b\rangle$ to use for the output. Call this new gate the f^{flip} gate.

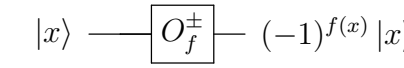


In particular, if we have $|b\rangle$ be an ancillary $|0\rangle$ qubit, it will end up as $|f(x)\rangle$ after passing through the gate. However, this gate is somewhat unwieldy because we have to carry around that extra qubit $|b\rangle$ with us. That brings us to our next fix.

Our second possible fix is to flip the input if and only if $f(x)$ is 1. We will call the gate that does this $O_f^{\pm}$. Given an input $|x\rangle$, $O_f^{\pm}$ will output

$$(-1)^{f(x)}|x\rangle = \begin{cases} |x\rangle & \text{if } f(x) = 0 \\ -|x\rangle & \text{if } f(x) = 1. \end{cases}$$

The diagram of the gate is given below.



This avoids the annoyance of the extra bit that our first fix had.

These two gates are both valid versions of the oracle, and they are equivalent in the sense that one can be constructed from the other. For example, we will prove that $O_f^{\pm}$ may be constructed using f^{flip}.

Proof. Suppose we wanted to construct the $O_f^{\pm}$ gate given access to the f^{flip} gate. Then we choose our ancillary bit that we send along with $|x\rangle$ into f^{flip} to be $|-\rangle$.

Before we hit the f^{flip} gate, we have the state

$$|x\rangle \oplus |-\rangle = |x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{1}{\sqrt{2}}(|x0\rangle - |x1\rangle).$$

After we pass the gate, there are two cases. If $f(x) = 0$, then the state is unchanged. Otherwise, if $f(x) = 1$, then we end up with the state

$$|x\rangle \otimes \frac{1}{\sqrt{2}}(|0 \oplus 1\rangle - |1 \oplus 1\rangle) = |x\rangle \otimes \frac{1}{\sqrt{2}}(|1\rangle - |0\rangle) = \frac{1}{\sqrt{2}}(|x1\rangle - |x0\rangle)$$

which is the negative of the state we began with. Thus we have

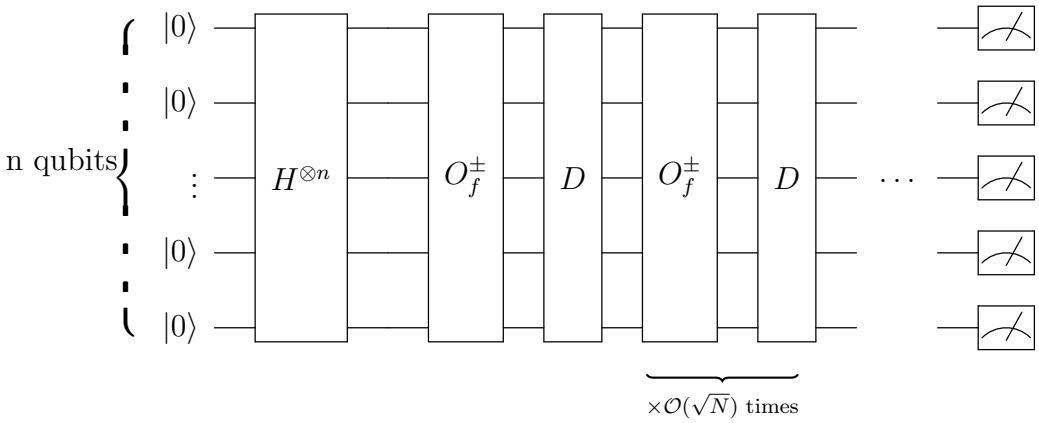
$$|x\rangle \otimes |-\rangle \mapsto (-1)^{f(x)} (|x\rangle \otimes |-\rangle)$$

a desired to simulate the $O_f^{\pm}$ gate. □

Because the gates are equivalent, we may use either. For the circuits in this lecture, we will use the $O_f^{\pm}$ gate.

The algorithm

Recall that our goal is to, given the ability to query f using our oracle gate, find x^* such that $f(x^*) = 1$. We shall start by diagramming the entire circuit below and explain the details following that.



We start with n qubits all initialized to $|0\rangle$. Think of these as forming an n-bit string, or an input to f. Let us make our goal concrete by saying that we would like the amplitude of $|x^*\rangle$ to be at least .1 in magnitude at the end of the circuit so that when we measure at the end, we have a $.1^2 = .01$ probability of measuring $|x^*\rangle$.

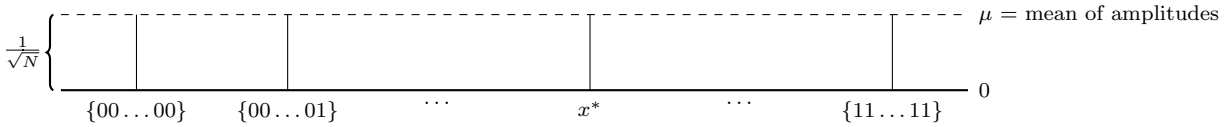
What can we do with our quantum circuit that is not possible classically? We can perhaps exploit superposition. We shall begin by transforming our input into the uniform superposition

$$\sum_{x \in \{0,1\}^n} \frac{1}{\sqrt{N}} |x\rangle.$$

The uniform amplitudes of all the n-bit strings, in a sense, represents our current complete uncertainty as to what x^* is.

We achieve this uniform superposition of all the basis states by sticking a Hadamard gate on every wire, or equivalently a $H^{\otimes n}$ gate on all wires. This superposition trick is common in quantum algorithms, so get used to it.

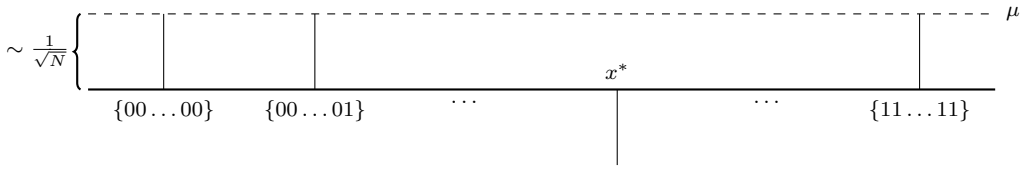
The current state can be visualized with a diagram representing amplitudes in a fashion similar to a bar graph.



Now we shall query this state by adding an oracle $O_f^{\pm}$ gate. That flips the amplitude of the x^* component and leaves everything else unchanged, giving us a state of

$$-\frac{1}{\sqrt{N}}\left|x^*\right\rangle + \sum_{\substack{x\in\{0,1\}^n \\ x\neq x^*}} \frac{1}{\sqrt{N}}\left|x\right\rangle .$$

Graphically, we now have



What we want is to increase the amplitude of x^* absolutely. This motivates the introduction of a new gate that performs what is called the *Grover diffusion operator*. What does this gate do? First, we have to define a new number

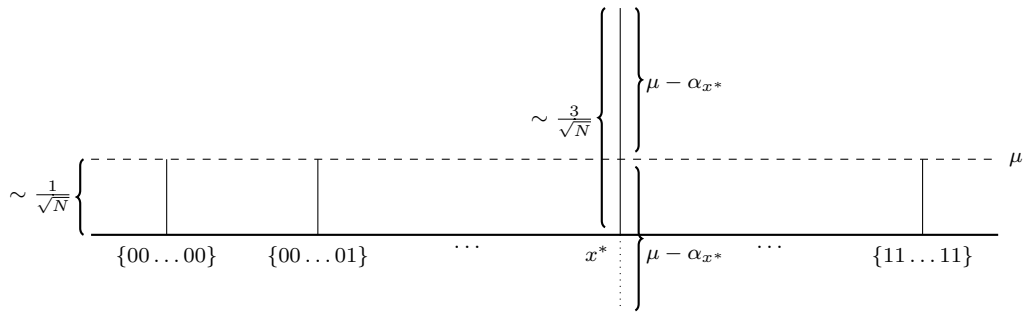
$$\mu = \frac{1}{N}\sum_x \alpha_x = \text{average of } \alpha_x \text{ for } x \in \{0,1\}^n.$$

where for α_x is the amplitude of x for a given x in $\{0,1\}^n$. Our Grover diffusion gate is defined to have the mapping

$$\sum_{x\in\{0,1\}} \alpha_x \left|x\right\rangle \mapsto \sum_{x\in\{0,1\}^n} (2\mu - \alpha_x)\left|x\right\rangle .$$

Note that this is a valid quantum gate; it is linear since all the operations in the mapping are linear, and it is also unitary. We can show it is unitary by verifying that the operation preserves the norm of the input. In addition, we will later construct the gate out of other unitary gates.

O.K., what does this gate *really* do? It flips amplitudes around the average, μ. To illustrate this, we will apply the gate, labeled D on the original circuit diagram, after the oracle gate. Here is what our amplitudes look like after application:

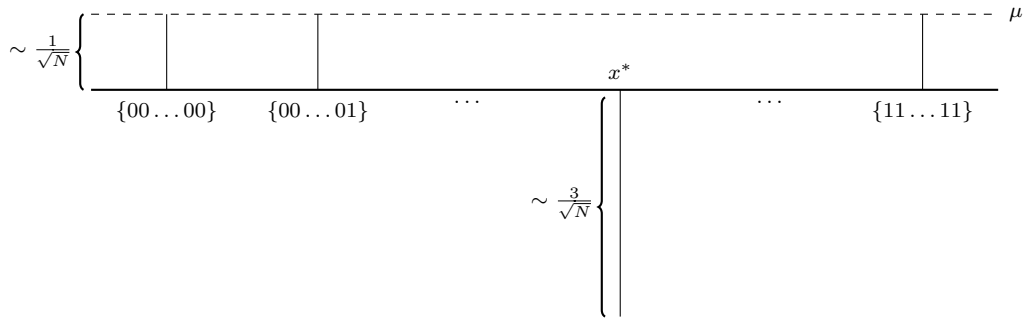


The mean amplitude prior to applying the gate was

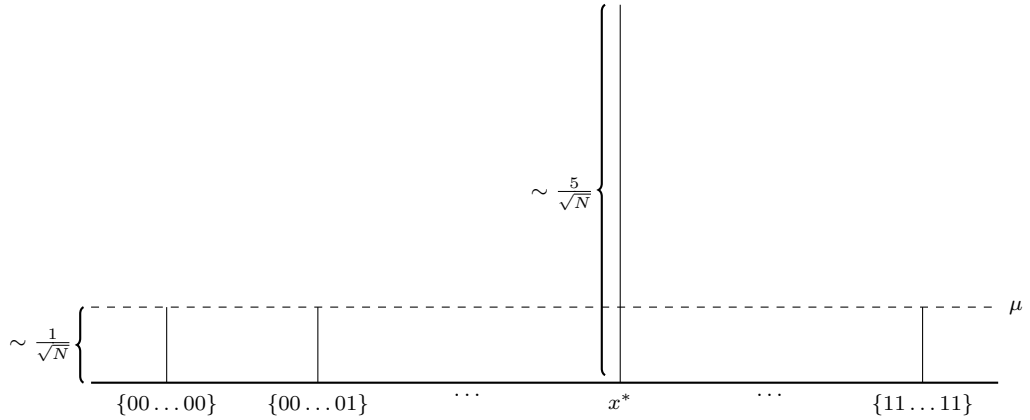
$$\sum_{x\in\{0,1\}^n} \alpha_x = \frac{2^n-1}{\sqrt{N}} - \frac{1}{\sqrt{N}} = \frac{2^n-2}{\sqrt{N}} \approx \frac{N}{\sqrt{N}} = \frac{1}{\sqrt{N}}$$

i.e. $\frac{1}{\sqrt{N}}$ minus an insignificant amount. That means when the "reflection around the mean" occurs with the application of the Grover diffusion operator, almost all the entries $x \in \{0,1\}^n$ stay roughly the same. But – and here is where the magic happens – the amplitude of x^* gets magnified to about $\frac{3}{\sqrt{N}}$ in magnitude!

Still, this is quite far from our goal of making the amplitude of x^* greater than a constant. To make further progress, we can try the most brain-dead thing possible: apply the oracle and the Grover diffusion gate again! After applying $O_f^{\pm}$ for the second time, we have



and after applying the Grover diffusion for the second time, we obtain



The quantum Fourier transform

One of the most useful ways of solving a problem in mathematics or computer science is to *transform* it into some other problem for which a solution is known. There are a few transformations of this type which appear so often and in so many different contexts that the transformations are studied for their own sake. A great discovery of quantum computation has been that some such transformations can be computed much faster on a quantum computer than on a classical computer, a discovery which has enabled the construction of fast algorithms for quantum computers.

One such transformation is the *discrete Fourier transform*. In the usual mathematical notation, the discrete Fourier transform takes as input a vector of complex numbers, $x_0,...,x_{N-1}$ where the length N of the vector is a fixed parameter. It outputs the transformed data, a vector of complex numbers $y_0,...,y_{N-1}$, defined by

$$y_k \equiv \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k/N} .$$

The *quantum Fourier transform* is exactly the same transformation, although the conventional notation for the quantum Fourier transform is somewhat different. The quantum Fourier transform on an orthonormal basis $|0\rangle, \ldots, |N-1\rangle$ is defined to be a linear operator with the following action on the basis states,

$$|j\rangle \longrightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k/N} |k\rangle .$$

Equivalently, the action on an arbitrary state may be written

$$\sum_{j=0}^{N-1} x_j |j\rangle \longrightarrow \sum_{k=0}^{N-1} y_k |k\rangle ,$$

where the amplitudes y_k are the discrete Fourier transform of the amplitudes x_j. It is not obvious from the definition, but this transformation is a unitary transformation, and thus can be implemented as the dynamics for a quantum computer. We shall demonstrate the unitarity of the Fourier transform by constructing a manifestly unitary quantum circuit computing the Fourier transform. It is also easy to prove directly that the Fourier transform is unitary:

In the following, we take $N = 2^n$, where n is some integer, and the basis $|0\rangle, \ldots, |2^n - 1\rangle$ is the computational basis for an n qubit quantum computer. It is helpful to write the state $|j\rangle$ using the binary representation $j = j_1 j_2 \ldots j_n$. More formally, $j = j_1 2^{n-1} + j_2 2^{n-2} + \cdots + j_n 2^0$. It is also convenient to adopt the notation $0.j_l j_{l+1} \ldots j_m$ to represent the *binary fraction* $j_l/2 + j_{l+1}/4 + \cdots + j_m/2^{m-l+1}$.

With a little algebra the quantum Fourier transform can be given the following useful *product representation*:

$$|j_1, \ldots, j_n\rangle \rightarrow \frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_{n-1}j_n}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \cdots j_n}|1\rangle\right)}{2^{n/2}}.$$

This product representation is so useful that you may even wish to consider this to be the *definition* of the quantum Fourier transform. As we explain shortly this representation allows us to construct an efficient quantum circuit computing the Fourier transform, a proof that the quantum Fourier transform is unitary, and provides insight into algorithms based upon the quantum Fourier transform. As an incidental bonus we obtain the classical fast Fourier transform, in the exercises!

The equivalence of the product representation (5.4) and the definition (5.2) follows from some elementary algebra:

$$\begin{aligned}
|j\rangle &\rightarrow \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i j k/2^n}|k\rangle \\
&= \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} e^{2\pi i j \left(\sum_{l=1}^{n} k_l 2^{-l}\right)}|k_1 \ldots k_n\rangle \\
&= \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} \bigotimes_{l=1}^{n} e^{2\pi i j k_l 2^{-l}}|k_l\rangle \\
&= \frac{1}{2^{n/2}} \bigotimes_{l=1}^{n} \left[\sum_{k_l=0}^{1} e^{2\pi i j k_l 2^{-l}}|k_l\rangle\right] \\
&= \frac{1}{2^{n/2}} \bigotimes_{l=1}^{n} \left[|0\rangle + e^{2\pi i j 2^{-l}}|1\rangle\right] \\
&= \frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_{n-1}j_n}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \cdots j_n}|1\rangle\right)}{2^{n/2}}
\end{aligned}$$

The product representation (5.4) makes it easy to derive an efficient circuit for the quantum Fourier transform. Such a circuit is shown in Figure 5.1. The gate R_k denotes the unitary transformation

$$R_k \equiv \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}.$$

To see that the pictured circuit computes the quantum Fourier transform, consider what happens when the state $|j_1 \ldots j_n\rangle$ is input. Applying the Hadamard gate to the first bit produces the state

$$\frac{1}{2^{1/2}}\left(|0\rangle + e^{2\pi i 0.j_1}|1\rangle\right)|j_2 \ldots j_n\rangle,$$

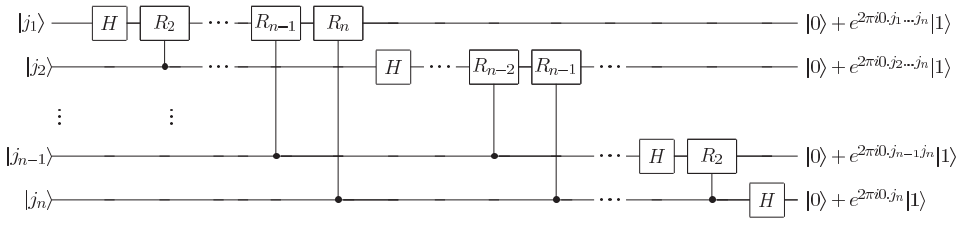


Figure 5.1. Efficient circuit for the quantum Fourier transform. This circuit is easily derived from the product representation (5.4) for the quantum Fourier transform. Not shown are swap gates at the end of the circuit which reverse the order of the qubits, or normalization factors of $1/\sqrt{2}$ in the output.

since $e^{2\pi i 0.j_1} = -1$ when $j_1 = 1$, and is +1 otherwise. Applying the controlled-R_2 gate produces the state

$$\frac{1}{2^{1/2}} \left(|0\rangle + e^{2\pi i 0.j_1 j_2} |1\rangle \right) |j_2 \ldots j_n\rangle .$$

We continue applying the controlled-R_3, R_4 through R_n gates, each of which adds an extra bit to the phase of the co-efficient of the first $|1\rangle$. At the end of this procedure we have the state

$$\frac{1}{2^{1/2}} \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n} |1\rangle \right) |j_2 \ldots j_n\rangle .$$

Next, we perform a similar procedure on the second qubit. The Hadamard gate puts us in the state

$$\frac{1}{2^{2/2}} \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n} |1\rangle \right) \left(|0\rangle + e^{2\pi i 0.j_2} |1\rangle \right) |j_3 \ldots j_n\rangle ,$$

and the controlled-R_2 through R_{n-1} gates yield the state

$$\frac{1}{2^{2/2}} \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n} |1\rangle \right) \left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n} |1\rangle \right) |j_3 \ldots j_n\rangle .$$

We continue in this fashion for each qubit, giving a final state

$$\frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n} |1\rangle \right) \left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n} |1\rangle \right) \ldots \left(|0\rangle + e^{2\pi i 0.j_n} |1\rangle \right)$$

Swap operations (see Section 1.3.4 for a description of the circuit), omitted from Figure 5.1 for clarity, are then used to reverse the order of the qubits. After the swap operations, the state of the qubits is

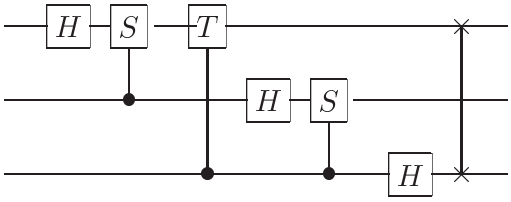
$$\frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i 0.j_n} |1\rangle \right) \left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n} |1\rangle \right) .... \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \cdots j_n} |1\rangle \right) .$$

Comparing with Equation (5.4) we see that this is the desired output from the quantum Fourier transform. This construction also proves that the quantum Fourier transform is unitary, since each gate in the circuit is unitary. An explicit example showing a circuit for the quantum Fourier transform on three qubits is given in Box 5.1.

How many gates does this circuit use? We start by doing a Hadamard gate and $n-1$ conditional rotations on the first qubit – a total of n gates. This is followed by a Hadamard gate and $n-2$ conditional rotations on the second qubit, for a total of $n + (n-1)$ gates. Continuing in this way, we see that $n + (n-1) + \cdots + 1 = n(n+1)/2$ gates are required,

Three qubit quantum Fourier transform

For concreteness it may help to look at the explicit circuit for the three qubit quantum Fourier transform:



Recall that S and T are the phase and $\pi/8$ gates (see page xxiii). As a matrix the quantum Fourier transform in this instance may be written out explicitly, using $\omega = e^{2\pi i/8} = \sqrt{i}$, as

$$\frac{1}{\sqrt{8}} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ 1 & \omega^2 & \omega^4 & \omega^6 & 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^1 & \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^5 & \omega^2 & \omega^7 & \omega^4 & \omega^1 & \omega^6 & \omega^3 \\ 1 & \omega^6 & \omega^4 & \omega^2 & 1 & \omega^6 & \omega^4 & \omega^2 \\ 1 & \omega^7 & \omega^6 & \omega^5 & \omega^4 & \omega^3 & \omega^2 & \omega^1 \end{bmatrix}. \tag{5.19}$$

plus the gates involved in the swaps. At most $n/2$ swaps are required, and each swap can be accomplished using three controlled-NOT gates. Therefore, this circuit provides a $\Theta(n^2)$ algorithm for performing the quantum Fourier transform.

In contrast, the best classical algorithms for computing the discrete Fourier transform on 2^n elements are algorithms such as the *Fast Fourier Transform (FFT)*, which compute the discrete Fourier transform using $\Theta(n2^n)$ gates. That is, it requires exponentially more operations to compute the Fourier transform on a classical computer than it does to implement the quantum Fourier transform on a quantum computer.

At face value this sounds terrific, since the Fourier transform is a crucial step in so many real-world data processing applications. For example, in computer speech recognition, the first step in phoneme recognition is to Fourier transform the digitized sound. Can we use the quantum Fourier transform to speed up the computation of these Fourier transforms? Unfortunately, the answer is that there is no known way to do this. The problem is that the amplitudes in a quantum computer cannot be directly accessed by measurement. Thus, there is no way of determining the Fourier transformed amplitudes of the original state. Worse still, there is in general no way to efficiently prepare the original state to be Fourier transformed. Thus, finding uses for the quantum Fourier transform is more subtle than we might have hoped. In this and the next chapter we develop several algorithms based upon a more subtle application of the quantum Fourier transform.

MODULE 4

Quantum Information Theory

Quantum Error Correction: Decoherence, Bit flip code, Phase flip code, Shor code. Quantum Information Theory: Von Neumann entropy, Quantum information over noisy quantum channels.
Quantum Key Distribution: Encryption, Classical solution, Quantum solution.

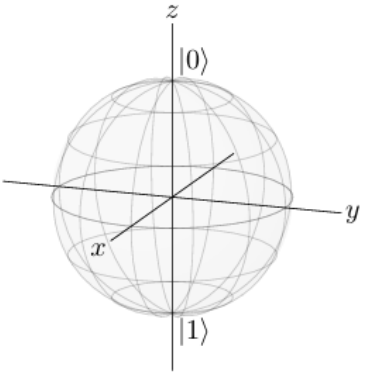
Quantum Error Correction

Quantum Error Correction comprises a set of techniques used in quantum memory and quantum computing to protect quantum information from errors arising from decoherence and other sources of quantum noise. QEC schemes that employ codewords stabilized by a set of commuting operators are known as stabilizer codes, and the corresponding codewords are referred to as **quantum error-correcting codes** (QECCs).

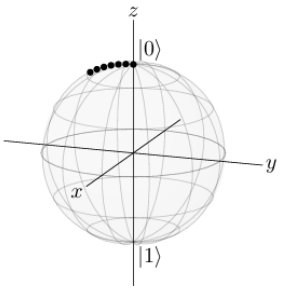
Conceptually, to use a quantum error-correcting code, one can append ancilla qubits to qubits that need protection, and apply a unitary encoding circuit to rotate the global state into a subspace of a larger Hilbert space. This highly entangled, encoded state corrects for local noisy errors. A quantum error-correcting code makes quantum computation and quantum communication practical by providing a way for a sender and receiver to simulate a noiseless qubit channel given a noisy qubit channel whose noise conforms to a particular error model.

1. Decoherence

Recall a qubit can be represented by a point on the Bloch sphere:

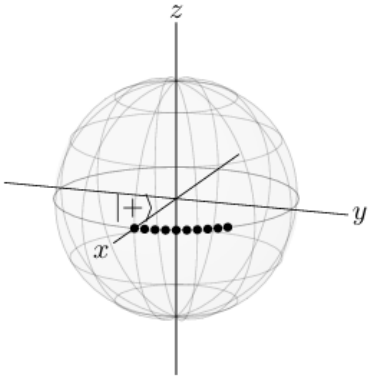


The north pole corresponds to $|0\rangle$ and the south pole corresponds to $|1\rangle$. For a classical bit, these would be the only possible states, and the only error is for the bit to completely flip between the north and south poles. For a qubit, however, every location on the Bloch sphere is a different state. For example, beginning at $|0\rangle$, instead of completely flipping to $|1\rangle$, a qubit could experience a partial bit flip error, where it only rotates a little toward $|1\rangle$:



Since a full bit flip corresponds to the X gate, and the X gate is a rotation about the x-axis by $\pi = 180^\circ$, a partial bit flip corresponds to rotating about the x-axis by some angle. So, in the above figure, the state is moving leftward, down the Bloch sphere, in the yz-plane. This small change is an error.

To further complicate matters, a qubit's state is not just its latitude up and down the Bloch sphere, but also its longitude around the Bloch sphere. For example, if a qubit initially in the $|+\rangle$ state gets bumped to the side, we get a different state:



This is called a phase flip error, because rotations around the z-axis correspond to changes in the relative phase. For example, $|+\rangle = (|0\rangle+|1\rangle)/\sqrt{2}$ and $|-\rangle = (|0\rangle- |1\rangle)/\sqrt{2}$ lie on opposite sides of the equator.

Since qubits are more sensitive to errors than classical bits, small interactions with the environment can move the qubit to a different location on the Bloch sphere. This process is called decoherence. In practice, decoherence is the biggest obstacle to building large-scale quantum computers, since it is very difficult to isolate a qubit from its environment while making it accessible for quantum gates and measurements

Next, we will see how to correct for bit-flip errors and then phase-flip errors. Then, we will combine both types of error correction into what is known as the Shor code.

2 Bit-Flip Code

To make it possible to correct bit-flip errors, we use three physical qubits to encode each logical qubit:

$$|0_L\rangle = |000\rangle, \quad |1_L\rangle = |111\rangle,$$

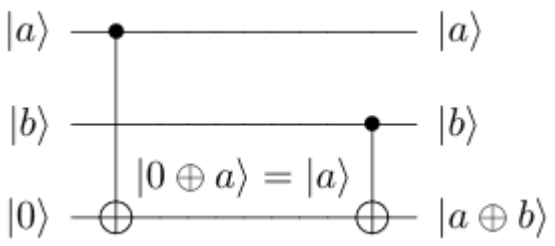
where subscript L denotes a logical qubit. A logical qubit is, in general, a superpo sition of $|0_L\rangle$ and $|1_L\rangle$:

$$\alpha|0_L\rangle + \beta|1_L\rangle = \alpha|000\rangle + \beta|111\rangle.$$

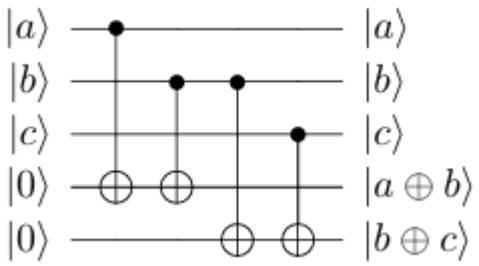
Let us first consider the case where a bit is completely flipped (ε =1). For example, say the left qubit flips, so

$$\begin{aligned}\alpha|000\rangle + \beta|111\rangle &\rightarrow \alpha|100\rangle + \beta|011\rangle \\ &= \beta|011\rangle + \alpha|100\rangle.\end{aligned}$$

We would like to detect this error and correct it. Classically, we could just measure the bits, see which one disagrees with the others, and then flip it back to correct it. Quantumly, however, if we measure the bits (or even just a single bit), the state collapses to |100⟩ or |011⟩, and we lose the superposition. So, instead of measuring the bits, we measure the parity of adjacent qubits. Recall that the parity of two bits, a and b, can be calculated using Exclusive OR. That is, parity(a,b) = a⊕b. Also recall that CNOT|a⟩|b⟩ = |a⟩|a⊕b⟩. Then, we can use two CNOTs to calculate the parity of two qubits, putting the answer in an ancilla qubit:



With three qubits, we can calculate the parities of adjacent qubits by doing this twice:



In this example, the parity of the left two qubits is 1, and the parity of the right two qubits is 0. This tells us that the left two qubits differ, and the right two qubits are the same. Then, we know the left qubit has flipped, and we inferred this without directly measuring and collapsing the state. This is called an error syndrome. To correct the error, we can simply apply (X ⊗I ⊗I), which results in β|111⟩+α|000⟩ = α|000⟩+β|111⟩, thus correcting the error.

3. Phase-Flip Code

We can similarly correct phase-flip errors by using three physical qubits to encode each logical qubit, but instead of using three |0⟩'s and |1⟩'s,we use three |+⟩'s and |−⟩'s, i.e.,

$$|0_L\rangle = |+++\rangle, \quad |1_L\rangle = |---\rangle,$$

so a general superposition is,

$$\alpha|0_L\rangle + \beta|1_L\rangle = \alpha|+++\rangle + \beta|---\rangle.$$

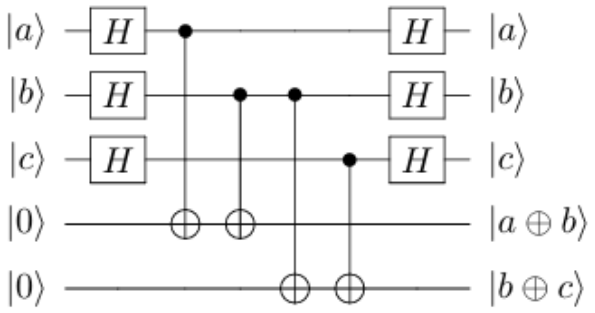
The reason why we use |+⟩ and |−⟩ is because a complete phase flip (the Zgate) switches between these states:

$$|+\rangle = \frac{1}{2}(|0\rangle + |1\rangle) \xrightarrow{Z} \frac{1}{2}(|0\rangle - |1\rangle) = |-\rangle,$$
$$|-\rangle = \frac{1}{2}(|0\rangle - |1\rangle) \xrightarrow{Z} \frac{1}{2}(|0\rangle + |1\rangle) = |+\rangle.$$

Say the left qubit experiences a complete phase flip:

$$\alpha|+++\rangle + \beta|---\rangle \rightarrow \alpha|-++\rangle + \beta|+--\rangle$$

Then,we detect and correct this just like we did for the bit-flip error, except working in the X-basis.So,we measure the parity of consecutive qubits in the X-basis,which is 0 if the number of minuses is even and 1 if the number of minuses is odd. In,you will show that the parities can be calculated using



In our example where the left qubit experienced a phaseflip,we get parity 1 for the left two qubits and parity 0 for the right two qubits, implying that the first qubit is flipped.So we apply (Z⊗I⊗I), restoring α|+++⟩+β|−−−⟩.

Now for partial phase flips, a partial phase flip corresponds to a rotation about the z-axis by some angle θ, so again with α=π/2 and ε=sin(θ/2), but now with nˆ=(0,0,1),we get that the rotation is

$$\begin{aligned} i\sqrt{1-\varepsilon^2}I+\varepsilon Z &= i\sqrt{1-\varepsilon^2}\begin{pmatrix}1&0\\0&1\end{pmatrix}+\varepsilon\begin{pmatrix}1&0\\0&-1\end{pmatrix} \\ &= \begin{pmatrix} i\sqrt{1-\varepsilon^2}+\varepsilon & 0 \\ 0 & i\sqrt{1-\varepsilon^2}-\varepsilon \end{pmatrix} \end{aligned}$$

Thus,the partial phase flip maps,

$$\begin{aligned} |0\rangle &\to \left(i\sqrt{1-\varepsilon^2}+\varepsilon\right)|0\rangle, \\ |1\rangle &\to \left(i\sqrt{1-\varepsilon^2}-\varepsilon\right)|1\rangle. \end{aligned}$$

Note when θ =π, ε = 1, and we get |0⟩→ |0⟩ and |1⟩→−|1⟩, which is a complete phase flip, or the Z gate. Let us see how a partial phase flip transforms |+⟩ and |−⟩:

$$\begin{aligned} |+\rangle &= \frac{1}{\sqrt{2}}(|0\rangle+|1\rangle) \\ &\to \frac{1}{\sqrt{2}}\left[\left(i\sqrt{1-\varepsilon^2}+\varepsilon\right)|0\rangle+\left(i\sqrt{1-\varepsilon^2}-\varepsilon\right)|1\rangle\right] \\ &= i\sqrt{1-\varepsilon^2}\frac{1}{\sqrt{2}}(|0\rangle+|1\rangle)+\varepsilon\frac{1}{\sqrt{2}}(|0\rangle-|1\rangle) \\ &= i\sqrt{1-\varepsilon^2}|+\rangle+\varepsilon|-\rangle, \\ |-\rangle &= \frac{1}{\sqrt{2}}(|0\rangle-|1\rangle) \\ &\to \frac{1}{\sqrt{2}}\left[\left(i\sqrt{1-\varepsilon^2}+\varepsilon\right)|0\rangle-\left(i\sqrt{1-\varepsilon^2}-\varepsilon\right)|1\rangle\right] \end{aligned}$$

$$= \varepsilon \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) + i\sqrt{1-\varepsilon^2} \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$
$$= \varepsilon |+\rangle + i\sqrt{1-\varepsilon^2} |-\rangle.$$

Using this, if we have a logical qubit in the state

$$\alpha|0_L\rangle + \beta|1_L\rangle = \alpha|+++\rangle + \beta|---\rangle,$$

a partial phase flip on the left qubit transforms this to

$$\begin{aligned}
&\alpha \left(i\sqrt{1-\varepsilon^2}|+++\rangle + \varepsilon|-++\rangle \right) + \beta \left(\varepsilon|+--\rangle + i\sqrt{1-\varepsilon^2}|---\rangle \right) \\
&\quad = \alpha i\sqrt{1-\varepsilon^2}|+++\rangle + \alpha\varepsilon|-++\rangle + \beta\varepsilon|+--\rangle + \beta i\sqrt{1-\varepsilon^2}|---\rangle \\
&\quad = \alpha i\sqrt{1-\varepsilon^2}|+++\rangle + \beta\varepsilon|+--\rangle + \alpha\varepsilon|-++\rangle + \beta i\sqrt{1-\varepsilon^2}|---\rangle.
\end{aligned}$$

Now, we measure the parity of adjacent qubits in the X-basis (i.e., whether the number of $|-\rangle$'s are even or odd). We get:

- parity$(q_2, q_1) = 0$ and parity$(q_1, q_0) = 0$ with probability

$$\begin{aligned}
\left| \alpha i\sqrt{1-\varepsilon^2} \right|^2 + \left| \beta i\sqrt{1-\varepsilon^2} \right|^2 &= |\alpha|^2 \left(1-\varepsilon^2\right) + |\beta|^2 \left(1-\varepsilon^2\right) \\
&= \left(|\alpha|^2 + |\beta|^2\right) \left(1-\varepsilon^2\right) \\
&= 1-\varepsilon^2,
\end{aligned}$$

  and the state collapses to

$$A \left(\alpha i\sqrt{1-\varepsilon^2}|+++\rangle + \beta i\sqrt{1-\varepsilon^2}|---\rangle \right) = \alpha|+++\rangle + \beta|---\rangle,$$

  where $A = 1/i\sqrt{1-\varepsilon^2}$ is a normalization constant. We see that the resulting state is already corrected, so we do not need to do anything further to correct the error. That is, the measurement fixed the error.
- parity$(q_2, q_1) = 1$ and parity$(q_1, q_0) = 0$ with probability

$$\begin{aligned}
|\beta\varepsilon|^2 + |\alpha\varepsilon|^2 &= |\alpha|^2|\varepsilon|^2 + |\beta|^2|\varepsilon|^2 \\
&= \left(|\alpha|^2 + |\beta|^2\right)|\varepsilon|^2 \\
&= |\varepsilon|^2,
\end{aligned}$$

  and the state collapses to

$$B(\beta\varepsilon|+--\rangle + \alpha\varepsilon|-++\rangle) = \beta|+--\rangle + \alpha|-++\rangle,$$

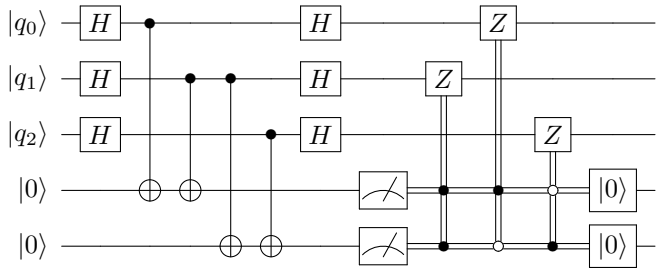
  where $B = 1/\varepsilon$ is a normalization constant. To correct this state, we apply $(Z \otimes I \otimes I)$ so that it becomes

$$\beta|---\rangle + \alpha|+++\rangle = \alpha|+++\rangle + \beta|---\rangle,$$

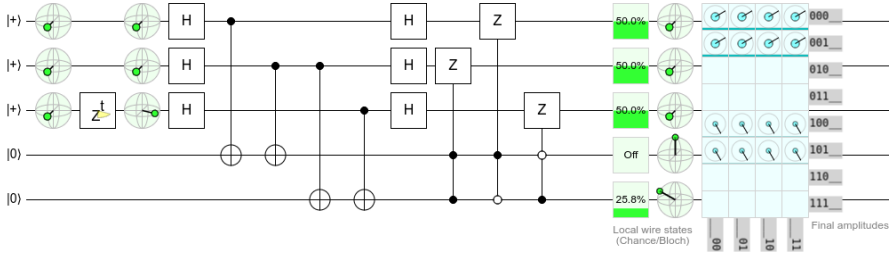
so we have corrected the error.

- parity$(q_2, q_1) = 0$ and parity$(q_1, q_0) = 1$ with probability 0.
- parity$(q_2, q_1) = 1$ and parity$(q_1, q_0) = 1$ with probability 0.

To summarize, when we have a partial phase flip, the measurement forces it to be corrected or to become a complete phase flip, which we can correct by applying a Z gate. The quantum circuit for this procedure is shown below:



Simulating this in Quirk (`https://bit.ly/3e7dNQR`):



The top three qubits are $|0_L\rangle = |+++\rangle$, and the bottom two qubits will be used to calculate the parities. In the first column of the circuit, we can visualize the states on Bloch spheres and confirm that we have $|+++\rangle$. Next, we simulate an error by applying a phase-flip Z^t, with t varying from 0 to 360°, to the middle qubit. Now, another set of Bloch spheres confirms that the middle qubit has changed. We want to correct this so that we end up with $|+++\rangle$ again. The rest of our circuit is the same as the bit-flip circuit, except we apply Hadamard gates before and after it so that we work in the X-basis. In the output, we see that we have restored $|+++\rangle$. We can move the phase flip to any of the top three qubits, and our error-correcting circuit will restore the state to $|+++\rangle$. Note we also need to reset the ancilla qubits.

4. Shor-Code

We can combine the phase-flip code and bit-flip code to correct both kinds of errors.We begin with the phase-flip code, so we can correct phase-flip errors. That is,

$$\begin{aligned}|0_L\rangle &= |+++\rangle \\ &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ &= \frac{1}{2^{3/2}}(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle + |1\rangle).\end{aligned}$$

Then, so we can correct bit-flip errors, we replace each of the three qubits with three qubits using the bit-flip encoding, i.e., $|0\rangle \rightarrow |000\rangle$ and $|1\rangle \rightarrow |111\rangle$, so that each logical qubit is encoded using nine physical qubits:

$$|0_L\rangle = \frac{1}{2^{3/2}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)$$

Similarly, we begin with $|1L\rangle = |{-}{-}{-}\rangle$ and replace $|0\rangle \rightarrow |000\rangle$ and $|1\rangle \rightarrow |111\rangle$:

$$|1_L\rangle = \frac{1}{2^{3/2}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle).$$

Then, the state of a general logical qubit is

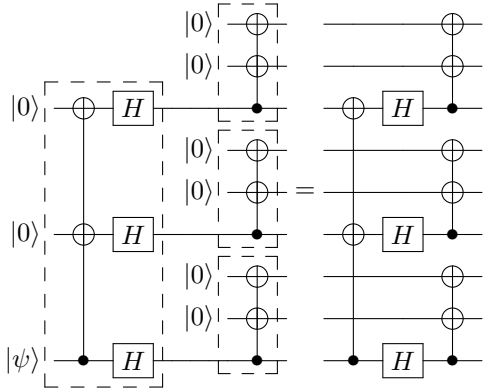
$$\begin{aligned}\alpha|0_L\rangle + \beta|1_L\rangle = &\frac{\alpha}{2^{3/2}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle) \\ &+ \frac{\beta}{2^{3/2}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)\end{aligned}$$

This encoding is called the Shor code, and it is named after its inventor, Peter Shor, who proposed it in 1995 and, by doing so, invented quantum error correction. It uses nine physical qubits to encode one logical qubit.

By alternating between correcting bit-flip errors and phase-flip errors, the Shor code corrects all quantum errors, assuming each triplet experiences at most one bit-flip error per correction cycle, and at most one triplet experiences a phase-flip error per correction cycle.

A quantum computer that accumulates errors slowly enough that errors can be corrected is called fault tolerant. Depending on the error correcting code that is used, the maximum correctable error rate can vary, and this is an area of active research. At the time of this writing, a fault tolerant quantum computer does not yet exist, and one could argue that building one is the “holy grail” of the field.

A qubit can be encoded using the Shor code by first encoding it in the three-qubit phase-flip code (Exercise 4.37) followed by encoding each of the three qubits using the three-qubit bit-flip code (Exercise 4.35), which results in nine qubits total. Applying these encodings one after another, a method called *concatenation*, yields the following circuit:



In the above circuit, the large dashed box to the left is the phase-flip encoding, which turns $|0\rangle \to |+++\rangle$ and $|1\rangle \to |---\rangle$. Then, the three dashed boxes in the middle are each the bit-flip encoding, which turns $|0\rangle \to |000\rangle$ and $|1\rangle \to |111\rangle$.

If the initial state of the circuit is $|\psi 00000000\rangle$, where $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, show that:

(a) The state of the circuit after the first column (after the CNOT with two targets) is

$$\alpha|000\rangle|000\rangle|000\rangle + \beta|100\rangle|100\rangle|100\rangle.$$

(b) The state of the circuit after the second column (after the Hadamard gates) is

$$\frac{\alpha}{\sqrt{2}}(|000\rangle + |100\rangle)(|000\rangle + |100\rangle)(|000\rangle + |100\rangle) + \frac{\beta}{\sqrt{2}}(|000\rangle - |100\rangle)(|000\rangle - |100\rangle)(|000\rangle - |100\rangle).$$

(c) The final state of the circuit is

$$\frac{\alpha}{2^{3/2}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle) + \frac{\beta}{2^{3/2}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle).$$

This is precisely $\alpha|0_L\rangle + \beta|1_L\rangle$.

Let us see how to correct bit flips and phase flips using the Shor code, beginning with bit flips. First, remember that the qubits are ordered $q_8 q_7 \ldots q_0$. Say q_8 and q_3 both experience complete bit flips. Then, the state of the system is

$$\frac{\alpha}{2^{3/2}}(|100\rangle + |011\rangle)(|001\rangle + |110\rangle)(|000\rangle + |111\rangle) + \frac{\beta}{2^{3/2}}(|100\rangle - |011\rangle)(|001\rangle - |110\rangle)(|000\rangle - |111\rangle).$$

To detect this, we measure the parities of adjacent qubits within each triplet. In this example, we would get:

$$
\begin{array}{ll}
\text{left triplet:} & \text{parity}(q_8, q_7) = 1, \;\; \text{parity}(q_7, q_6) = 0, \\
\text{middle triplet:} & \text{parity}(q_5, q_4) = 0, \;\; \text{parity}(q_4, q_3) = 1, \\
\text{right triplet:} & \text{parity}(q_2, q_1) = 0, \;\; \text{parity}(q_1, q_0) = 0.
\end{array}
$$

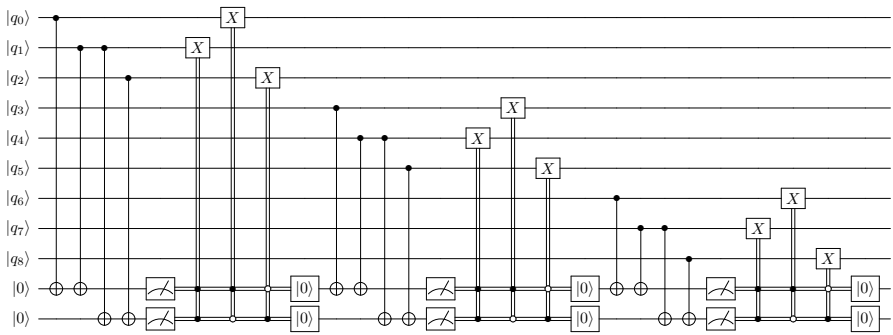
This tells us that the eighth qubit and third qubit have flipped, so we can apply X gates to those two qubits to correct them. This also works with partial bit flips. Measuring the parities of adjacent qubits might collapse the state and correct the errors, or it might collapse the state into one with full bit flips, which we correct by applying X gates to the appropriate qubits.

A logical qubit is encoded using nine physical qubits in the Shor code. In each triplet, you measure the parity of adjacent qubits and get the following results:

$$
\begin{array}{ll}
\text{left triplet:} & \text{parity}(q_8, q_7) = 0, \;\; \text{parity}(q_7, q_6) = 1, \\
\text{middle triplet:} & \text{parity}(q_5, q_4) = 1, \;\; \text{parity}(q_4, q_3) = 0, \\
\text{right triplet:} & \text{parity}(q_2, q_1) = 1, \;\; \text{parity}(q_1, q_0) = 1.
\end{array}
$$

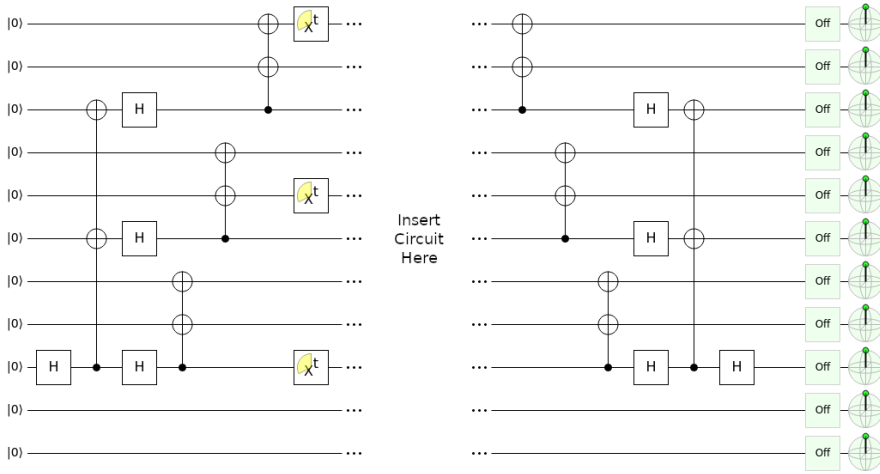
Are there any bit flip errors? If so, which bits flipped, and what can you do to correct them?

Bit flips can be corrected in the Shor code using the following quantum circuit:



The first third of the circuit measures the parities of adjacent qubits in the top three qubits, correct any errors, and reset the ancillas. The middle third of the circuit calculates the parities of adjacent qubits in the next triplet, correcting any errors. Finally, it does the same for the last triplet of qubits.

Using Quirk, simulate this circuit by inserting it into the following circuit (see `https://bit.ly/3D1kRKI`):



The first part of this circuit applies the Hadamard gate to $|q_8\rangle$, turning it into $|+\rangle$. Then, it uses the circuit in Exercise 4.41 to encode this in the Shor code. Then, the X^t gates applies a partial bit flip to one qubit in each triplet. In the middle section, you should add the previously given circuit. Although there is no reset feature in Quirk, you can use the *postselection* tool as a workaround. In Quirk, it is drawn as the outer product $|0\rangle\langle 0|$). It measures a qubit in the $\{|0\rangle, |1\rangle\}$ basis, and if the result is $|0\rangle$, the calculation continues. Otherwise, the simulation starts over. For our purposes, it has the effect of guaranteeing that the ancilla qubit is $|0\rangle$ before proceeding. A true "reset" feature would allow us to continue with the ancilla as $|0\rangle$ without the risk of restarting the simulation. In the last section of the circuit, we undo the Shor encoding and Hadamard gate so that all the qubits are $|0\rangle$ again. Verify that your circuit does this.

Next, let us see how the Shor code also allows us to correct phase flips. Say q_3 experiences a complete phase flip. Then, the state of the system is

$$\frac{\alpha}{2^{3/2}}\left(|000\rangle + |111\rangle\right)\left(|000\rangle - |111\rangle\right)\left(|000\rangle + |111\rangle\right)$$
$$+ \frac{\beta}{2^{3/2}}\left(|000\rangle - |111\rangle\right)\left(|000\rangle + |111\rangle\right)\left(|000\rangle - |111\rangle\right).$$

Then, we can measure the "phase parity" of adjacent triplets, i.e., whether the number of $(|000\rangle - |111\rangle)/\sqrt{2}$ triplets is even or odd. This is similar to the phase flip code, where we measured the parity in the X basis to determine if the number of $|-\rangle$'s was even or odd. How to measure this parity is shown in Exercise 4.44). In our example, we would get

$$\text{parity}(\text{triplet}_2, \text{triplet}_1) = 1, \quad \text{parity}(\text{triplet}_1, \text{triplet}_0) = 1.$$

This indicates that the middle triplet needs to be flipped, so we apply the Z gate to any one of the three qubits in that triplet. That is, we can apply the Z gate to either q_5, q_4, or q_3, correcting the error. Similarly, when there is a partial phase flip, if we measure all the phase parities and get zero, the state collapsed and corrected the error, and if there was a discrepancy in phase parities, we apply a Z gate to the appropriate triplet to correct it.

Quantum Information Theory

1. Von Neumann entropy

The Shannon entropy measures the uncertainty associated with a classical probability distribution. Quantum states are described in a similar fashion, with density operators replacing probability distributions. In this section we generalize the definition of the Shannon entropy to quantum states.

Von Neumann defined the entropy of a quantum state ρ by the formula

$$S(\rho) \equiv -\mathrm{tr}(\rho \log \rho).$$

In this formula logarithms are taken to base two, as usual. If λx are the eigenvalues of ρ then von Neumann's definition can be re-expressed

$$S(\rho) = -\sum_x \lambda_x \log \lambda_x,$$

where we define 0log 0 ≡ 0, as for the Shannon entropy. For calculations it is usually this last formula which is most useful. For instance, the completely mixed density operator in a d-dimensional space, I/d, has entropy log d.

From now on, when we refer to entropy, it will usually be clear from context whether we mean the Shannon or von Neumann entropy.

(Example calculations of entropy) Calculate $S(\rho)$ for

$$\rho = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\rho = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\rho = \frac{1}{3}\begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}.$$

Quantum relative entropy

As for the Shannon entropy, it is extremely useful to define a quantum version of the relative entropy. Suppose ρ and σ are density operators. The relative entropy of ρ to σ is defined by

$$S(\rho||\sigma) \equiv \mathrm{tr}(\rho \log \rho) - \mathrm{tr}(\rho \log \sigma)$$

As with the classical relative entropy, the quantum relative entropy can sometimes be infinite. In particular, the relative entropy is defined to be $+\infty$ if the kernel of σ (the vector space spanned by the eigenvectors of σ with eigenvalue 0) has non-trivial intersection with the support of ρ (the vector space spanned by the eigenvectors of ρ with non-zero eigenvalue), and is finite otherwise. The quantum relative entropy is non-negative, a result sometimes known as Klein's inequality:

The quantum relative entropy is non-negative,

$$S(\rho||\sigma) \geq 0,$$

with equality if and only if $\rho = \sigma$.

Proof
Let $\rho = \sum_i p_i |i\rangle\langle i|$ and $\sigma = \sum_j q_j |j\rangle\langle j|$ be orthonormal decompositions for ρ and σ. From the definition of the relative entropy we have

$$S(\rho||\sigma) = \sum_i p_i \log p_i - \sum_i \langle i|\rho \log \sigma|i\rangle. \tag{11.52}$$

We substitute into this equation the equations $\langle i|\rho = p_i\langle i|$ and

$$\langle i| \log \sigma|i\rangle = \langle i| \left(\sum_j \log(q_j)|j\rangle\langle j| \right) |i\rangle = \sum_j \log(q_j) P_{ij}, \tag{11.53}$$

where $P_{ij} \equiv \langle i|j\rangle\langle j|i\rangle \geq 0$, to give

$$S(\rho||\sigma) = \sum_i p_i \left(\log p_i - \sum_j P_{ij} \log(q_j) \right). \tag{11.54}$$

Note that P_{ij} satisfies the equations $P_{ij} \geq 0$, $\sum_i P_{ij} = 1$ and $\sum_j P_{ij} = 1$. (Regarding P_{ij} as a matrix, this property is known as *double stochasticity*.) Because $\log(\cdot)$ is a strictly concave function it follows that $\sum_j P_{ij} \log q_j \leq \log r_i$, where $r_i \equiv \sum_j P_{ij} q_j$, with equality if and only if there exists a value of j for which $P_{ij} = 1$. Thus

$$S(\rho||\sigma) \geq \sum_i p_i \log \frac{p_i}{r_i}, \tag{11.55}$$

with equality if and only if for each i there exists a value of j such that $P_{ij} = 1$, that is, if and only if P_{ij} is a permutation matrix. This has the form of the classical relative entropy. From the non-negativity of the classical relative entropy, Theorem 11.1, we deduce that

$$S(\rho||\sigma) \geq 0, \tag{11.56}$$

with equality if and only if pi = ri for all i,and Pij is a permutation matrix. To simplify the equality conditions further, note that by relabeling the eigenstates of σ if necessary, we can assume that Pij is the identity matrix, and thus that ρ and σ are diagonal in the same basis. The condition pi = ri tells us that the corresponding eigenvalues of ρ and σ are identical, and thus $\rho = \sigma$ are the equality conditions.

Basic properties of entropy

The von Neumann entropy has many interesting and useful properties:

(1) The entropy is non-negative. The entropy is zero if and only if the state is pure.

(2) In a d-dimensional Hilbert space the entropy is at most logd.Theentropyis equal to log d if and only if the system is in the completely mixed state I/d.

(3) Suppose a composite system AB is in a pure state. Then S(A)=S(B).

(4) Suppose pi are probabilities, and the states ρi have support on orthogonal subspaces. Then

$$S\left(\sum_i p_i \rho_i\right) = H(p_i) + \sum_i p_i S(\rho_i).$$

(5) **Joint entropy theorem:** Suppose pi are probabilities, |i> are orthogonal states for a system A, and ρi is any set of density operators for another system, B.Then

$$S\left(\sum_i p_i |i\rangle\langle i| \otimes \rho_i\right) = H(p_i) + \sum_i p_i S(\rho_i).$$

Proof:

(1) Clear from the definition.

(2) The result follows from the non-negativity of the relative entropy,
$0 \leq S(\rho||I/d) = -S(\rho) + \log d$.

(3) From the Schmidt decomposition we know that the eigenvalues of the density operators of systems A and B are the same. The entropy is determined completely by the eigenvalues, so S(A)=S(B).

(4) Let $\lambda^j{}_i$ and $|e^j{}_i\rangle$ be the eigenvalues and corresponding eigenvectors of ρi.Observe that $p_i\lambda^j{}_i$ and $|e^j{}_i\rangle$ are the eigenvalues and eigenvectors of $\Sigma_i p_i \rho_i$, and thus

$$\begin{aligned} S\left(\sum_i p_i \rho_i\right) &= -\sum_{ij} p_i \lambda_i^j \log p_i \lambda_i^j \\ &= -\sum_i p_i \log p_i - \sum_i p_i \sum_j \lambda_i^j \log \lambda_i^j \\ &= H(p_i) + \sum_i p_i S(\rho_i), \end{aligned}$$

as required.

(5) Immediate from the preceding result.

Measurements and entropy

How does the entropy of a quantum system behave when we perform a measurement on that system? Not surprisingly, the answer to this question depends on the type of measurement which we perform. Nevertheless, there are some surprisingly general assertions we can make about how entropy behaves.

Suppose, for example, that a projective measurement described by projectors Pi is performed on a quantum system, but we never learn the result of the measurement. If the state of the system before the measurement was ρ then the state after is given by

$$\rho' = \sum_i P_i \rho P_i.$$

The following result shows that the entropy is never decreased by this procedure, and remains constant only if the state is not changed by the measurement:

Theorem: (**Projective measurements increase entropy**) Suppose Pi is a complete set of orthogonal projectors and ρ is a density operator. Then the entropy of the state $\rho' \equiv \Sigma_i P_i \rho P_i$ of the system after the measurement is at least as great as the original entropy,

$$S(\rho') \geq S(\rho)$$

with equality if and only if ρ = ρ'.

Subadditivity

Suppose distinct quantum systems A and B have a joint state ρ^{AB}. Then the joint entropy for the two systems satisfies the inequalities

$$S(A,B) \leq S(A) + S(B)$$
$$S(A,B) \geq |S(A) - S(B)|.$$

The first of these inequalities is known as the subadditivity inequality for Von Neumann entropy, and holds with equality if and only if systems A and B are uncorrelated, that is, $\rho^{AB} = \rho^A \otimes \rho^B$.

The second is called the triangle inequality, or sometimes the Araki-Lieb inequality; it is the quantum analogue of the inequality H(X,Y) ≥ H(X) for Shannon entropies.

Concavity of the entropy

The entropy is a concave function of its inputs. That is, given probabilities pi–non negative real numbers such that $\Sigma_i \rho i = 1$– and corresponding density operators ρi,the entropy satisfies the inequality:

$$S\left(\sum_i p_i \rho_i\right) \geq \sum_i p_i S(\rho_i).$$

The intuition is that the $\Sigma_i$piρi expresses the state of a quantum system which is in an unknown state ρi with probability pi, and our uncertainty about this mixture of states should be higher than the average uncertainty of the states ρi, since the state $\Sigma_i$piρi expresses ignorance not only due to the states ρi, but also a contribution due to our ignorance of the index i.

Suppose the ρi are states of a system A. Introduce an auxiliary system B whose state space has an orthonormal basis |i> corresponding to the index i on the density operators ρi. Define a joint state of AB by:

$$\rho^{AB} \equiv \sum_i p_i \rho_i \otimes |i\rangle\langle i|.$$

The entropy of a mixture of quantum states

The flip side of concavity is the following useful theorem which provides an upper bound on the entropy of a mixture of quantum states. Taken together the two results imply that for a mixture $\Sigma_i$piρi of quantum states ρi the following inequality holds:

$$\sum_i p_i S(\rho_i) \leq S\left(\sum_i p_i \rho_i\right) \leq \sum_i p_i S(\rho_i) + H(p_i).$$

The intuition behind the upper bound on the right hand side is that our uncertainty about the state $\Sigma_i$piρi is never more than our average uncertainty about the state ρi,plusan additional contribution of H(pi) which represents the maximum possible contribution our uncertainty about the index i contributes to our total uncertainty.

2.Quantum information over noisy quantum channels

How much quantum information can be reliably transmitted over a noisy quantum channel? This problem of determining the quantum channel capacity is less well understood than is the problem of determining the capacity for sending classical information through a noisy quantum channel. We now present some of the information-theoretic tools that have been developed to understand the capacity of a quantum channel for quantum in formation, most notably quantum information-theoretic analogues to the Fano inequality, data processing inequality and Singleton bound.

As for quantum data compression, our point of view in studying these problems is to regard a quantum source as being a quantum system in a mixed state ρ which is entangled with another quantum system, and the measure of reliability for transmission of quantum information by the quantum operation E is the entanglement fidelity F(ρ,E). It is useful to introduce labels Q for the system ρ lives in and R, the reference system, which initially purifies Q. In this picture the entanglement fidelity is a measure of how well the entanglement between Q and R was preserved by the action of E on system Q.

Entropy exchange and the quantum Fano inequality

How much noise does a quantum operation cause when applied to the state ρ of a quantum system Q? One measure of this is the extent to which the state of RQ, initially pure, becomes mixed as a result of the quantum operation. We define the entropy exchange of the operation E upon input of ρ by

$$S(\rho, \mathcal{E}) \equiv S(R', Q')$$

Suppose that the action of the quantum operation E is mocked up by introducing an environment E, initially in a pure state, and then causing a unitary interaction between Q and E. Then the state of RQE after the interaction is a pure state, whence S(R,Q)=S(E), so the entropy exchange may also be identified with the amount of entropy introduced by the operation E into an initially pure environment E.

Note that the entropy exchange does not depend upon the way in which the initial state of Q, ρ, is purified into RQ. The reason is because any two purifications of Q into RQ are related by a unitary operation on the system R. This unitary operation on R obviously commutes with the action of the quantum operation on Q, and thus the final states of RQ induced by the two different purifications are related by a unitary transformation on R, and thus give rise to the same value for

the entropy exchange. Furthermore, it follows from these results that S(E) does not depend upon the particular environmental model for E which is used, provided the model starts with E in a pure state.

A useful explicit formula for the entropy exchange can be given based upon the operator-sum representation for quantum operations. Suppose a trace-preserving quan tum operation E has operation elements {Ei}. Then, a unitary model for this quantum operation is given by defining a unitary operator U on QE such that

$$U|\psi\rangle|0\rangle = \sum_i E_i|\psi\rangle|i\rangle \, ,$$

where |0> is the initial state of the environment, and |i> is an orthonormal basis for the environment. Note that the state of E after application of E is

$$\rho^{E'} = \sum_{i,j} \mathrm{tr}(E_i \rho E_j^\dagger)|i\rangle\langle j| \, .$$

The quantum data processing inequality

As we discussed classical data processing inequality. Recall that the data processing inequality states that for a Markov process X → Y → Z,

$$H(X) \geq H(X:Y) \geq H(X:Z),$$

with equality in the first stage if and only if the random variable X can be recovered from Y with probability one. Thus the data processing inequality provides information theoretic necessary and sufficient conditions for error-correction to be possible.

There is a quantum analogue to the data processing inequality applicable to a two stage quantum process described by quantum operations E_1 and E_2,

$$\rho \xrightarrow{\mathcal{E}_1} \rho' \xrightarrow{\mathcal{E}_2} \rho'' .$$

We define the quantum coherent information by

$$I(\rho, \mathcal{E}) \equiv S(\mathcal{E}(\rho)) - S(\rho, \mathcal{E})$$

This quantity, coherent information, is suspected (but not known) to play a role in quantum information theory analogous to the role played by the mutual information H(X:Y) in classical information theory. One reason for this belief is that coherent information satisfies a quantum data processing inequality analogous to the classical data processing inequality.

Quantum Singleton bound

The information-theoretic approach to quantum error-correction can be used to prove a beautiful bound on the ability of quantum error-correcting codes to correct errors, the quantum Singleton bound. Recall that an [n,k,d] code uses n qubits to encode k qubits, and is able to correct located errors on up to $d - 1$ of the qubits. The quantum Singleton bound states that we must have $n - k \geq 2(d - 1)$. Contrast this with the classical Singleton bound, which states that for an [n, k,d] classical code we must have $n - k \geq d-1$. Because a quantum code to correct errors on up to t qubits must have distance at least $2t + 1$ it follows that $n - k \geq 4t$. Thus, for example, a code to encode $k = 1$ qubits and capable of correcting errors on $t = 1$ of the qubits must satisfy $n-1 \geq 4$, that is, n must beat least 5,so the five qubit code is the smallest possible code for this task.

Quantum error-correction, refrigeration and Maxwell's demon

Quantum error-correction may be thought of as a type of refrigeration process, capable of keeping a quantum system at a constant entropy, despite the influence of noise processes which tend to change the entropy of the system. Indeed, quantum error-correction may even appear rather puzzling from this point of view, as it appears to allow a reduction in entropy of the quantum system in apparent violation of the second law of thermody namics! To understand why there is no violation of the second law we do an analysis of quantum error-correction similar to that used to analyze Maxwell's demon.

Quantum error-correction is essentially a special type of Maxwell's demon - we can imagine a 'demon' performing syndrome measurements on the quantum system, and then correcting errors according to the result of the syndrome measurement.

Just as in the analysis of the classical Maxwell's demon, the storage of the syndrome in the demon's memory carries with it a thermodynamic cost, in accordance with Landauer's principle. In particular, because any memory is finite, the demon must eventually begin erasing information from its memory, in order to have space for new measurement results.

Landauer's principle states that erasing one bit of information from the memory increases the total entropy of the system– quantum system, demon, and environment by at least one bit.

four-stage error-correction 'cycle'

(1) The system, starting in a state ρ, is subjected to a noisy quantum evolution that takes it to a state ρ'. In typical scenarios for error-correction, we are interested in cases where the entropy of the system increases, S(ρ') >S(ρ), although this is not necessary.

(2) A demon performs a (syndrome) measurement on the state ρ' described by measurement operators {Mm}, obtaining result m with probability

$$p_m = \mathrm{tr}(M_m \rho' M_m^\dagger), \text{ resulting in the posterior state } \rho'_m = M_m \rho' M_m^\dagger / p_m.$$

(3) The demon applies a unitary operation Vm (the recovery operation) that creates a final system state

$$\rho''_m = V_m \rho'_m V_m^\dagger = \frac{V_m M_m \rho' M_m^\dagger V_m^\dagger}{p_m}.$$

(4) The cycle is restarted. In order for this to actually be a cycle and that it be a successful error-correction, we must have $\rho''_m = \rho$ for each measurement outcome m.

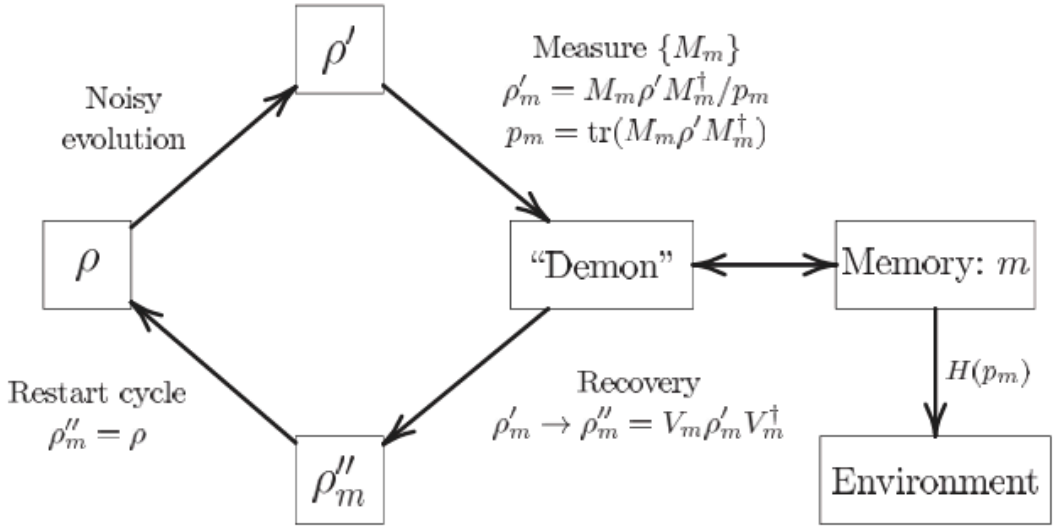


Figure. The quantum error-correction cycle

Quantum Key Distribution

1. Encryption

Alice and Bob would like to send private messages to each other over the internet. This means others can see the bits they send, but the meaning should be hidden from everyone except Alice and Bob.

To do this, Alice and Bob need to have a secret key or code that only they know. Using this secret key, they can encrypt their messages to each other.

For example, say Alice and Bob share a secret key of fourteen random bits that only they know:

$$\text{key} = 11010110011011.$$

Alice wants to send "Hi" to Bob, which in ASCII is 1001000 1101001. This is called the plaintext. If she sends these bits, everyone will know that the message is "Hi." So instead, she takes the XOR of each bit of the message with each bit of the secret key, yielding the *ciphertext*

$$\begin{aligned} \text{plaintext} &= 1001000\ 1101001 \\ \oplus\quad \text{key} &= 1101011\ 0011011 \\ \hline \text{ciphertext} &= 0100011\ 1110010 \end{aligned}$$

Now Alice sends this ciphertext to Bob over the internet. If someone intercepts it along the way, like Eve the eavesdropper, then she will not be able to determine the original plaintext since she does not have the secret key. Now Bob has the ciphertext, and he takes the XOR of it with the secret key, which he knows:

$$\begin{aligned} \text{ciphertext} &= 0100011\ 1110010 \\ \oplus\quad \text{key} &= 1101011\ 0011011 \\ \hline \text{plaintext} &= 1001000\ 1101001 \end{aligned}$$

Bob decodes the plaintext as "Hi," receiving the message.

This scheme is called a one-time pad, and it assumes that the secret key is only used once and is random. If the secret key is used more than once, then Eve might be able to discern the key, and if it is not random, Eve might be able to guess the pattern. But as long as these assumptions are satisfied, it is information-theoretically secure, meaning it is secure from a mathematical standpoint, but perhaps not secure from someone breaking into Bob's office and stealing the secret key.

Due to this information-theoretic security, the one-time pad is used for the most critical of communications, such as the Moscow–Washington hotline that allows direct, secure communication between Russia and the United States. This hotline was developed during the Cold War, and is still used today. Since the secret keys cannot just be sent over the internet for all to see, they have to be delivered in-person to each country's embassy.

For those who need strong security, but not at the level of a one-time pad, the Advanced Encryption Standard (AES) can be used. The secret key can be shorter than the message, and the ciphertext is created through a series of substitutions, shifts, and mixes. Nevertheless, a secret key must be established.

2. Classical Solution : Public Key Cryptography

A classical way to send a secret message, such as to establish a secret key, is the RSA cryptosystem, which is an acronym for its inventors Rivest, Shamir, and Adleman.1 It is an example of a public-key cryptosystem, where each user reveals some public information while still maintaining some private (secret) information.

Say Alice wants to send a message to Bob. To do this securely, Bob prepares some public and private information. The public information allows anyone to send him encrypted messages. The private information allows Bob, and only Bob, to decrypt the message.

To do this, Bob begins by choosing two distinct prime numbers p and q, with some conditions that are beyond the scope. A prime number is any whole number greater than 1 whose only factors are 1 and itself. Prime numbers can be listed using Mathematica or SageMath:

- In Mathematica, the first ten prime numbers can be listed using

  Table[Prime[i], {i, 10}]

  The output is

  {2,3,5,7,11,13,17,19,23,29}
- In SageMath, the prime numbers less than 30 can be listed using

  sage: list(primes(30))

  [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

We can also use Mathematica or SageMath to check if a number is prime.

- In Mathematica, we can check if a number is prime using the PrimeQ function, where the "Q" stands for "query" meaning to ask. With this function, we are asking if a particular number is prime.

For example, consider

|| PrimeQ[2003]

The output of this is True, so 2003 is a prime number. As another example, consider

|| PrimeQ[2005]

The output of this is False, so 2005 is not a prime number.

• In SageMath, we can check if a number is prime using the is prime function.

For example,

```
sage: is_prime(2003)
True
sage: is_prime(2005)
False
```

For example, say Bob chooses p = 17 and q = 41. Bob keeps these two numbers a secret, but he does reveal their product. That is, bob computes n = pq = 17·41 = 697 and makes the number 697 known to Alice and the world. This product n is one of Bob's two public keys, and he publishes it so that Alice can use it to send him a secret message. The length of n in bits is called the key length, and at the time of this writing, the recommended key length of RSA is 2048-bits.

For example, 697 in binary is 1010111001, so we are using a key length of 10, which is much too short in practice.

Bob then computes the product φ = (p−1)(q−1). Continuing our example, Bob computes φ = 16·40 = 640. Using this, he then finds some integer e between 1 and φ that is relatively prime or coprime to φ, meaning the greatest common divisor of e and φ is 1, which we write as gcd(e,φ) = 1. Put another way, e and φ share no common factors except 1. Note there is a fast method called Euclid's algorithm from 300 BC/BCE for calculating the greatest common divisor of two integers, and the number of steps is at most five times the number of digits of the smaller integer.

We can use Euclid's algorithm to find an e such that gcd(e,φ) = 1, or we can just use Mathematica or SageMath:

• In Mathematica, we can list all the numbers between 2 and 639, inclusive, that are relatively prime to 640 using the following:

|| Table[If[GCD[i, 640] == 1, i, Nothing], {i, 2, 639}]

The output of this is

{3,7,9,11,13,17,19,21,23,27,29,31,33,37,39,41,43,47,49,51,53,57,59,61,63,67,69,71,73,77,79,81,83,87,89,91,93,97,99,101,103,107,109,111,113,117,119,121,123,127,129,131,133,137,139,141,143,147,149,151,153,157,159,161,163,167,169,171,173,177,179,181,183,187,189,191,193,197,199,201,203,207,209,211,213,217,219,221,223,227,229,231,233,237,239,241,243,247,249,251,253,257,259,

261,263,267,269,271,273,277,279,281,283,287,289,291,293,297,299,301,303,307,309,311,313,317,319,321,323,327,329,331,333,337,339,341,343,347,349,351,353,357,359,361,363,367,369,371,373,377,379,381,383,387,389, 391,393,397,399,401,403,407,409,411,413,417,419,421,423,427,429, 431,433,437,439,441,443,447,449,451,453,457,459,461,463,467,469,471,473,477,479,481,483,487,489,491,493,497,499,501,503,507,509,511,513,517,519,521,523,527,529,531,533,537,539,541,543,547,549,551,553,557,559,561,563,567,569,571,573,577,579,581,583,587,589,591,593,597,599,601,603,607,609,611,613,617,619,621,623,627,629, 631,633,637,639}

• In SageMath, we can list all the numbers less than 640 that are relatively prime to 640 using the coprime integers function:

```
sage: phi = 640
sage: phi.coprime_integers(phi)
[1, 3, 7, 9, 11, 13, 17, 19, 21, 23, 27, 29, 31, 33, 37, 39,
41, 43, 47, 49, 51, 53, 57, 59, 61, 63, 67, 69, 71, 73, 77,
79, 81, 83, 87, 89, 91, 93, 97, 99, 101, 103, 107, 109, 111,
113, 117, 119, 121, 123, 127, 129, 131, 133, 137, 139, 141,
143, 147, 149, 151, 153, 157, 159, 161, 163, 167, 169, 171,
173, 177, 179, 181, 183, 187, 189, 191, 193, 197, 199, 201,
203, 207, 209, 211, 213, 217, 219, 221, 223, 227, 229, 231,
233, 237, 239, 241, 243, 247, 249, 251, 253, 257, 259, 261,
263, 267, 269, 271, 273, 277, 279, 281, 283, 287, 289, 291,
293, 297, 299, 301, 303, 307, 309, 311, 313, 317, 319, 321,
323, 327, 329, 331, 333, 337, 339, 341, 343, 347, 349, 351,
353, 357, 359, 361, 363, 367, 369, 371, 373, 377, 379, 381,
383, 387, 389, 391, 393, 397, 399, 401, 403, 407, 409, 411,
```

```
413, 417, 419, 421, 423, 427, 429, 431, 433, 437, 439, 441,
443, 447, 449, 451, 453, 457, 459, 461, 463, 467, 469, 471,
473, 477, 479, 481, 483, 487, 489, 491, 493, 497, 499, 501,
503, 507, 509, 511, 513, 517, 519, 521, 523, 527, 529, 531,
533, 537, 539, 541, 543, 547, 549, 551, 553, 557, 559, 561,
563, 567, 569, 571, 573, 577, 579, 581, 583, 587, 589, 591,
593, 597, 599, 601, 603, 607, 609, 611, 613, 617, 619, 621,
623, 627, 629, 631, 633, 637, 639]
```

Note we require e > 1, so we should ignore the first entry.

Bob can choose any of these numbers. Say he chooses e = 3. This number e is Bob's second public key, and he publishes it. It is called e because later, Alice will use it as an exponent.

Finally, Bob computes d = e−1 mod φ, where mod refers to the modulus or remainder when dividing by φ.2 For example, the 12-hour clock is modulo 12, since 7 + 8 = 15 = 3 mod 12. Continuing our example, we can compute d using Mathematica or SageMath:

• In Mathematica, the inverse of a number modulo some other number can be found using

|| ModularInverse[3,640]

The output is 427.

• In SageMath, the inverse of a number modulo some other number can be found using

```
sage: e = 3
sage: phi = 640
sage: e.inverse_mod(phi)
427
```

So, Bob computed d = 3−1 mod 640 = 427. Since d is the inverse of e modulo φ, if we multiply e and d modulo φ, we should get 1, i.e., ed = 1 mod φ. The number d is Bob's private key. It will allow him to decrypt messages sent to him, so he keeps it a secret. No one else knows d because even though Bob published e, one also needs to know φ in order to calculate d. Bob may now throw away p and q, but keeping it does allow him to decrypt a little faster using the Chinese remainder theorem.

Alice wants to send Bob a plaintext message that is encoded as a number M, such that 0 < M < n. She finds Bob's public keys n and e and computes the ciphertext C = Me mod n, and then she sends C to Bob. For example, Alice wants to send M = 104 to Bob, which in binary is 1101000. She can compute the ciphertext using Mathematica or SageMath:

• In Mathematica,

|| PowerMod[104,3,697]

The output is 603.

• In SageMath,

```
sage: power_mod(104,3,697)
603
```

Thus, $C = 104^3 \bmod 697 = 603$, and alice sends the number 603 to Bob.

When Bob receives the ciphertext C, he computes $C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n = M^1 \bmod n = M$, receiving the message.

Continuing our example, Bob computes $603^{427} \bmod 697$. Computing this in Mathematica or SageMath,

- In Mathematica,

```
PowerMod[603,427,697]
```

The output is 104.

- In SageMath,

```
sage: power_mod(603,427,697)
104
```

So, 603^{427} mod 697 = 104, which is the message.

The sequence of steps in this example is summarized below:

Alice	Bob
	Chooses primes $p = 17, q = 41$.
	Calculates $n = pq = 697$.
	Publishes n.
	Computes $\phi = 16 \cdot 40 = 640$.
	Chooses $e = 3$ since $\gcd(3, 640) = 1$.
	Publishes e.
	Computes $d = 3^{-1} \bmod 640 = 427$.
Chooses message $M = 104$.	
Computes $C = 104^3 \bmod 697 = 603$.	
Alice Sends to C to Bob.	
	Computes $603^{427} \bmod 697 = 104 = M$.

Decryption is hard for an eavesdropper, Eve, to do, because she does not know the secret key d. She only knows Bob's public keys, n and e. To find the secret key $d = e^{-1} \bmod \varphi$, she needs to know $\varphi = (p - 1)(q - 1)$, but this requires knowing p and q, which involves factoring n. There is no known efficient, classical algorithm for factoring large numbers, although a quantum computer can efficiently factor numbers using Shor's algorithm, which is the culminating algorithm. As on complexity classes, this is evidence, but not proof, that BQP is larger than P.

For example, say we want to find $\gcd(1122, 422)$. We begin by dividing the larger number by the smaller number. That is, $1122/442$ is 2 with a remainder of 238, which we can write as

$$1122 = 2 \cdot 442 + 238.$$

The claim is that the greatest common divisor of the dividend 1122 and divisor 442 is equal to the greatest common divisor of the divisor 442 and the remainder 238, i.e., $\gcd(1122, 442) = \gcd(442, 238)$. This is because they have the same common divisors, including their greatest common divisor.[4]

Using this fact, let us instead find $\gcd(442, 238)$. Dividing these numbers, $442/238$ is 1 with a remainder of 204, so we can write:

$$442 = 1 \cdot 238 + 204.$$

Using the same fact from before, the greatest common divisor of the dividend and divisor is equal to the greatest common divisor of the divisor and remainder, i.e., $\gcd(442, 238) = \gcd(238, 204)$.

So, we instead find $\gcd(238, 204)$. Again, we divide these numbers. Since $238/204$ is 1 with a remainder of 34, we write,

$$238 = 1 \cdot 204 + 34.$$

Again using the fact, $\gcd(238, 204) = \gcd(204, 34)$.

Now, we instead find $\gcd(204, 34)$. Dividing, $204/34$ is 6 with no remainder, so

$$204 = 6 \cdot 34 + 0.$$

We again use the fact, $\gcd(204, 34) = \gcd(34, 0)$.

Finally, we have $\gcd(34, 0) = 34$. This is because 34 is the largest integer that divides both 34 and 0 with no remainder. Since this is also the greatest common divisor of our original question, we have

$$\gcd(1122, 442) = 34.$$

3. Quantum Solution: BB84

A quantum method for establishing a shared secret key was introduced by Bennett and Brassard in 1984, and it is called the BB84 protocol. It is an example of quantum key distribution (QKD). Even though it does not use entanglement, it is a quantum protocol, so it is included in this chapter. There do exist other protocols for QKD that do utilize entanglement, such as E91, which is named after Ekert who discovered it in 1991, but they are beyond the scope of this concept at present.

In BB84, Alice begins with a bunch of random bits, and for each bit, she randomly chooses either the Z-basis $\{|0\rangle, |1\rangle\}$ or the X-basis $\{|+\rangle, |-\rangle\}$. For example,

Alice's Bits	0 1 0 1 1 0 1 1 1
Alice's Bases	Z Z X Z X X X Z Z

If the bit she wants to send is a 0, and she picked the Z-basis, then she sends Bob $|0\rangle$, and if she picked the X-basis, then she sends Bob $|+\rangle$. If Alice instead wants to send the bit 1, and she picked the Z-basis, then she sends Bob $|-\rangle$.

Continuing the example,

Alice's Bits	0	1	0	1	1	0	1	1	1
Alice's Bases	Z	Z	X	Z	X	X	X	Z	Z
Alice Sends	$\|0\rangle$	$\|1\rangle$	$\|+\rangle$	$\|1\rangle$	$\|-\rangle$	$\|+\rangle$	$\|-\rangle$	$\|1\rangle$	$\|1\rangle$

Bob receives the qubits, and he randomly measures each one in either the Z-basis or X-basis. If the basis he picked was the same as Alice's, then he will get the same result as Alice. If he picked the opposite basis, however, then he will get each possible result with probability 1/2. For example, if Alice sends $|0\rangle$ and Bob measures in the Z-basis, he is certain to get $|0\rangle$. But if he measures in the X-basis, he gets $|+\rangle$ with probability 1/2 or $|-\rangle$ with probability 1/2. He interprets $|0\rangle$ and $+\rangle$ as 0, and $|1\rangle$ and $|-\rangle$ as 1. Continuing the example,

Alice's Bits	0	1	0	1	1	0	1	1	1
Alice's Bases	Z	Z	X	Z	X	X	X	Z	Z
Alice Sends	$\|0\rangle$	$\|1\rangle$	$\|+\rangle$	$\|1\rangle$	$\|-\rangle$	$\|+\rangle$	$\|-\rangle$	$\|1\rangle$	$\|1\rangle$
Bob's Bases	Z	X	X	Z	Z	X	Z	X	Z
Bob's Measurement	$\|0\rangle$	$\|-\rangle$	$\|+\rangle$	$\|1\rangle$	$\|0\rangle$	$\|+\rangle$	$\|1\rangle$	$\|+\rangle$	$\|1\rangle$
Bob's Bits	0	1	0	1	0	0	1	0	1

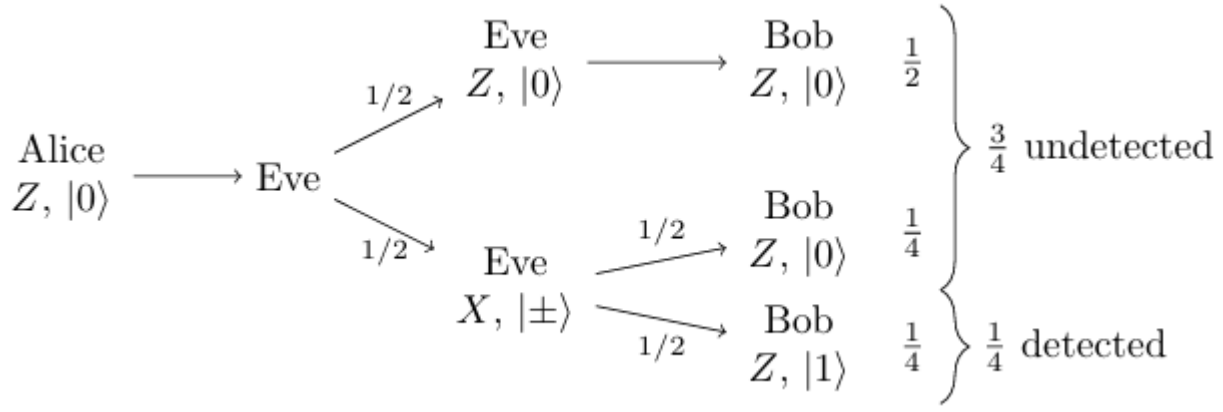
Now Alice and Bob call each other and openly (publicly) share what basis they used for each measurement. If they used the same basis, then they know that their measurement outcomes should agree, and they have a shared secret bit. If they used different bases, then their measurement outcomes might agree or disagree, and they discard these bits. Continuing the example.

Alice's Bits	0	1	0	1	1	0	1	1	1
Alice's Bases	Z	Z	X	Z	X	X	X	Z	Z
Alice Sends	$\|0\rangle$	$\|1\rangle$	$\|+\rangle$	$\|1\rangle$	$\|-\rangle$	$\|+\rangle$	$\|-\rangle$	$\|1\rangle$	$\|1\rangle$
Bob's Bases	Z	X	X	Z	Z	X	Z	X	Z
Bob's Measurement	$\|0\rangle$	$\|-\rangle$	$\|+\rangle$	$\|1\rangle$	$\|0\rangle$	$\|+\rangle$	$\|1\rangle$	$\|+\rangle$	$\|1\rangle$
Bob's Bits	0	1	0	1	0	0	1	0	1
Public Discussion of Basis									
Shared Secret Key	0		0	1		0			1

So,their shared secret key is **00101.**

To ensure Eve did not measure the qubits along the way, Alice and Bob can reveal a fraction of their shared secret key and make sure they agree. For example, if Alice and Bob want 256 bits in their shared secret key, they can generate 306 bits using BB84, reveal 50 of them to ensure there was no eavesdropper, and then use the remaining 256 for their shared secret key.

If Alice and Bob reveal 50 bits of their shared secret key, what is the probability they will catch Eve, if Eve is measuring every qubit along the way? To answer this, let us start with revealing one bit. Say Alice and Bob are revealing a bit where they both used the Z-basis, and say Alice sent a qubit in the state $|0\rangle$. Then, Alice will reveal that her bit is 0, while Bob could reveal that his bit is 0 or 1, and we determine the probabilities of these outcomes using the following diagram:



On the left side of the diagram, Alice is using the Z-basis, and she sends a qubit in the $|0\rangle$ state. In the middle of the diagram, Eve intercepted the qubit and measured it in either the Z-basis or the X-basis, each with probability 1/2. If Eve measured in the Z-basis, she got $|0\rangle$, and then forwarded the qubit to Bob. Bob measured the qubit in the Z-basis, so he also got $|0\rangle$. This is the top row of the above diagram.

In this scenario, which occurs with probability 1/2, Alice and Bob both reveal that they got the bit 0, and Eve's eavesdropping was undetected. Now, if Eve measured in the X-basis instead, then she collapsed the state to $|+\rangle$ or $|-\rangle$ and forwarded it to Bob. In the above figure, this is the bottom row.

Bob then measured the qubit in the Z-basis, getting $|0\rangle$ with probability 1/2 or $|1\rangle$ with probability 1/2. Overall, each of these outcomes occur with probability 1/4. If Bob got $|0\rangle$, Eve is undetected, but if he got $|1\rangle$, Alice and Bob will realize there was an eavesdropper when they reveal their results. Overall, Eve has a probability of 3/4 of being undetected and a probability of 1/4 of being detected, as indicated by the curly brace in the above figure, when Alice and Bob reveal this bit of their shared secret key.

In other words, there is a probability of 3/4 that Alice and Bob both have 0 as their bits, and probability 1/4 that Alice has 0 and Bob has 1.

If Alice and Bob share n bits of their shared secret key, the probability that Eve is undetected for all n bits is (3/4)n. Then, the probability that Eve is detected is one minus this, or

$$\text{Probability Alice and Bob detect Eve} = 1 - \left(\frac{3}{4}\right)^n.$$

Thus, if Alice and Bob share 50 bits of their shared secret key, the probability that they detect Eve is $1 - (3/4)^{50} = 0.99999943$, which is very close to certainty.

In order to use BB84 in practice, we need the ability to send qubits to each other through a network. This is called a **quantum network,** and building a quantum network is an area of active research.

MODULE 5

AI in Quantum Computing

Quantum machine learning, Types in QML, Quantum K means clustering, variational Quantum circuits, QSVM.

5.1 Quantum Machine Learning

Machine Learning

Machine learning (ML) emerged as a subfield of research in artificial intelligence and cognitive science. The three main types of machine learning are supervised learning, unsupervised learning, and reinforcement learning. There are several others, including semisupervised learning, batch learning, and online learning. Deep learning (DL) is a special subset that includes these various types of ML. It extends them to solve other problems in artificial intelligence, such as reasoning, planning, and knowledge representation.

What is machine learning? In 1959, Arthur Samuel defined machine learning as "the field of study that gives computers the ability to learn without being explicitly programmed." ML is a subject that focuses on teaching computers how to learn without any need for task-specific programming in today's context. Machine learning explores the creation of algorithms that learn from and make predictions on data.

Traditionally, handcrafting and optimizing algorithms to complete a specific task has been the way to build legacy software and address workloads on data manipulation and analysis. At a high-level, traditional programming focuses on a deterministic outcome for well-understood input data. In contrast, ML has the power to solve a whole class or range of problems for datasets whose input-output correspondence is not clearly understood.

ML constitutes learning from data. One example of ML is a spam filter. A spam filter basically "learns" from examples of spam emails flagged by users, examples of regular non-spam emails, examples of known cases of widespread frauds created by spam (such as phishing emails, well-known frauds that ask for personal information, etc.), and other types of use cases.

A spam filter governed by ML learns from examples and applies what it has learned to suspected spam emails. However, this learning process and the subsequent algorithmic application of the results of the "learning" are probabilistic (i.e., it is not always perfect or deterministic). Hence, occasionally, you find legitimate emails in your spam folder and spam in your inbox. The more examples the spam filter gets to "see," the more it trains itself to detect legitimate spam, and the better it performs at its job of protecting our mailbox.

One of the main reasons behind the success of ML over traditional programming in this type of task is the ever-increasing volume of data. Spam filters typically act on rules. In the past, we had to manually define rules to stop spams in our mailboxes where we had to define "from," "to," "subject," and so forth.

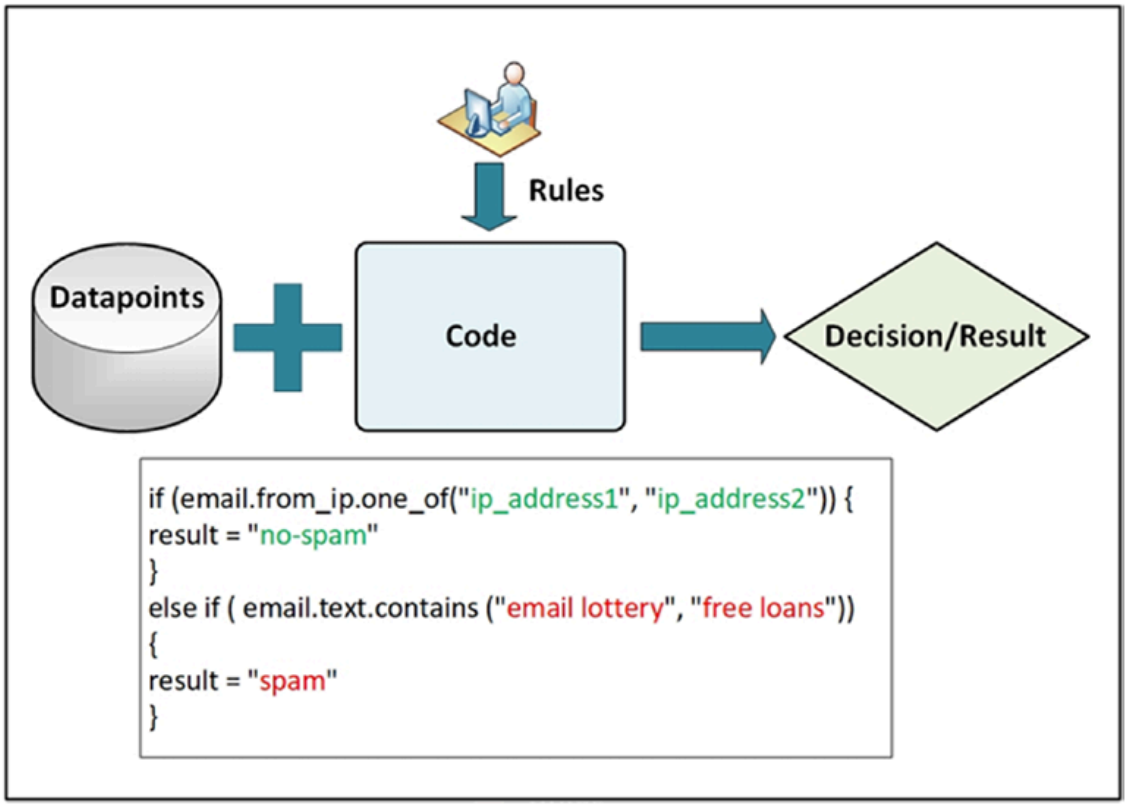


Figure. The legacy method of manually entering rules for desirable results

With the explosion of traffic and data in this age of big data in the modern world, the ability of humans to manually define rules by hand and keep up with the increasing volume of data has fallen well below the required efficiency level. Hence, the machines have taken over that task to define comprehensive rules to process large chunks of data and apply those same rules to provide solutions to problems.

The data that the ML system uses to learn from is called a training set. In the example of the spam filter, flagging and filtering the spam or the training data to define the performance of the algorithm. The performance measure, in this case, can be the ratio of correctly or incorrectly spammed emails. The performance measure is called accuracy and this value, as demonstrated in some of the examples in this chapter, usually improves with training and time. The workflow of the application of ML for a spam filter is shown in Figure below.

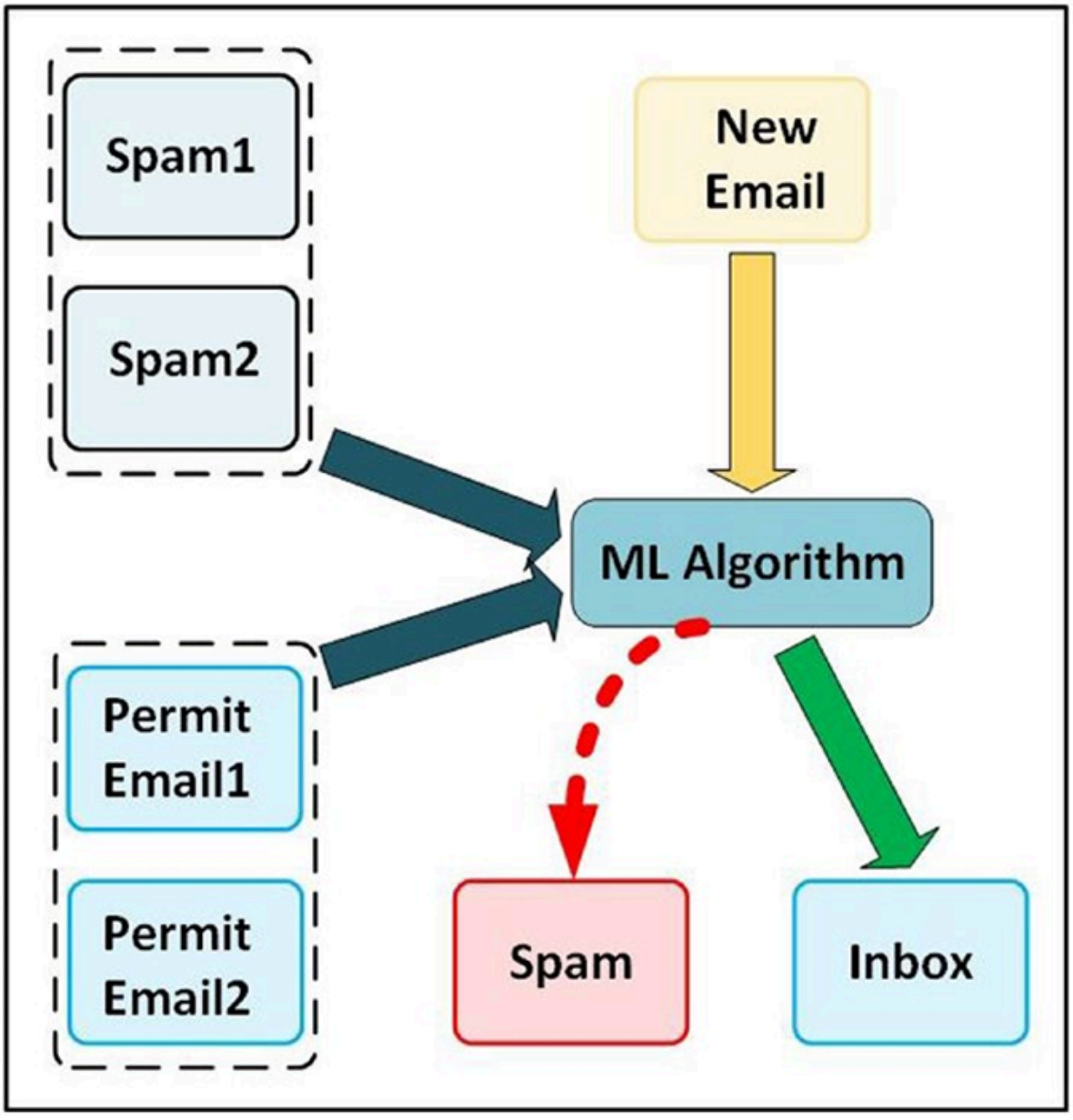


Figure. Spam filtering with ML

Machine learning and big data: Until recently, most ML workloads were performed on single machines (computers). However, developments in GPUs (graphics processing units), Google's TPU (Tensor Processing Unit), and distributed computing now make it possible to run machine learning algorithms for big data at a massive scale, distributed among clusters.

Quantum Machine Learning

Computational intelligence uses machine learning algorithms to solve optimization problems. One of the largest challenges of ML is handling data in multiple dimensions. Principles of superposition, entanglement, and contextual sensitivity have proven to be strong proponents of offering solutions for hard problems. Quantum mechanics have been useful in solving problems where the systems under investigation show contextual behavior. Natural quantum mechanical properties have proven to be an ideal match in such cases .

In recent years, quantum neural networks (QNN) and quantum convolutional neural networks (QCNN) have become areas of deep interest to researchers. These areas investigate options of classical data encodement in quantum states and leverage superposition properties of quantum systems to solve specific problems, such as image classification utilizing QCNN or QNN algorithms and quantum image processing (QIP) using algorithms such as flexible representation of quantum images (FRQI), novel enhanced quantum representation of digital images (NEQR), and quantum boolean image processing (QBIP).

The prospect and promise of quantum theory are becoming more attractive to data scientists because of its inherent nature, which unites geometry and probability in one framework. As the surge of big data takes over the digital world, handling the ever-increasing volume of data in different dimensional space in real time has thrown up a challenge to industries and governments alike; for example, financial industries are trying to fathom depths of QML and quantum field theory to develop financial algorithms for risk analysis, forecasting, and so forth, while federal governments are looking at QML and optimization theories from security and threat mitigation perspective. As global security threats arise, modeling the same threats and forecasting on that basis is becoming an increasing challenge with classical algorithms.

Quantum machine learning (QML) is an interdisciplinary field that integrates quantum physics concepts with machine learning to produce algorithms that employ quantum computer's processing power to address specific sorts of issues more effectively than classical computers. The quantum computer, a whole new class of computing system, increases the hardware available for machine learning through quantum machine learning. Quantum theory, which is based on completely different physics, is the foundation for information processing using quantum computers.

The collaboration of machine learning, quantum computing, and intelligent optimization algorithms serves as the foundation for addressing the complexities of future communication systems and optimizing their performance.

Machine Learning is all about complicated mathematical calculations with large dimensions of data. Since a qubit can contain both 0 and 1 so we can say

$$NQubit \mapsto 2^N \ Bits$$

So 1 qubit is 2 bits,2 qubit is 4 bits, 3 qubit is 8 bits... and so on. Wait, are you not impressed? Then let's just imagine 300 Qubits which is 2^300 i.e 2^270 GB. We don't even have that many atoms in the world(never underestimate the power of exponentials). Taking advantage of this classical data is converted into the Quantum data with dimensions reduced exponentially. Quantum data is any data source that occurs in a natural or artificial quantum system. Heuristic machine learning techniques can create models that maximize the extraction of useful classical information from noisy entangled data.

The TensorFlow Quantum (TFQ) library provides primitives for developing models that disentangle and generalise correlations in quantum data, allowing for the improvement of existing quantum algorithms as well as the discovery of new quantum algorithms. In addition, unlike classical computers, where a fixed input bit is sent to a logic gate module for some arithmetic operation and a corresponding fixed bit is obtained as output, quantum computers use quantum logic gates where the input is a superposition bit and the output is also an entangled superposition bit. In this process, the same input qubit can be manipulated to the various required outputs with varying entanglement in the logic gates, allowing us to find patterns and relations in the data more quickly. While performing the same task in a classical computer, and in some cases, supercomputers also can take very years to calculate because of limitations in the computation power. And that’s what machine learning is all about finding hidden patterns in the data.

Uses of Quantum Machine Learning:

- Quantum data analysis: Big quantum datasets produced by quantum sensors or simulators can be analyzed using quantum machine learning. This might make it possible to analyze data more effectively and find patterns and insights more quickly.
- Quantum chemistry: Drug development and materials science benefit greatly from the ability to model and anticipate molecular behavior and chemical reactions through the use of quantum machine learning.

- Quantum optimization: Identifying the ideal configuration for a sizable system of interconnected components is one optimization problem that can be resolved using quantum machine learning. Supply chain management, scheduling, and logistics can all benefit from this.
- Quantum anomaly detection: When a quantum computer is being hacked, for example, anomalies in the system can be found using quantum machine learning.
- Quantum cryptography: The security of quantum cryptography systems can be increased by using quantum machine learning, for example, by identifying and blocking eavesdropping on quantum communication channels.

Issues in Quantum Machine Learning:

- Hardware limitations: In order to be used for machine learning applications in the real world, quantum computers still have a long way to go due to significant hardware limitations.
- Noise and error correction: The accuracy and dependability of quantum machine learning models may be adversely affected by the inherent noise and error-proneness of quantum systems. One of the biggest challenges in the field is creating efficient error correction techniques.
- Algorithm development: Considering quantum algorithms are fundamentally different from classical algorithms, developing effective and efficient quantum machine learning algorithms is a significant challenge.
- Data preparation: Designed data preparation techniques are necessary for quantum machine learning, and they can be difficult and time-consuming.
- Results interpretation: Since quantum machine learning models are based on states and processes that are fundamental to quantum mechanics and may be challenging to comprehend and visualize, they can be challenging to interpret.
- Scalability: Given that the resources needed to carry out quantum computations grow exponentially with the size of the problem, scaling up quantum machine learning models to handle big datasets and complicated problems is extremely difficult.

Classical and quantum probability

To understand what quantum computing might mean for machine learning, we must distinguish between and understand the differences between classical and quantum data, as well as classical and quantum processing devices. We will begin by contrasting classical probability with quantum probability. Our approach will be less rigorous and formal, and we intend to provide as much intuition about quantum probability as possible, drawing on our knowledge and experience with classical probability theory.

5.2 Types of QML

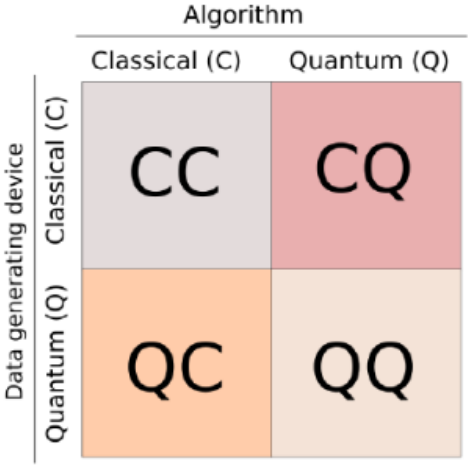
Quantum Machine Learning (QML) can be categorized in **two** primary ways:

1. by the type of data and processing device (classical or quantum)
2. by the specific machine learning task or algorithm being implemented.

Classification by Data and Processing Type

A common theoretical framework categorizes QML into four types based on whether the data is classical (C) or quantum (Q), and whether the algorithm runs on a classical or quantum computer:

• **CC (Classical Data, Classical Computer):** This refers to quantum-inspired classical algorithms. These methods use concepts from quantum theory (like tensor networks) to improve classical machine learning tasks, but run entirely on classical hardware.
• **CQ (Classical Data, Quantum Computer):** This is the most common approach in the current "Noisy Intermediate-Scale Quantum" (NISQ) era. The goal is to use a quantum computer to speed up computationally intensive subroutines (e.g., in optimization or kernel estimation) using classical data inputs.
• **QC (Quantum Data, Classical Computer):** This involves using classical machine learning algorithms to analyze data generated from quantum systems or experiments, such as in quantum chemistry or material science research.
• **QQ (Quantum Data, Quantum Computer):** This represents a fully quantum approach, where both the data and the processing are quantum. This area is less explored due to hardware limitations but could be used for tasks like learning unknown quantum states or quantum cryptography.



Quantum-inspired methods (CC)

Quantum-inspired methods draw their inspiration from quantum computing and algorithms. But what does that mean? A couple of examples will clarify things.

The first is the "de-quantization" of quantum algorithms. This line of research burst into the scene with a result back in 2018 from Ewin Tang showing how a quantum algorithm for recommendation systems (aka, solving the Netflix problem) could be turned into a classical one. Incredibly, this result showed the resulting classical algorithm was faster than any known to that point, and also only slightly slower than the quantum algorithm it was inspired by. Tang's result has inaugurated a line of work in which researchers try to de-quantize other quantum algorithms.

This kind of work – while depressing in the sense of "Those darn classical algorithms are getting better compared to quantum!" – is quite useful, precisely because it helps us understand the performance gap between classical and quantum computers. Further, as quantum algorithms are de-quantized, the ones which remain (and their associated provable speedups) give us a better sense of what kinds of computational tasks are actually amenable to quantum.

The second example is the introduction of tensor network methods in classical AI. Tensor networks are a kind of classical data structure physicists created to study the behavior of highly-correlated quantum-mechanical systems (e.g., spin chains). These data structures have tunable properties which allow them to capture different kinds of correlations in the system. Interestingly, the use of tensor networks has emerged as an alternative to some existing approaches in classical ML.

Quantum-enhanced machine learning (CQ)

This sub-category has exploded in popularity in recent years, as quantum computing started to make contact with classical data science. The key idea for methods in this category is to simply augment (or "enhance") classical ML algorithms with access to quantum computers. Some examples of methods in this category include:

- Quantum kernels. Kernels are similarity measures between two pieces of data, say x and y; kernels are used quite a lot in classical ML to help boost the performance of the algorithm. Quantum kernels are computed using a parameterized quantum circuit. The resulting similarity measure K(x,y) can be used in any classical kernel-based algorithm. A great overview of quantum kernels can be found here.
- Quantum neural networks (QNNs). The literature hasn't yet converged on what exactly a QNN means, but quantum analogues have been proposed for the classical neuron, convolutional neural networks, and recurrent neural networks.
- Quantum generative adversarial networks (qGANs). I call these out as separate from QNNs, because generative modeling has emerged as one of the most interesting ways quantum could be applied to ML. Why? The intuition is quantum computers can generate probability distributions for which classical computers struggle to generate samples. (Though "quantum ChatGPT" is unlikely to be a thing for a while!) Depending on the paper or implementation, a qGAN may use a classical or quantum neural network for both the generator and the discriminator. Examples of qGANs can be found here and here.

One reason why it is excited about this category is the fact that it seems the most straightforward way to connect quantum computing to classical ML without having to wait for fault-tolerant quantum computers.

Further, there are formal proofs for quantum kernels that they can yield an advantage , though with respect to a highly-contrived classification problem, as well as proofs for quantum generative modeling (though again, with respect to a highly-contrived problem). These sorts of results give us a small toehold on the power of quantum-enhanced machine learning, and also give us new techniques with which we can refine our understanding.

This said, there are definitely some obstacles to using quantum-enhanced machine learning in practice. One of the most prominent ones is the notion of "barren plateaus", which is the phenomenon that, as the number of qubits in the circuit increases, any gradients with respect to the parameters of that circuit tends to decay exponentially. This makes it hard to train quantum neural networks, because gradient-based algorithms would tend to get stuck. That said, some workarounds have been proposed. Barren plateaus also have implications for quantum kernels; namely, the resulting similarity measure can become easily-approximable by a purely-classical kernel.

Another obstacle is simply a data throughput problem – quantum computers can't magically process an entire data set at once (it has to be loaded piece by piece, somehow!), meaning the rate at which data can be loaded and processed can form a significant bottleneck in the overall wall-clock time of the algorithm. That is, for these algorithms usually you load a data point (or two, in the case of quantum kernels), and you do some quantum computation, and then you load the next data point(s). This all happens in serial, so the rate at which you can load and process data will become a factor to using these algorithms in practice.

ML for Physics (QC)

This sub-category really isn't very applicable for end-users of quantum computers, as it is concerned with the use of classical machine learning methods applied to classical data generated by quantum computers. (That is, the output bitstrings which come from measuring a quantum state.) Examples of activities in this sub-category include using machine learning to design quantum computing experiments, characterizing the noise affecting quantum computers, and predicting the capabilities of quantum computers.

Quantum Learning (QQ)

This subcategory can be subdivided in at least 2 ways. The first is some specialized quantum algorithms for machine learning problems, such as linear systems solvers, clustering, and topological data analysis. The second is processing data in the form of quantum states.

The first division is the one you'll probably commonly hear about when people talk about QML, and one which receives a lot of attention due to the promise (with quite a few caveats!) of exponential speedups for some tasks. In particular, back in 2008, Harrow, Hassidim, and Lloyd proposed an algorithm for linear systems solvers with such an exponential speedup. (For an overview of the "HHL algorithm", see here.) However, this algorithm relies on a particular data structure, qRAM, for which it is currently unclear if it can be actually implemented at scale.

The two other common ML tasks falling into this division are clustering and topological data analysis. The former is the problem of determining, given a collection of points, the most optimal way to group them, whereas the latter is a family of problems related to analyzing data sets using ideas from topology. A nice discussion of quantum algorithms for TDA is available here.

The second division (processing data in the form of quantum states) seems a bit similar to quantum-enhanced ML (CQ) above. At first glance, both involve encoding data into quantum states, and then doing some processing, so what's the difference? The primary difference is that in quantum-enhanced ML, we're encoding a purely classical data set into quantum states, whereas here, the quantum state being processed originates from some kind of quantum-mechanical process.

An example might help – suppose I have a quantum sensor somewhere, whose state changes in response to the environment (e.g., a spin in a magnetic field). I can take that state and send it to a quantum computer (e.g., via a quantum communication link). Then I might run some kind of data processing on that state. Or, as another example, I might use a quantum computer to simulate the time dynamics of a condensed matter or chemical system, then use a circuit to calculate some property of that state. In both of these examples, the state being processed by the algorithm has its origins in some kind of quantum-mechanical system.

Classification by ML Task or Algorithm

QML also adapts classical machine learning paradigms (supervised, unsupervised, reinforcement learning) and specific algorithms to the quantum domain:

• Quantum Neural Networks (QNNs):

Quantum analogues of classical neural networks that use parameterized quantum circuits (PQCs) to process information.

Quantum Neural Networks (QNNs) are computational models that integrate quantum computing principles with neural network structures inspired by classical artificial neural networks (ANNs). While classical neural networks utilize interconnected layers of artificial neurons to model complex data relationships, QNNs employ qubits and quantum gates, leveraging quantum phenomena such as superposition, entanglement, and interference to potentially enhance computational efficiency and representational capacity.

Quantum Neural Networks operate by encoding data into quantum states, manipulating these states using quantum gates, and measuring outputs to obtain predictions or decisions. Unlike classical neurons, quantum neurons can exist in multiple states simultaneously, thanks

to quantum superposition. Entanglement between qubits allows quantum neural networks to explore correlations between data points more efficiently than classical systems under certain conditions.

A typical QNN architecture involves three primary steps:

Quantum Encoding: Classical data is translated into quantum states. Common encoding strategies include amplitude encoding, where data is represented by amplitudes of quantum states, and angle encoding, in which classical features are encoded in complex phases of the states.

Quantum Processing: Quantum circuits, typically composed of parameterized quantum gates, manipulate encoded quantum states. Parameters are optimized via classical algorithms using feedback from measurement outcomes.

Measurement and Interpretation: Quantum measurements collapse the quantum state, translating quantum information back into classical data. This measured output is analyzed or further processed classically, integrating quantum computations into broader computational workflows.

• Quantum Support Vector Machines (QSVMs):

These leverage quantum mechanics for efficient kernel estimation, allowing for better classification of complex data in high-dimensional feature spaces than might be possible classically.

• Quantum Principal Component Analysis (QPCA):

A quantum algorithm for dimensionality reduction that can process high-dimensional data exponentially faster for certain problems.

Quantum Principal Component Analysis (QPCA) is a hybrid dimensional-reduction algorithm that translates the covariance structure of classical data into a quantum density matrix and then employs Quantum Phase Estimation (QPE) to extract its eigenvalues. By simulating the unitary (where the density matrix is, QPCA can reveal the principal components of a data set while potentially leveraging quantum parallelism for speed-ups on large, high-dimensional data.

How Does Quantum PCA Differ from Classical PCA? Classical PCA diagonalizes a covariance matrix on a classical computer, typically in time proportional to d features, returning eigenvectors and eigenvalues that quantify variance.

QPCA instead:

- Encodes the normalised covariance matrix as a quantum density matrix.
- Evolves that matrix under a simulated Hamiltonian to obtain a unitary operator.
- Applies Quantum Phase Estimation to recover phase angles proportional to the eigenvalues.
- Reconstructs principal-component variances from measured ancilla statistics.

The quantum-enhanced subroutines replace matrix diagonalisation with potentially poly-logarithmic depth circuits, shifting the computational bottleneck to state preparation and QPE rather than classical eigen decomposition.

Advantages

- Exponential Feature Space Representation – Encoding the covariance matrix as a quantum state embeds dd-dimensional statistics into a 2^n -2^n-dimensional Hilbert space (with n =[log2d]), letting a relatively small qubit register capture correlations that would require massive classical memory.
- Poly-logarithmic Eigenvalue Estimation – Quantum Phase Estimation extracts eigenvalues in O(poly log d) depth under fault-tolerant assumptions, versus classical eigen decomposition, offering asymptotic speed-ups on very large feature sets.
- Spectral-Gap Amplification – Phase amplitudes scale with the eigenvalue gap, allowing rare-variance directions to be distinguished more cleanly than in floating-point classical PCA, which can suffer from numerical round-off.
- Privacy Preservation Through Quantum States – Because only aggregated density-matrix statistics are uploaded to the quantum back-end, raw records remain on-prem, providing an intrinsic data-obfuscation layer.
- Hybrid Synergy – QPCA slots into classical analytics pipelines: state preparation and result interpretation stay classical, while the expensive diagonalisation step is offloaded to quantum hardware, minimising refactor cost for existing workflows.

Disadvantages

- State-Preparation Bottleneck – Constructing σ requires O(nd^2) classical time and memory O(d^2), then converts it into quantum amplitudes via O(d) controlled rotations; this overhead can eclipse the quantum speed-up for moderate problem sizes.
- Deep-Circuit Requirements – QPE needs coherent application of U^2k for k ancilla bits; depth grows exponentially with precision, demanding fault-tolerant qubits well beyond today's NISQ limits.
- Noise-Induced Phase Uncertainty – Gate and read-out errors blur measured phases, collapsing small eigenvalues into sampling noise and requiring heavy error-mitigation or repetition.
- Eigenvector Recovery Still Classical – QPCA returns eigen-values natively; reconstructing eigen-vectors generally needs classical post-processing (or additional tomography), limiting full quantum advantage.
- Parameter-Sensitivity – Poor choices of evolution time t or ancilla precision cause eigen-phase aliasing; tuning these hyperparameters often falls back on classical grid-search.
- Limited Demonstrated Scale – Published demonstrations remain on small (≤ 8-qubit) simulators; no public hardware run has yet beaten classical PCA time-to-solution on real-world data.

• Quantum Optimization:

At its core, quantum optimization is about finding the best possible answer to a problem from a list of many feasible options. In mathematics, this is often visualized as a "cost landscape" full of hills and valleys. The goal is to find the lowest point in the lowest valley—the global minimum.

In classical computing, solving complex combinatorial optimization problems (like routing thousands of delivery trucks) is notoriously difficult. As you add more variables, the number of possible combinations explodes. Classical computers often have to settle for "good enough" solutions because checking every single path would take billions of years. Combinatorial optimization with quantum computing changes the game by using quantum properties like superposition and interference to "sculpt" the probability of finding the best path far more efficiently than a classical search.

How Quantum Algorithms Tackle Optimization Problems?

Quantum optimization algorithms don't just check every option one by one. Instead, they represent the entire problem as a physical system. By encoding the "cost" of a solution into the energy level of a quantum state, the computer can evolve toward the state with the lowest energy.

Two main phenomena give quantum optimization its edge:

Quantum Superposition: The ability to represent a massive number of potential solutions simultaneously.
Quantum Tunneling: In classical optimization, an algorithm might get stuck in a "local minimum"—a valley that looks like the bottom but isn't. To get out, a classical algorithm must "climb" back up the hill. A quantum algorithm can "tunnel" through the hill to find a deeper valley on the other side.

Techniques like quantum annealing or the Quantum Approximate Optimization Algorithm (QAOA) are used to find optimal solutions for complex problems, such as portfolio optimization or logistics.

• Quantum Generative Models:

This includes Quantum Boltzmann Machines (QBMs) and Quantum Generative Adversarial Networks (QGANs), which are designed to learn and sample from complex probability distributions more efficiently.

In recent years, the advent of quantum computing has spurred a wave of innovation in the field of artificial intelligence (AI). Quantum generative models (QGMs) have emerged as a vital component in this convergence of quantum computing and AI, presenting new opportunities and challenges. This article aims to provide a comprehensive understanding of QGMs, shedding light on their significance, functioning, applications, and impact on the AI landscape.

What are quantum generative models?

Quantum generative models refer to a class of algorithms that leverage the principles of quantum mechanics to generate complex data distributions. In the context of AI, these models

are designed to create and simulate large datasets in a manner that mirrors the characteristics of real-world data. By harnessing the unique properties of quantum systems, QGMs offer a novel approach to data generation, distinct from classical methods.

Significance of quantum generative models:

The advent of QGMs has marked a significant milestone in the AI landscape, offering unprecedented opportunities and capabilities. The impact of these models can be observed across various domains of AI, such as machine learning, data analysis, and optimization. Their significance lies in their ability to tackle complex data generation tasks that were previously considered challenging for classical generative models.

Characteristics and Features

QGMs operate on the principles of quantum mechanics, harnessing the phenomena of superposition and entanglement to process and manipulate data. Unlike classical techniques, these models embrace the non-deterministic nature of quantum systems, enabling them to explore vast solution spaces efficiently. As a result, QGMs exhibit enhanced parallelism and the potential to outperform classical generative models in certain scenarios.

Benefits and Advantages

Enhanced Data Fidelity: QGMs can generate datasets that closely resemble real-world data, offering improved fidelity and accuracy in simulations and modeling.

Exploration of Complex Distributions: These models excel at exploring and simulating complex high-dimensional data distributions, providing valuable insights into intricate systems.

Drawbacks and Limitations

Resource Intensiveness: The computational resources required for implementing QGMs can be considerably high, posing challenges for practical deployment in certain scenarios.

Algorithmic Complexity: The inherent complexity of quantum generative models can introduce challenges in terms of interpretability and customization, requiring specialized expertise for effective utilization.

• Quantum Reinforcement Learning (QRL):

In this case, the algorithms which make up the learning system are known as agents. The agents observe, select, and perform specific actions in return for rewards or penalties. An example of the application of reinforcement learning is how robots are taught to walk or perform tasks. They get “rewarded” for doing the correct work and “penalized” for getting them wrong. Another example is how sometimes people learn to play video games.

They try out new things, such as clicking places, opening doors, getting ammunition or money, and eventually learning how to play. Similarly, the agents in reinforcement learning algorithms try out new things as it continues to learn from data based observations and learns automatically. One of the areas of reinforcement learning that has become immensely popular is neural networks,

This explores how quantum agents can learn from interacting with an environment to achieve a speedup in decision-making or learning time.

Classical vs Quantum Reinforcement Learning

- Classical RL relies on classical computation and optimization techniques to learn optimal policies
- Quantum RL leverages quantum computing principles and algorithms to enhance RL performance and tackle complex problems
- Quantum RL can exploit quantum parallelism to efficiently explore large state and action spaces
- Quantum algorithms (amplitude amplification, quantum walks) can accelerate the learning process in RL
- Quantum RL can handle high-dimensional and continuous state and action spaces more effectively than classical RL
- Quantum RL has the potential to provide quantum speedups and improved sample efficiency compared to classical RL
- Quantum RL can be applied to quantum control problems where the environment is a quantum system

QRL Algorithms and Techniques

- Quantum Q-learning is an extension of classical Q-learning that uses quantum circuits to represent and update Q-values

  Quantum circuits encode Q-values in the amplitudes of quantum states. Quantum gates are applied to perform Q-value updates and action selection
- Quantum Value Iteration is a quantum version of the classical value iteration algorithm for solving MDPs

  Quantum circuits are used to represent value functions and perform Bellman updates
- Quantum Policy Gradient methods use quantum circuits to represent policies and estimate policy gradients for policy optimization
- Quantum Advantage Actor-Critic (QAAC) is a quantum-enhanced version of the actor-critic algorithm that uses quantum circuits for both policy and value function approximation
- Quantum Variational Circuits (QVCs) are parameterized quantum circuits used as function approximators in QRL.QVCs can represent complex non-linear functions and are optimized using classical optimization techniques
- Quantum Generative Adversarial Networks (QGANs) can be used for generating realistic quantum states and exploring complex environments in QR

Advantage in RL Problems

- Quantum RL has the potential to provide quantum speedups and improved performance in certain RL problems
- Quantum algorithms can efficiently search large state and action spaces enabling faster convergence and better exploration
- Quantum entanglement can capture complex correlations and dependencies in RL environments leading to more accurate value function approximation.
- Quantum RL can handle high-dimensional and continuous state and action spaces more effectively than classical RL
- Quantum algorithms can provide quadratic speedups in solving linear systems which are commonly encountered in RL (Bellman equations)
- Quantum RL can be particularly advantageous in quantum control problems where the environment is a quantum system
- Quantum RL has shown promising results in applications such as quantum error correction, quantum chemistry, and quantum communication

5.3 Quantum K means clustering

What is Clustering?

Cluster analysis is a technique in data mining and machine learning that groups similar objects into clusters. K-means clustering, a popular method, aims to divide a set of objects into K clusters, minimizing the sum of squared distances between the objects and their respective cluster centers.

Hierarchical clustering and k-means clustering are two popular techniques in the field of unsupervised learning used for clustering data points into distinct groups. While k-means clustering divides data into a predefined number of clusters, hierarchical clustering creates a hierarchical tree-like structure to represent the relationships between the clusters.

K means clustering

The main goal of any clustering algorithm is to assign data points to clusters that represent their similarity in an N-dimensional space. k-means clustering is an unsupervised learning algorithm. Unsupervised learning is learning without labels or targets. k-means clustering is an algorithm that produces data clusters. Clustering involves assigning cluster names to all similar data points. Usually, clusters are named 1, 2, 3, and so forth.

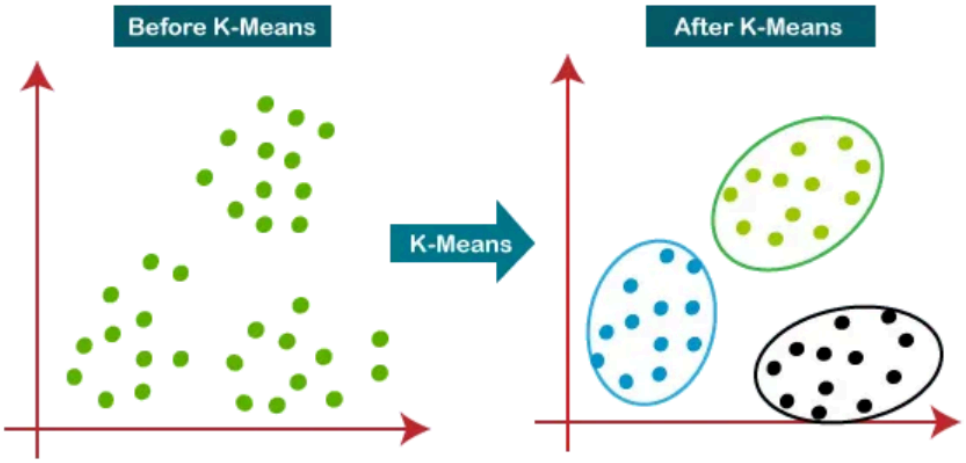
k-means clustering takes several centroids as an input where each centroid defines a cluster. When the algorithm starts, the centroids are placed in a random location in the data point vector space. k-means clustering has a phase called assign, and another phase called minimize. The algorithm cycles through these two phases several times. During the assign phase, the algorithm uses Euclidean distance calculation to place each data point next to the centroid nearest to it. During the minimize phase, the sum of the distances between the centroids and all the data points assigned to them is minimized by moving the centroids in a direction that brings them closer to the data points. This is called a cycle. The end of a cycle produces a ready hyperplane.

After one cycle ends, the next cycle starts by disassociating the data points from their centroids. Locationally, the centroids stay where they are. With that in place, a new association phase starts, which may decide to make assignments different from the one at the previous cycle. When a new data point is encountered, it is assigned to the closest centroid and inherits the name of that centroid as a label. A cluster in k-means is a region around a centroid.

In a theoretical setting, labels are not required in clustering. But the evaluation metrics become irrelevant if there are no labels to associate them. What will the metrics define if there is no element to define? Without labels, we cannot calculate true positives, false positives, true

negatives, and false negatives. A detailed explanation of the usage of classification evaluation metrics in clustering, called the external evaluation of clustering.

However, there are times when we simply do not have access to labels and are forced to work without them. In this case, we can use an internal evaluation of clustering. Among several evaluation metrics available, the Dunn evaluation metric is most popular. The focus of the Dunn coefficient is to measure the cluster density in an N-dimensional space. The quality of each cluster is evaluated by calculating the Dunn coefficient for each cluster.



The above k-means clustering example illustrates how this method assigns data points to the nearest centroid, refining the clusters iteratively. Understanding what is k-means clustering will enhance your grasp of data analysis and pattern recognition.

How K-Means Clustering Works?

Steps :

- Initialization: Start by randomly selecting K points from the dataset. These points will act as the initial cluster centroids.
- Assignment: For each data point in the dataset, calculate the distance between that point and each of the K centroids. Assign the data point to the cluster whose centroid is closest to it. This step effectively forms K clusters.
- Update centroids: Once all data points have been assigned to clusters, recalculate the centroids of the clusters by taking the mean of all data points assigned to each cluster.
- Repeat: Repeat steps 2 and 3 until convergence. Convergence occurs when the centroids no longer change significantly or when a specified number of iterations is reached.
- Final Result: Once convergence is achieved, the algorithm outputs the final cluster centroids and the assignment of each data point to a cluster.

Properties of K means Clustering

1. All the data points in a cluster should be similar to each other.
2. The data points from different clusters should be as different as possible. This will intuitively make sense if you've grasped the above property.

Quantum K means clustering

As k-means is an unsupervised clustering algorithm that groups together n observations into k clusters, making sure intracluster variance is minimized.

Given dataset D ∈ {x1, x2, …, xN} with N observations, the objective is to cluster this dataset into k partitions. We want to assign a set of points grouped near each other in one cluster and another set of points grouped into another cluster.

The k centroids are initialized randomly. The objective is to find sets S ∈ {s1, s2, …, sK} such that the intra-set variance is minimum and the inter-set variance is maximum.

The quantum version of the k-means algorithm is an unsupervised algorithm that aims to classify data to k clusters based on an unlabeled set of training vectors. The training vectors are reassigned to the nearest centroid in each iteration, and then a new centroid is calculated, averaging the vectors belonging to the current cluster.

It provides an exponential speed-up for very high dimensional input vectors utilizing the fact that only logN qubits are required to load N-dimensional input vectors using amplitude encoding. The time complexity O(NM) of the classical algorithm using Lloyd's version of the algorithm is linearly dependent on several N features and training examples M.

The most resource-consuming operation for a k-means algorithm is the calculation of a distance between vectors. We are expecting to gain speed-up by retrieving the distance using a quantum computer.

There are a few versions of the Quantum k-means algorithm. One version of the quantum k-means algorithm utilizes three different quantum subroutines to perform k-means clustering: Swap test, distance calculation (DistCalc), and Grover's optimization (GroverOptim).

As seen before, the swap test subroutine measures the overlap between two quantum states ⟨x| y⟩ based on the measurement probability of the control qubit in state |0⟩. The overlap is a measure of similarity between two states.

If the probability of the control qubit in state |0⟩ is 0.5, it means the |x⟩ and |y⟩ states are orthogonal; whereas if the probability is 1, then the states are identical. The subroutine should be repeated several times to obtain a good estimator of probability.

The advantage of using the swap test is that the states |x⟩ and |y⟩ can be unknown before

the procedure is performed. A simple measurement is required on the control qubit, which has two eigenstates. The time complexity is negligible as the procedure does not depend on the number of qubits representing the input states.

The probability of control qubit being in state |0⟩ is given by

$$p(0) = \left|\langle 0|\psi\rangle\right|^2 = \frac{1}{2}\left[1 - \left|\langle x|y\rangle\right|^2\right]$$

The distance calculation (DistCalc) subroutine follows the Swap Test and presents an algorithm to retrieve the Euclidean distance |x − y|2 between two real valued vectors x and y. The algorithm was described by Lloyd, Mohseni, and Rebentrost.

Let's discuss the representation of classical data into a quantum state. The classical information in vector x is encoded as

$$|x|^{-1} x \mapsto |x\rangle = \sum_{i=1}^{N} |x|^{-1} x_i |i\rangle$$

In the DistCalc subroutine, two states are prepared, then a subroutine swap test is applied, and the procedure is repeated to obtain an acceptable estimate of probability. Providing that the states are already prepared, the subroutine swap test does not depend on the size of a feature vector.

The preparation of states in subroutine DistCalc is proven to have O(logN) time complexity as the classical information is encoded in n = log2N qubits, and the time complexity is expected to be proportional. The classical algorithms require O(N) to calculate Euclidean distance between two vectors; hence, there is an exponential speed-up.

After the swap test and DistCalc algorithm, the last step is to run Grover's optimizer, which chooses the closest centroid of the cluster. This completes an overview of the quantum k-means algorithm.

5.4 Variational Quantum Circuits

We saw the inspiration drawn for quantum optimization from the Ising models, which relates to the physics of ferromagnetism. Typically, the Schrödinger equation can be used to study and simulate the behavior of and the evolution of a single quantum particle. However, real-world phenomena are dependent on the interactions between many different quantum systems. The study of many-body Hamiltonians that model physical systems are the central theme of condensed matter physics, molecular dynamics, and modeling. Variational methods are of primary importance in quantum machine learning practice and current research, and hence, we spend a bit of time trying to address the fundamentals of these ideas.

As it stands today, many-body Hamiltonians are inherently hard to model and investigate on classical computers due to the characteristic of the Hilbert space, whose dimension grows exponentially with the number of particles in the system. As such, finding the eigenvalues of these operations becomes infeasible for classical computers. This is where a quantum computer becomes very useful. It allows us to study these many-body systems with less overhead as the number of qubits required only grows polynomially. For a perfect quantum computer, the variational tasks should become bread-and-butter and part of daily routine.

However, as attractive as it may seem to use quantum computers for problems related to many-body physics and molecular simulations, current and near-term quantum computers are far from being perfect and suffer from imperfections. This is why it is still a challenge (currently subject of intense research) to run long algorithms that require deep circuits on near-term noisy quantum computers. The preparation of quantum states using low-depth quantum circuits is one of the most promising near-term applications of small quantum computers, especially if the circuit is short enough and the fidelity of gates are high enough to be executed without quantum error correction. Such quantum state preparation can be used in variational approaches, optimizing parameters in the circuit to minimize the energy of the constructed quantum state for a given problem Hamiltonian.

Since 2013, a breed of algorithms has been researched and developed, which focus on getting an advantage from imperfect quantum computers. The basic idea is to run a short sequence of gates where some gates are parametrized. The process then reads out the result, adjusts the parameters on a classical computer, and repeats the calculation with the new parameters on the quantum hardware. This way, an iterative loop is created between the quantum and the classical processing units, creating classical-quantum hybrid algorithms as shown in Figure below.

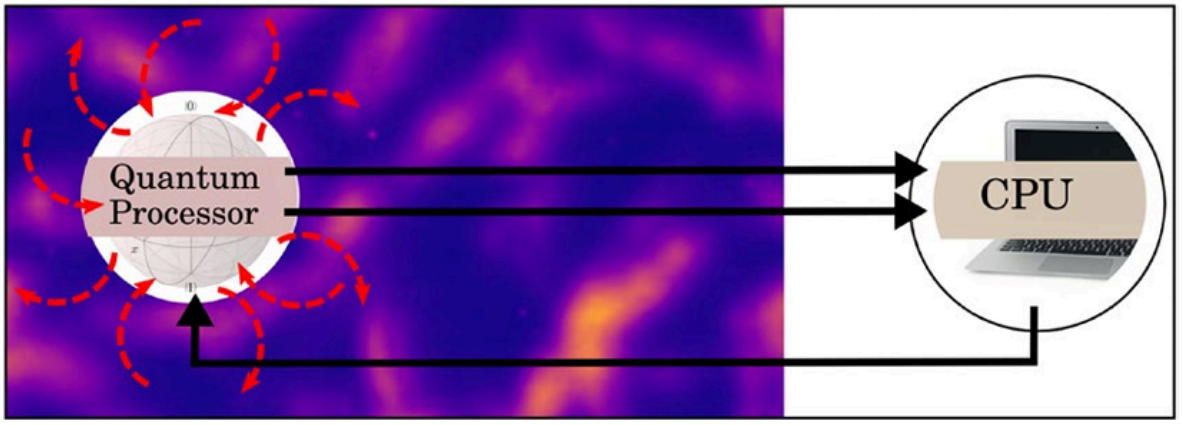


Figure. Variational circuits for hybrid quantum-classical systems

These algorithms are called variational to reflect the variational approach to changing the parameters.

Such quantum state preparation can be used in variational approaches, optimizing parameters in the circuit to minimize the energy of the constructed quantum state for a given problem Hamiltonian. For these reasons, variational circuits are also known as parameterized quantum circuits.

These variational algorithms, illustrated in below Figure, give us a close approximation of the problems. Though not 100% perfect, they can give us answers which are close to perfection. This is why they are so useful, especially the VQE algorithms.

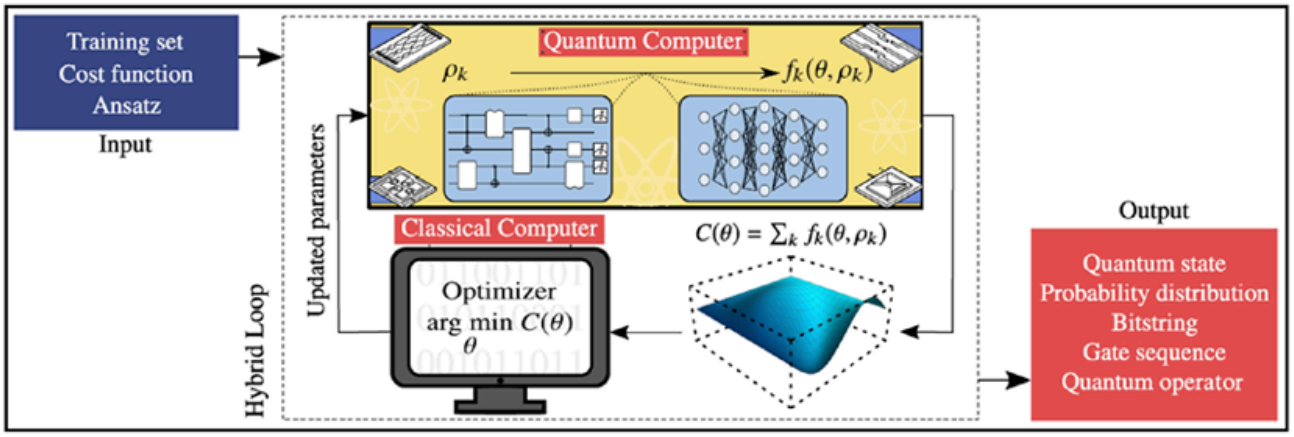


Figure. Diagram of a quantum variational algorithm

Above figure is a schematic diagram of a variational quantum algorithm (VQA). The first step of the process is to develop a VQA to define a cost (or loss) function C. The cost function C encodes the solution to the problem.

In the next step, an ansatz is proposed (i.e., a quantum operation depending on a set of continuous or discrete parameters θ that can be optimized. This ansatz is then trained with data from the training set in a hybrid quantum-classical loop to solve the following optimization task

$$\theta^* = \arg\min_{\theta} C(\theta)$$

The following are VQA inputs: a cost function C(θ) encoding the solution to the problem, an ansatz whose parameters are trained to minimize the cost, and optionally (if required), a set of training data used during the optimization. At each iteration of the loop, a quantum computer efficiently estimates the cost or its gradients. The output of this information is fed into a classical computer that leverages the power of optimizers to navigate the cost derivatives and solve the optimization problem in the above equation. Once a termination condition is met, the VQA estimates an output of the solution to the problem. The form of the output depends on the precise task. Above figure indicates some of the most common types of output.

The cost function, maps the values of the trainable parameters θ to real numbers. The cost defines a hypersurface, usually called the cost landscape (see the above Figure). The task of the optimizer is to navigate through the cost landscape and obtain the global minima. Without any loss of generality, the cost can be expressed as

$$C(\theta) = \sum_k f_k \left[\mathrm{Tr}\left(O_k U(\theta) \rho_k U^{\dagger}(\theta) \right) \right]$$

where f_k are functions that encode the task. Discrete and continuous parameters make up θ. $\{O_k\}$ defines a set of observables. U(θ) is a parametrized unitary. $\{\rho_k\}$ consists of input states (from a training set). For a given problem, f_k can differ, which may lead to useful costs.

As cost functions are important for VQAs, so is the choice of ansatz. The form of the ansatz defines the θ parameters and how they can be trained to minimize the cost. The specific structure of an ansatz generally depends on the task. In some instances, information on the problem can tailor an ansatz, which is a problem-inspired ansatz. Some other ansatz architectures are generic and independent of the problem, meaning they can be used even when no relevant information is readily available. For the cost function in the equation, the parameters θ can be encoded in a unitary U(θ) that is applied to the input states to the quantum circuit. As shown in the above figure, U(θ) can be generally expressed as the product of S sequentially applied unitaries.

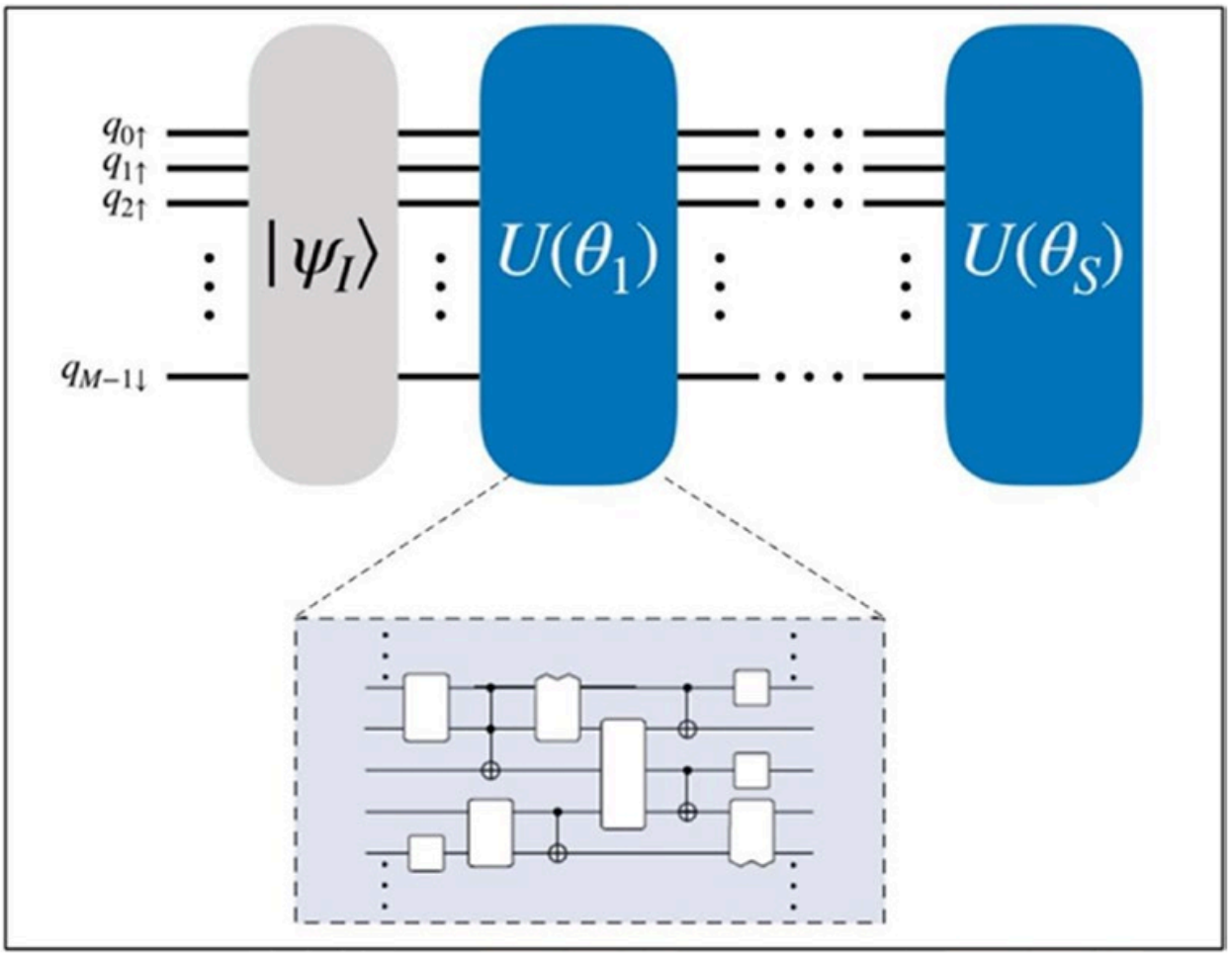


Figure. Schematic of an ansatz

$$U(\theta)=U_S(\theta_S)\ldots U_2(\theta_2)U_1(\theta_1)$$

with,

$$U_S(\theta_S)=\prod_n W_n e^{-i\theta_n \mathbb{H}_n}$$

where $\mathbb{H}n$ is a Hermitian operator, and Wn is an unparametrized unitary.

5.5 Quantum Support-Vector Machines (Supervised Learning)

Data classification is one of the most important machine learning tools as it can identify, group, and study new data pertaining to use cases as varied as computer vision problems, medical imaging, drug discovery, handwriting recognition, and geostatistics and many other fields.

In machine learning, one of the most common data classification methods is support-vector machines (SVM). SVM is a supervised learning method and particularly useful because it allows classification into one of two categories of an input set of training data by using a hyperplane to separate the two classes. The support vectors used along with the hyperplane are essentially data points used to maximize the distance between classes.

Quantum support-vector machines (qSVMs) use feature maps to map data points to quantum circuits. qSVMs are the subject of several studies both theoretically and experimentally. qSVMs use qubits instead of classical bits to solve problems. Many qSVMs and quantum-inspired SVM algorithms leverage the kernel trick in the quantum domain.

Generating a qSVM-based exercise requires the following.

1. Scaling, normalization, and principal component analysis
2. Generating a kernel matrix
3. Estimating the kernel for a new set of data points (test data) for QSVM classification

In the QSVM classification phase, classical SVM generates the separating hyperplane rather than a quantum circuit, and here the quantum computer is used twice. In the first instance, the kernel is estimated for all pairs of training data. The second time, the kernel is estimated for a new date or test data.

In a continuation from classical SVM we inspect the requirements to translate that knowledge into the quantum domain.

- To achieve this task, the first consideration is about how to translate the classical data point x into a quantum data point $|\psi(x)\rangle$. This can be achieved by creating a circuit $C(\psi(x))$ encompassing $\psi()$ where ψ is any classical function applied to the classical data.
- Next, a parametrized quantum circuit $V(\vartheta)$ is built with parameters ϑ to process the data.
- A measurement is performed that returns a classical value –1 or +1 for each classical input x that identifies the label of the classical data.

- Finally, we leverage quantum variational circuit (QVC) concepts and define an ansatz for the problem in the form $V(\vartheta)C(\psi(x))\,|0\rangle$.

At this moment, establishing use cases to state the advantage of qSVM over classical SVM is very much an area of active research. For example, the researchers at the CERN Large Hadron Collider (LHC) in Switzerland are actively working to see if the enormous computing challenge associated with the demands of their experiments can be met with the help of quantum computers. To this end, they have successfully employed qSVM for a ttH (H to two photons), Higgs coupling to top quarks analysis at LHC.

Many data analysis and machine learning techniques involve optimization. To this end, using D-Wave processors to solve combinatorial optimization problems through quantum annealing has been of deep interest for many years.

How to implement QSVM

Quantum Support Vector Machines (QSVMs) combine the principles of quantum computing with classical machine learning to perform classification tasks, particularly on complex or high-dimensional datasets. The following steps outline the standard workflow to use QSVM effectively:

1. Load and Preprocess the Dataset

First Step is to load and preprocess your dataset to ensure it is suitable for training. This involves steps like normalizing feature values, encoding class labels if necessary, and splitting the dataset into training and testing sets. Proper preprocessing ensures compatibility with quantum feature maps and improves model performance.

2. Quantum State Encoding

Quantum state encoding is a crucial step in QSVMs. Here, classical input data is encoded into quantum states using quantum circuits. Each data point is transformed into a quantum state using techniques such as angle encoding, amplitude encoding, or basis encoding, where the features of a data vector are translated into rotation angles of quantum gates (such as Rot or RY gates). This representation allows the quantum computer to exploit superposition and entanglement to explore complex data relationships.

3. Defining the Quantum Feature Map

Once the data is encoded into quantum states, a quantum feature map (or embedding circuit) is defined. This feature map determines how input data is mapped into a high-dimensional quantum Hilbert space. Commonly used maps include the ZZFeatureMap or PauliFeatureMap, which use parameterized quantum gates to introduce non-linearity and entanglement across qubits. Importantly, users are not limited to predefined feature maps—they can design custom quantum circuits tailored to the specific structure or properties of their data. This flexibility allows for optimization and fine-tuning of the feature representation, potentially improving model performance in specialized tasks.

4. Computing the Quantum Kernel

With the feature map in place, the quantum kernel is computed. This involves measuring the inner product (fidelity) between pairs of quantum states, representing the similarity between data points in quantum space. These values populate a kernel matrix, which is used by the classical SVM to find an optimal decision boundary in the transformed feature space.

5. Model Training

After computing the quantum kernel matrix using a chosen feature map, the QSVM is trained using this kernel. Typically, a classical Support Vector Machine (SVM) algorithm is used to perform the training, which involves solving a quadratic optimization problem to determine the optimal separating hyperplane. However, it is also possible to design and implement a custom quantum-enhanced classifier that goes beyond classical SVMs. This allows researchers to explore novel quantum machine learning approaches that fully utilize quantum hardware, making the model even more tailored to the problem. The strength of the QSVM approach lies in its flexibility—the quantum kernel captures complex relationships in the data, while the classification layer can be classical or quantum, depending on the use case and available resources.

6. Prediction and Classification

After training, the QSVM model can predict the class of unseen data. Each test point is encoded, and its similarity to training data is calculated using the same quantum feature map and kernel. The classical SVM then uses these similarities to make predictions based on the learned decision boundary.

7. Evaluation and Iteration

The model's performance is evaluated using metrics such as accuracy, precision, recall, and F1-score. Users can tune hyperparameters (like kernel choice, regularization strength, and circuit depth), modify the quantum feature map, or use different encoding techniques to enhance performance.